

Tugas Besar 1 IF3270 Pembelajaran Mesin
Feed Forward Neural Network



Oleh:

Kelompok 27 – CapCipCup

Jonathan Emmanuel Saragih (13522121)

Jason Fernando (13522156)

Bryan P. Hutagalung (18222130)

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika – Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

2025

Daftar Isi

Daftar Isi.....	1
Bab I Deskripsi Tugas.....	4
1.1 Spesifikasi Utama.....	4
1.2 Bonus.....	11
Bab II Landasan Teori.....	13
2.1 Feed Forward Neural Network.....	13
2.2 Fungsi Aktivasi.....	13
2.3 Fungsi Loss.....	15
2.4 Inisialisasi Bobot.....	16
2.5 Forward Propagation dan Backward Propagation.....	17
2.6 Optimasi dan Pembaruan Bobot.....	17
Bab III Metodologi Implementasi.....	18
3.1 Class.....	18
3.1.1 activations.....	18
3.1.2 layers.....	25
3.1.3 loss.....	34
3.1.4 neural_network.....	39
3.1.5 weight_initializers.....	54
3.1.6 metrics.....	59
3.1.7 trainer.....	70
3.1.8 visualization.....	78
3.2 Forward Propagation.....	89
3.3 Backward Propagation and Update Weight.....	90
Bab IV Hasil dan Pembahasan.....	92
4.1 Layers.....	92
4.1.1 Depth 1 Width 1.....	92
4.1.2 Depth 1 Width 2.....	93

4.1.3 Depth 1 Width 3.....	94
4.1.4 Depth 2 Width 1.....	95
4.1.5 Depth 2 Width 2.....	96
4.1.6 Depth 2 Width 3.....	97
4.1.7 Depth 3 Width 1.....	98
4.1.8 Depth 3 Width 2.....	99
4.1.9 Depth 3 Width 3.....	100
4.1.10 Comparison Hidden Layer Depth and Width.....	102
4.2 Hidden Layer Activations Function.....	103
4.2.1 Linear.....	103
4.2.2 ReLU.....	104
4.2.3 Sigmoid.....	106
4.2.4 TanH.....	107
4.2.5 Softmax.....	108
4.2.6 Softplus.....	110
4.2.7 ELU.....	111
4.2.8 Comparison Hidden Layer Activation Function.....	112
4.3 Output Layer Activation Function.....	113
4.3.1 Sigmoid.....	113
4.3.2 Softmax.....	114
4.3.3 Comparison Output Layer Activation Function.....	115
4.4 Learning Rate.....	116
4.4.1 0.01.....	116
4.4.2 0.001.....	117
4.4.3 0.0001.....	118
4.4.4 Comparison Learning Rate.....	119
4.5 Loss Function.....	120
4.5.1 MSE.....	120
4.5.2 BCE.....	121

4.5.3 CCE.....	122
4.5.4 Comparison Loss Function.....	123
4.6 Weight Initialization.....	124
4.6.1 Zero.....	124
4.6.2 Random Uniform.....	125
4.6.3 Random Normal.....	126
4.6.4 Xavier.....	127
4.6.5 He.....	128
4.6.6 Comparison Weight Initialization.....	129
4.7 Regularization.....	130
4.7.1 None.....	130
4.7.2 L1.....	131
4.7.3 L2.....	132
4.7.4 Comparison Regularization.....	133
4.8 Normalization.....	135
4.8.1 None.....	135
4.8.2 RMS.....	136
4.8.3 Comparison Normalization.....	137
4.9 CapCipCup Scratch vs MLP Library.....	138
4.9.1 CapCipCup Scratch.....	138
4.9.2 MLP Library.....	138
4.9.3 Comparison CapCipCup Scratch vs MLP Library.....	139
Bab V Penutup.....	140
5.1 Kesimpulan.....	140
5.2 Saran.....	142
Lampiran.....	144
Daftar Pustaka.....	145

Bab I

Deskripsi Tugas

1.1 Spesifikasi Utama

Implementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat **menerima jumlah neuron dari tiap layer** (termasuk input layer dan output layer)
- FFNN yang diimplementasikan dapat **menerima fungsi aktivasi dari tiap layer**. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Loss	Definisi Fungsi
<u>MSE</u>	$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
<u>Binary Cross-Entropy</u>	$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n (y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i))$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
<u>Categorical Cross-Entropy</u>	$\mathcal{L}_{CCE} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C (y_{ij} \log \hat{y}_{ij})$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

- Catatan:
 - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2
 - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e)

- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
 - **Zero initialization**
 - Random dengan distribusi **uniform**.
 - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal)
 - Menerima parameter seed untuk reproducibility
 - Random dengan distribusi **normal**.
 - Menerima parameter mean dan variance
 - Menerima parameter seed untuk reproducibility
- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias)
- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias)
- Instance model memiliki method untuk **menampilkan model** berupa **struktur jaringan** beserta **bobot** dan **gradien bobot** tiap neuron dalam bentuk graf. (Format graf dibebaskan)
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
 - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot
- Instance model memiliki method untuk **save** dan **load**

- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:
 - Dapat menerima input berupa **batch**.
- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
 - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
 - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
 - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d(Linear(x))}{dx} = 1$
ReLU	$\frac{d(ReLU(x))}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d(\sigma(x))}{dx} = \sigma(x)(1 - \sigma(x))$
Hyperbolic Tangent (tanh)	$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}} \right)^2$

Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $\frac{d(\text{softmax}(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_1} & \dots & \frac{\partial(\text{softmax}(\vec{x})_n)}{\partial x_n} \end{bmatrix}$ Dimana untuk $i, j \in \{1, \dots, n\}$, $\frac{\partial(\text{softmax}(\vec{x})_i)}{\partial x_j} = \text{softmax}(\vec{x})_i (\delta_{i,j} - \text{softmax}(\vec{x})_j)$ $\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$
---------	---

- Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
<u>MSE</u>	$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n} \sum_{i=1}^n (y_i - \hat{y}_i) \frac{\partial \hat{y}_i}{\partial W}$
<u>Binary Cross-Entropy</u>	$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} \frac{\partial \hat{y}_i}{\partial W}$
<u>Categorical Cross-Entropy</u>	$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^c \frac{y_{ij}}{\hat{y}_{ij}} \frac{\partial \hat{y}_{ij}}{\partial W}$

- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

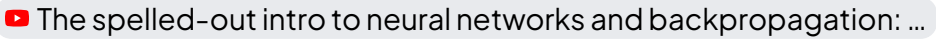
α = Learning rate

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
 - Dapat menerima parameter berikut:
 - Batch size
 - Learning rate
 - Jumlah epoch
 - Verbose
 - Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan
 - Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu
 - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss tiap epoch**.
- Lakukan **pengujian** terhadap implementasi FFNN dengan ketentuan sebagai berikut:
 - Analisis pengaruh beberapa hyperparameter sebagai berikut:
 - Pengaruh **depth (banyak layer)** dan **width (banyak neuron per layer)**
 - Pilih 3 variasi kombinasi width (depth tetap) dan 3 variasi depth (width semua layer tetap)
 - Bandingkan hasil akhir prediksinya
 - Bandingkan grafik loss pelatihnnya
 - Pengaruh **fungsi aktivasi** hidden layer
 - Lakukan untuk setiap fungsi aktivasi yang diimplementasikan **kecuali softmax**.

- Bandingkan hasil akhir prediksinya
- Bandingkan grafik loss pelatihannya
- Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model
- Pengaruh **learning rate**
 - Lakukan 3 variasi learning rate (nilainya dibebaskan)
 - Bandingkan hasil akhir prediksinya
 - Bandingkan grafik loss pelatihannya
 - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model
- Pengaruh **inisialisasi bobot**
 - Lakukan untuk setiap metode inisialisasi bobot yang diimplementasikan
 - Bandingkan hasil akhir prediksinya
 - Bandingkan grafik loss pelatihannya
 - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model
- Analisis perbandingan hasil prediksi dengan library sklearn MLP
 - Lakukan satu kali pelatihan dengan hyperparameter yang sama untuk kedua model
 - Hyperparameter yang digunakan dibebaskan
 - Bandingkan hasil akhir prediksinya saja
- Gunakan dataset berikut untuk menguji model: mnist_784
 - Gunakan method fetch_openml dari sklearn untuk memuat dataset
 - Berikut contoh untuk memuat dataset: Contoh
- Note: Tetap akan ada test case untuk penilaian, akan dilakukan saat asisten memeriksa tugas.
- Pengujian dilakukan di file .ipynb terpisah

1.2 Bonus

Berikut merupakan beberapa spesifikasi bonus yang dapat Anda kerjakan:

- Implementasikan FFNN dengan menggunakan **automatic differentiation**.
 - Metode ini merupakan metode yang umum digunakan untuk perhitungan gradien pada library-library untuk deep learning seperti PyTorch atau TensorFlow.
 - Jika ingin mengimplementasikan bonus ini, sangat disarankan untuk mengimplementasikan dari sejak awal mengerjakan tugas supaya tidak terlalu banyak perubahan dari implementasi biasa.
 - Referensi video:

- Implementasikan minimal 2 fungsi aktivasi lain yang sering digunakan.
- Implementasikan 2 metode inisialisasi bobot berikut:
 - Xavier initialization
 - He initialization
- Implementasikan metode regularisasi L1 dan L2, kemudian lakukan analisis untuk beberapa hal berikut:
 - Lakukan eksperimen masing-masing 1 kali untuk model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.
 - Bandingkan hasil akhir prediksinya
 - Bandingkan grafik loss pelatihannya
 - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model
- Implementasikan metode normalisasi RMSNorm, kemudian lakukan analisis untuk beberapa hal berikut:
 - Lakukan eksperimen masing-masing 1 kali untuk model tanpa normalisasi dan dengan normalisasi.
 - Bandingkan hasil akhir prediksinya
 - Bandingkan grafik loss pelatihannya

- Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model

Bab II

Landasan Teori

2.1 Feed Forward Neural Network

Feedforward Neural Network (FFNN) adalah tipe jaringan saraf yang paling dasar, yang terdiri dari tiga lapisan utama: lapisan input, lapisan tersembunyi (hidden layer), dan lapisan output. Pada FFNN, data mengalir dari lapisan input ke lapisan output tanpa adanya umpan balik atau loop, berbeda dengan jaringan saraf berulang (RNN). Setiap lapisan dalam jaringan melakukan proses perhitungan linear yang kemudian diteruskan dengan penerapan fungsi aktivasi non-linear. FFNN banyak digunakan dalam berbagai aplikasi, terutama untuk masalah klasifikasi dan regresi.

Setiap lapisan dalam FFNN terdiri dari neuron-neuron yang saling terhubung dengan neuron di lapisan sebelumnya melalui bobot. Setiap neuron juga dilengkapi dengan bias yang ditambahkan setelah operasi linear. Hasil dari operasi linear ini kemudian diproses menggunakan fungsi aktivasi yang memberikan elemen non-linearitas, memungkinkan jaringan untuk menangkap hubungan yang kompleks dalam data.

2.2 Fungsi Aktivasi

Fungsi aktivasi memainkan peran penting dalam jaringan saraf karena menentukan bagaimana output dari sebuah neuron dihitung berdasarkan input yang diterimanya. Fungsi ini memberi kemampuan bagi jaringan saraf untuk mempelajari dan memodelkan hubungan non-linear dalam data. Berikut adalah beberapa fungsi aktivasi yang umum digunakan dalam FFNN:

1. Linear Activation

Fungsi aktivasi linear hanya mengembalikan nilai input tanpa perubahan.

Fungsi ini biasanya digunakan pada lapisan output untuk masalah regresi, di mana output model tidak memerlukan non-linearitas.

2. ReLU (Rectified Linear Unit)

ReLU adalah fungsi aktivasi yang paling banyak digunakan dalam beberapa tahun terakhir. Fungsi ini mengembalikan 0 jika input kurang dari 0, dan mengembalikan nilai input itu sendiri jika lebih besar dari 0. ReLU membantu mengatasi masalah *vanishing gradients* yang sering terjadi pada jaringan saraf yang dalam.

3. Sigmoid

Fungsi sigmoid memetakan input ke rentang antara 0 dan 1. Fungsi ini sering digunakan dalam masalah klasifikasi biner karena menghasilkan output probabilitas.

4. Tanh (Hyperbolic Tangent)

Fungsi tanh adalah varian dari sigmoid, tetapi lebih terpusat pada nilai -1 hingga 1. Fungsi ini sering digunakan karena dapat menangani nilai negatif dengan lebih baik.

5. Softmax

Softmax digunakan pada lapisan output untuk masalah klasifikasi multi-kelas. Fungsi ini mengubah output setiap neuron menjadi distribusi probabilitas, dengan jumlah output dari seluruh neuron pada lapisan output menjadi 1.

6. Softplus

Fungsi aktivasi Softplus merupakan versi yang lebih halus dari ReLU, yang dihitung menggunakan rumus $f(x) = \ln(1 + e^x)$. Fungsi ini memberikan transisi yang lebih mulus, memastikan output selalu positif, namun dengan perubahan yang lebih lembut dibandingkan dengan ReLU. Softplus sering dipilih ketika diperlukan transisi yang lebih stabil di sekitar nilai nol, dan dapat membantu mengatasi masalah gradien besar yang sering muncul pada ReLU.

7. ELU (Exponential Linear Unit)

ELU adalah fungsi aktivasi yang memberikan nilai negatif pada input yang lebih kecil atau sama dengan nol. Fungsi ini didefinisikan sebagai $f(x) = x$ jika $x > 0$, dan $f(x) = \alpha * (e^x - 1)$ jika $x \leq 0$, dengan α sebagai parameter yang dapat disesuaikan. Keunggulan utama ELU adalah kemampuannya mengurangi masalah *dying ReLU*, dengan memberikan gradien yang lebih besar pada input negatif, yang membantu mempercepat proses pembelajaran dan memberikan representasi yang lebih kaya pada jaringan saraf.

2.3 Fungsi Loss

Fungsi loss digunakan untuk mengukur sejauh mana kesalahan yang dibuat oleh model dalam setiap prediksi. Dalam pelatihan jaringan saraf, tujuan utamanya adalah meminimalkan nilai loss, sehingga model menjadi lebih akurat dalam memprediksi hasil. Berikut adalah beberapa fungsi loss yang sering digunakan:

1. Mean Squared Error (MSE)

MSE sering digunakan dalam masalah regresi. Fungsi ini dihitung dengan mencari rata-rata kuadrat perbedaan antara nilai prediksi dan nilai yang sebenarnya.

2. Binary Cross-Entropy

Digunakan dalam masalah klasifikasi biner, yaitu ketika model memiliki dua kelas output. Binary Cross-Entropy mengukur seberapa baik model memprediksi probabilitas untuk setiap kelas menggunakan logaritma natural.

3. Categorical Cross-Entropy

Fungsi loss ini digunakan untuk masalah klasifikasi multi-kelas. Categorical Cross-Entropy menghitung perbedaan antara distribusi probabilitas yang diprediksi oleh model dan distribusi kelas yang benar. Fungsi ini menghitung log-loss untuk setiap kelas dan menjumlahkan hasilnya.

2.4 Inisialisasi Bobot

Inisialisasi bobot adalah langkah awal yang sangat penting dalam melatih jaringan saraf. Pemilihan metode inisialisasi bobot yang tepat dapat mempengaruhi kecepatan konvergensi dan kinerja model. Beberapa metode inisialisasi yang umum digunakan adalah:

1. Zero Initialization

Inisialisasi bobot dengan nol adalah metode yang paling sederhana. Namun, ini sering tidak efektif karena dapat menyebabkan hilangnya gradien selama *backpropagation*, yang memperlambat pelatihan.

2. Random Uniform Initialization

Bobot diinisialisasi dengan nilai acak yang diambil dari distribusi seragam dalam rentang tertentu. Ini memastikan bobot memiliki variasi nilai, yang menghindari masalah gradien yang hilang.

3. Random Normal Initialization

Bobot diinisialisasi dengan distribusi normal yang memiliki rata-rata dan standar deviasi yang ditentukan. Metode ini sering digunakan untuk lapisan dengan fungsi aktivasi seperti ReLU.

4. Xavier Initialization

Xavier atau Glorot initialization digunakan untuk fungsi aktivasi yang simetris seperti *tanh*. Metode ini mengatur bobot sehingga variansnya tetap stabil selama proses propagasi maju dan mundur, yang mempercepat pelatihan.

5. He Initialization

He initialization dirancang khusus untuk ReLU. Metode ini mengatur standar deviasi distribusi normal berdasarkan jumlah neuron pada lapisan sebelumnya, yang membantu mengurangi masalah gradien yang hilang di lapisan-lapisan dalam jaringan.

2.5 Forward Propagation dan Backward Propagation

Forward propagation adalah proses di mana input dimasukkan ke jaringan saraf, dihitung melalui serangkaian lapisan, dan menghasilkan output prediksi. Proses ini melibatkan perhitungan nilai linear untuk setiap lapisan (produk dari input dan bobot, ditambah bias), yang kemudian diproses oleh fungsi aktivasi untuk menghasilkan output. Output dari setiap lapisan digunakan sebagai input untuk lapisan berikutnya.

Backward propagation adalah proses untuk menghitung gradien fungsi loss dengan menggunakan aturan rantai dan diteruskan kembali melalui jaringan untuk menghitung perubahan pada bobot. Gradien dihitung dengan mempertimbangkan turunan dari fungsi aktivasi dan fungsi loss. Bobot kemudian diperbarui dengan menggunakan algoritma seperti *gradient descent*, yang mengurangi bobot berdasarkan gradien yang dihitung.

2.6 Optimasi dan Pembaruan Bobot

Pembaruan bobot dalam FFNN dilakukan menggunakan algoritma *gradient descent*. Algoritma ini memperbarui bobot dengan cara mengurangnya berdasarkan gradien yang dihitung selama *backpropagation*. Pada setiap iterasi, bobot diperbarui dengan mengurangi nilai bobot yang dihitung dari gradien yang dikalikan dengan *learning rate*. Proses ini terus diulang selama beberapa epoch hingga model mencapai konvergensi.

Bab III

Metodologi Implementasi

3.1 Class

Bagian ini akan menjelaskan mengenai kelas apa saja yang ada pada implementasi tugas ini, serta atribut dan metode apa saja yang digunakan.

3.1.1 activations

```
class Activation:
    """Base class for activation functions"""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def backward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError
```

Kelas Activation adalah kelas dasar yang digunakan untuk semua jenis fungsi aktivasi dalam jaringan saraf. Kelas ini mendefinisikan dua metode penting yang harus diimplementasikan oleh setiap kelas turunannya: `forward()` dan `backward()`. Metode `forward()` bertugas menghitung keluaran dari fungsi aktivasi berdasarkan input yang diberikan, sementara `backward()` digunakan untuk menghitung gradien atau turunan yang diperlukan selama proses backpropagation dalam pembelajaran jaringan saraf.

1. Linear

```
class Linear(Activation):
    """Linear activation function:  $f(x) = x$ """

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        return inputs
```

```
def backward(self, inputs: np.ndarray) -> np.ndarray:
    # Derivative: 1
    return np.ones_like(inputs)
```

Kelas Linear mengimplementasikan fungsi aktivasi linear, yang pada dasarnya hanya mengembalikan nilai input tanpa perubahan ($f(x) = x$). Dalam kelas ini terdapat dua metode utama:

a. forward()

Mengembalikan nilai input tanpa ada modifikasi.

b. backward()

Mengembalikan array berisi nilai 1 untuk setiap elemen input, karena turunan dari fungsi linear selalu 1.

2. ReLU

```
class ReLU(Activation):
    """Rectified Linear Unit activation function:  $f(x) = \max(0, x)$ """

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        self.output = np.maximum(0, inputs)
        return self.output

    def backward(self, inputs: np.ndarray) -> np.ndarray:
        # Derivative: 1 if  $x > 0$  else 0
        return np.where(inputs > 0, 1, 0)
```

Kelas ReLU (Rectified Linear Unit) mengimplementasikan fungsi aktivasi yang mengubah nilai negatif menjadi 0, sementara nilai positif tetap dipertahankan. Metode-metode yang ada dalam kelas ini adalah:

a. forward()

Menggunakan `np.maximum(0, inputs)` untuk mengubah semua nilai negatif menjadi 0 dan mempertahankan nilai positif.

b. backward()

Mengembalikan gradien berdasarkan aturan ReLU, yaitu 1 untuk input yang lebih besar dari 0 dan 0 untuk input yang lebih kecil atau sama dengan 0, menggunakan np.where.

3. Sigmoid

```
class Sigmoid(Activation):
    """Sigmoid activation function:  $f(x) = 1 / (1 + e^{-x})$ """

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        self.inputs = inputs
        inputs = np.clip(inputs, -500, 500)
        self.output = 1.0 / (1.0 + np.exp(-inputs))
        return self.output

    def backward(self, inputs: np.ndarray) -> np.ndarray:
        # Derivative: sigmoid(x) * (1 - sigmoid(x))
        sigmoid_output = self.forward(inputs)
        return sigmoid_output * (1 - sigmoid_output)
```

Kelas Sigmoid mengimplementasikan fungsi aktivasi sigmoid yang menghasilkan nilai output dalam rentang 0 hingga 1. Kelas ini terdiri dari dua metode:

a. forward()

Menghitung keluaran menggunakan np.tanh(inputs), yang menghitung fungsi tanh dari input.

b. backward()

Menghitung turunan dari fungsi tanh, yaitu $1 - \tanh^2(x)$, dan mengembalikan gradien berdasarkan hasil perhitungan tanh.

4. Tanh

```
class Tanh(Activation):
```

```

        """Hyperbolic Tangent activation function:  $f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ """

        def forward(self, inputs: np.ndarray) -> np.ndarray:
            self.output = np.tanh(inputs)
            return self.output

        def backward(self, inputs: np.ndarray) -> np.ndarray:
            # Derivative:  $1 - \tanh^2(x)$ 
            tanh_output = np.tanh(inputs)
            return 1 - tanh_output ** 2

```

Kelas Tanh mengimplementasikan fungsi aktivasi hyperbolic tangent yang menghasilkan output dalam rentang antara -1 hingga 1. Kelas ini mencakup dua metode:

a. forward()

Menghitung keluaran menggunakan `np.tanh(inputs)`, yang menghitung fungsi tanh dari input.

b. backward()

Menghitung turunan dari fungsi tanh, yaitu $1 - \tanh^2(x)$, dan mengembalikan gradien berdasarkan hasil perhitungan tanh.

5. Softmax

```

class Softmax(Activation):
    """Softmax activation function."""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        shifted_inputs = inputs - np.max(inputs, axis=1,
keepdims=True)
        exp_values = np.exp(shifted_inputs)
        sum_exp = np.sum(exp_values, axis=1, keepdims=True)
        self.output = exp_values / sum_exp
        self.output = np.nan_to_num(self.output, nan=1e-8,

```

```

posinf=1.0, neginf=0.0)

        return self.output

    def backward(self, gradient: np.ndarray) -> np.ndarray:
        # Backward pass for softmax.
        return gradient

```

Kelas Softmax mengimplementasikan fungsi aktivasi softmax yang digunakan untuk mengubah input menjadi distribusi probabilitas. Fungsi ini memiliki dua metode utama:

a. forward()

Menghitung keluaran dengan pertama-tama menggeser nilai input (untuk stabilitas numerik), lalu menghitung eksponensial dari input yang sudah digeser, dan membaginya dengan jumlah total eksponensial untuk setiap contoh dalam batch. Dengan cara ini, keluaran akan menjadi distribusi probabilitas yang jumlahnya 1.

b. backward()

Pada umumnya, untuk fungsi softmax, metode backward() hanya mengembalikan gradien yang diterima tanpa perubahan besar, karena fungsi softmax biasanya digabungkan dengan fungsi loss (seperti cross-entropy), yang akan menghitung gradien lebih lanjut.

6. Softplus

```

class Softplus(Activation):
    """Softplus activation function:  $f(x) = \ln(1 + e^x)$ """

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        self.inputs = inputs
        return np.log1p(np.exp(-np.abs(inputs))) +

```

```

np.maximum(inputs, 0)

def backward(self, inputs: np.ndarray) -> np.ndarray:
    # Derivative: sigmoid
    return 1 / (1 + np.exp(-inputs))

```

Kelas Softplus mengimplementasikan fungsi aktivasi softplus, yang merupakan bentuk halus dari ReLU. Fungsi ini menggunakan rumus $f(x) = \ln(1 + e^x)$. Kelas ini memiliki dua metode utama:

a. forward()

Menghitung keluaran dengan menggabungkan fungsi logaritma dari nilai input dan fungsi maksimum, menggunakan $\text{np.log1p}(\text{np.exp}(-\text{np.abs}(\text{inputs}))) + \text{np.maximum}(\text{inputs}, 0)$.

b. backward()

Menghitung turunan dari fungsi softplus yang serupa dengan fungsi sigmoid, yaitu $1 / (1 + e^{-x})$, yang digunakan untuk menghitung gradien berdasarkan input yang diberikan.

7. ELU

```

class ELU(Activation):
    """Exponential Linear Unit activation function:  $f(x) = x$  if  $x > 0$  else  $\alpha * (e^x - 1)$ """

    def __init__(self, alpha: float = 1.0):
        self.alpha = alpha

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        self.inputs = inputs
        result = inputs.copy()
        mask = inputs <= 0
        safe_neg_inputs = np.clip(inputs[mask], -30.0, 0)
        result[mask] = self.alpha * (np.exp(safe_neg_inputs) - 1)

```

```

        return result

    def backward(self, inputs: np.ndarray) -> np.ndarray:
        # Derivative: 1 if x > 0 else  $\alpha * e^x$ 
        result = np.ones_like(self.inputs)
        mask = self.inputs <= 0
        safe_neg_inputs = np.clip(self.inputs[mask], -30.0,
0)

        result[mask] = self.alpha * np.exp(safe_neg_inputs)

        return result

```

Kelas ELU (Exponential Linear Unit) mengimplementasikan fungsi aktivasi ELU yang mirip dengan ReLU, namun memberikan output yang lebih halus untuk input negatif. Fungsi ini memiliki parameter α yang mengontrol bentuk kurva untuk nilai input negatif. Kelas ini memiliki dua metode utama:

a. forward()

Menghitung output dengan memberikan nilai input positif secara langsung, dan untuk input negatif, dihitung dengan rumus $\alpha * (e^x - 1)$, yang membuat output untuk nilai negatif menjadi lebih halus daripada ReLU.

b. backward()

Menghitung turunan dari fungsi ELU, yang bernilai 1 untuk input positif dan $\alpha * e^x$ untuk input negatif, kemudian mengembalikan gradien berdasarkan hasil ini.

3.1.2 layers

```

class Layer:
    """Base layer interface"""

```

```

def forward(self, inputs: np.ndarray) -> np.ndarray:
    raise NotImplementedError

def backward(self, inputs: np.ndarray, gradient: np.ndarray)
-> np.ndarray:
    raise NotImplementedError

def update_weights(self, learning_rate: float) -> None:
    raise NotImplementedError

```

Kelas **Layer** merupakan kelas dasar yang menyediakan antarmuka untuk lapisan-lapisan dalam jaringan saraf. Kelas ini menetapkan tiga metode utama yang harus diimplementasikan oleh setiap kelas turunan, yaitu:

a. forward()

Metode ini digunakan untuk menjalankan proses *forward pass* pada lapisan, yang menerima input dan menghasilkan output. Di kelas dasar ini, metode ini bersifat abstrak dan harus diimplementasikan oleh kelas turunan.

b. backward()

Metode ini menerima gradien dari lapisan berikutnya dan menghitung gradien yang diperlukan untuk lapisan ini serta lapisan sebelumnya selama proses *backward pass*.

c. update_weights()

Digunakan untuk memperbarui bobot lapisan berdasarkan gradien yang dihitung saat *backward pass*, dengan menggunakan pembaruan yang mengacu pada *learning rate*.

1. DenseLayer

```

class DenseLayer(Layer):
    """Standard fully-connected layer"""

```

```

    def __init__(self, input_size: int, output_size: int,
activation: Activation):
        self.input_size = input_size
        self.output_size = output_size
        self.activation = activation
        self.weights = None
        self.bias = None
        self.weights_gradient = None
        self.bias_gradient = None
        self.output = None
        self.z = None

    def initialize_weights(self, method: str, **params):
        """Set initial weights using specified method"""
        # Get initialization parameters
        seed = params.get('seed', None)
        init_params = {**params, 'input_size':
self.input_size, 'output_size': self.output_size}

        # Initialize weights
        self.weights = initialize_weights(
            method=method,
            shape=(self.input_size, self.output_size),
            **init_params
        )

        # Initialize bias
        if method in ['zero', 'xavier', 'he']:
            self.bias = np.zeros((1, self.output_size))
        elif method == 'random_uniform':
            lower = params.get('lower', -0.1)
            upper = params.get('upper', 0.1)
            rng = np.random.default_rng(seed)
            self.bias = rng.uniform(lower, upper, size=(1,
self.output_size)).astype(np.float32)

```

```

        elif method == 'random_normal':
            mean = params.get('mean', 0.0)
            std = params.get('std', 0.1)
            rng = np.random.default_rng(seed)
            self.bias = rng.normal(mean, std, size=(1,
self.output_size)).astype(np.float32)
        else:
            raise ValueError(f"Unknown weight
initialization method: {method}")

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        """Forward pass computation"""
        if self.weights is None or self.bias is None:
            raise ValueError("Weights and bias must be
initialized before forward pass")

        # Linear transformation
        self.z = np.dot(inputs, self.weights) + self.bias

        # Apply activation function
        self.output = self.activation.forward(self.z)

        return self.output

    def backward(self, previous_layer_activations:
np.ndarray, gradient: np.ndarray) -> np.ndarray:
        """Backward pass computation"""
        # Bias gradient
        self.bias_gradient = np.sum(gradient, axis=0,
keepdims=True)

        # Weight gradient
        self.weights_gradient =
np.dot(previous_layer_activations.T, gradient)

```



```

        # L1 regularization
        if hasattr(self, 'l1_lambda') and self.l1_lambda >
0:
            l1_grad_scale = self.l1_lambda /
self.weights.size
            epsilon = 1e-8
            l1_grad = l1_grad_scale * self.weights /
(np.abs(self.weights) + epsilon)
            self.weights_gradient += l1_grad

        # L2 regularization
        if hasattr(self, 'l2_lambda') and self.l2_lambda >
0:
            l2_grad_scale = self.l2_lambda /
self.weights.size
            self.weights_gradient += l2_grad_scale *
self.weights

        # Gradient for previous layer
        return np.dot(gradient, self.weights.T)

    def update_weights(self, learning_rate: float) -> None:
        """Update weights using gradients"""
        if self.weights_gradient is None or
self.bias_gradient is None:
            raise ValueError("Gradients must be computed
before updating weights")

        # Apply weight updates
        self.weights -= learning_rate *
self.weights_gradient
        self.bias -= learning_rate * self.bias_gradient

```

Kelas **DenseLayer** mengimplementasikan lapisan *fully-connected* (dense), yang merupakan jenis lapisan yang umum dalam jaringan saraf. Lapisan ini memiliki beberapa atribut penting:

a. Input_size

Menyimpan ukuran input yang diterima oleh lapisan ini.

b. Output_size

Menyimpan ukuran output yang dihasilkan oleh lapisan.

c. Activation

Menyimpan objek fungsi aktivasi yang digunakan dalam lapisan ini.

d. weights dan bias

Menyimpan bobot dan bias yang digunakan dalam lapisan.

e. weights_gradient dan bias_gradient

Menyimpan gradien dari bobot dan bias yang dihitung selama *backward pass*.

f. output dan z

Menyimpan hasil output lapisan dan nilai *z* (hasil dari transformasi linear) yang dihitung dalam *forward pass*.

Metode-metode yang ada dalam kelas ini meliputi:

a. initialize_weights()

Metode ini digunakan untuk menginisialisasi bobot dan bias lapisan menggunakan berbagai metode inisialisasi, seperti zero, Xavier, He, atau distribusi acak. Parameter yang diteruskan mengatur cara inisialisasi bobot.

b. forward()

Menghitung output lapisan dengan terlebih dahulu melakukan transformasi linear pada input (perkalian matriks antara input dan bobot, ditambah bias), kemudian menerapkan fungsi aktivasi pada hasil transformasi tersebut.

c. **backward()**

Menghitung gradien untuk bobot, bias, dan input sebelumnya selama *backward pass*. Juga menghitung regularisasi L1 dan L2 jika diterapkan. Gradien untuk lapisan sebelumnya dihitung dengan mengalikan gradien yang diterima dengan transpos dari bobot.

d. **update_weights()**

Memperbarui bobot dan bias lapisan menggunakan gradien yang dihitung selama *backward pass*, dengan mengaplikasikan pembaruan berdasarkan *learning rate* yang diberikan.

2. **RMSNormalizationLayer**

```
class RMSNormalizationLayer(Layer):
    """RMS Normalization Layer"""

    def __init__(self, input_size: int, epsilon: float = 1e-8):
        self.input_size = input_size
        self.output_size = input_size
        self.epsilon = epsilon
        self.gamma = np.ones((1, input_size))
        self.x = None
        self.rms = None
        self.x_normalized = None
        self.weights = self.gamma
        self.bias = np.zeros((1, input_size))
        self.weights_gradient = None
        self.bias_gradient = None
        self.output = None
        self.activation = Linear()

    def forward(self, inputs: np.ndarray, training: bool =
```

```

True) -> np.ndarray:
    """Apply RMS normalization"""
    # Store input
    self.x = inputs

    # Calculate root mean square
    ms = np.mean(np.square(inputs), axis=1,
keepdims=True)
    self.rms = np.sqrt(ms + self.epsilon)
    self.rms = np.maximum(self.rms, 1e-6) # Avoid
division by zero

    # Normalize input
    self.x_normalized = inputs / self.rms

    # Scale with gamma parameter
    self.output = self.gamma * self.x_normalized

    return self.output

    def backward(self, previous_layer_activations:
np.ndarray, gradient: np.ndarray) -> np.ndarray:
        """Compute gradient for RMS normalization"""
        # Handle NaN values
        gradient = np.nan_to_num(gradient, nan=0.0,
posinf=0.0, neginf=0.0)

        # Gradient for gamma
        self.weights_gradient = np.sum(gradient *
self.x_normalized, axis=0, keepdims=True)
        self.bias_gradient = np.zeros_like(self.bias)

        # Gradient for normalized input
        d_normalized = gradient * self.gamma

```

```

        # Gradient for RMS
        d_rms = -np.sum(d_normalized * self.x_normalized,
axis=1, keepdims=True) / self.rms
        d_ms = d_rms * 0.5 / self.rms

        # Gradient for input
        d_x = d_normalized / self.rms
        d_x += 2.0 * self.x * d_ms / self.input_size

        return d_x

    def update_weights(self, learning_rate: float) -> None:
        """Update gamma parameter"""
        if self.weights_gradient is not None:
            self.gamma -= learning_rate *
self.weights_gradient

    def initialize_weights(self, method: str, **params):
        """Dummy method for API consistency"""
        pass

```

Kelas **RMSNormalizationLayer** mengimplementasikan lapisan **RMS Normalization**, yang berfungsi untuk menormalkan input berdasarkan akar kuadrat dari rata-rata kuadrat nilai input. Lapisan ini memiliki atribut-atribut berikut:

a. input_size dan output_size

Menyimpan ukuran input dan output lapisan, yang biasanya sama dalam lapisan normalisasi ini.

b. Epsilon

Nilai kecil yang digunakan untuk mencegah pembagian dengan nol saat menghitung akar kuadrat dari rata-rata kuadrat.

c. Gamma

Parameter skala yang digunakan untuk menyesuaikan hasil normalisasi.

d. x, rms, x_normalized

Menyimpan input asli, nilai RMS yang dihitung, dan hasil normalisasi input.

e. Weights

Sama dengan gamma, digunakan untuk skala.

f. Bias

Bias lapisan ini, yang umumnya bernilai 0 dalam normalisasi RMS.

g. weights_gradient dan bias_gradient

Menyimpan gradien dari bobot dan bias yang dihitung selama *backward pass*.

h. Output

Menyimpan hasil output lapisan setelah normalisasi dan skala.

Metode-metode yang ada dalam kelas ini meliputi:

a. initialize_weights()

Metode ini adalah metode dummy yang diimplementasikan untuk konsistensi API, karena pada lapisan RMS Normalization, tidak diperlukan inisialisasi bobot selain parameter gamma.

b. forward()

Menghitung normalisasi RMS dari input, dengan cara pertama-tama menghitung akar kuadrat dari rata-rata kuadrat input, lalu menormalkan input dengan membaginya dengan nilai RMS yang dihitung. Setelah itu, hasil normalisasi dikalikan dengan parameter gamma untuk melakukan skala. Hasil akhirnya adalah output yang dinormalisasi dan diskalakan.

c. **backward()**

Menghitung gradien untuk lapisan ini selama backward pass. Gradien dihitung untuk parameter gamma, serta untuk input yang sudah dinormalisasi. Selain itu, gradien untuk RMS dihitung dan digunakan untuk menghitung gradien terhadap input.

d. **update_weights()**

Memperbarui parameter gamma berdasarkan gradien yang dihitung selama backward pass, dengan mengaplikasikan pembaruan berdasarkan learning rate yang diberikan.

3.1.3 loss

```
class Layer:
    """Base layer interface"""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def backward(self, inputs: np.ndarray, gradient: np.ndarray)
-> np.ndarray:
        raise NotImplementedError

    def update_weights(self, learning_rate: float) -> None:
        raise NotImplementedError
```

Kelas Loss adalah kelas dasar yang menyediakan struktur untuk berbagai fungsi loss dalam jaringan saraf. Kelas ini mendefinisikan dua metode yang wajib diimplementasikan oleh kelas turunannya, yaitu:

a. **loss()**

Metode ini digunakan untuk menghitung nilai loss atau kerugian antara hasil prediksi dan nilai target yang sebenarnya (true labels). Fungsi ini

mengukur seberapa jauh hasil prediksi model dari nilai yang sebenarnya.

b. gradient()

Metode ini digunakan untuk menghitung gradien dari loss terhadap prediksi, yang sangat berguna untuk optimisasi dan pembaruan model selama proses pembelajaran.

1. MeanSquaredError

```
class Layer:
    """Base layer interface"""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def backward(self, inputs: np.ndarray, gradient:
np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def update_weights(self, learning_rate: float) -> None:
        raise NotImplementedError
```

Kelas **MeanSquaredError** (MSE) mengimplementasikan fungsi loss Mean Squared Error, yang sering digunakan dalam masalah regresi untuk mengukur seberapa jauh prediksi model dari nilai yang sebenarnya. MSE dihitung dengan rumus yang memberikan nilai rata-rata selisih kuadrat antara nilai yang sebenarnya (y_{true}) dan prediksi model (y_{pred}).

a. loss()

Metode ini menghitung MSE dengan cara mencari selisih kuadrat antara nilai sebenarnya dan prediksi, kemudian menghitung rata-rata dari hasil tersebut.

b. **gradient()**

Metode ini menghitung gradien dari MSE terhadap prediksi. Gradien ini dihitung dengan rumus yang gradien ini digunakan untuk memperbarui parameter model selama proses pelatihan

2. **BinaryCrossEntropy**

```
class Layer:
    """Base layer interface"""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def backward(self, inputs: np.ndarray, gradient:
np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def update_weights(self, learning_rate: float) -> None:
        raise NotImplementedError
```

Kelas **BinaryCrossEntropy** mengimplementasikan fungsi loss Binary Cross-Entropy, yang sering digunakan dalam masalah klasifikasi biner (dua kelas). Fungsi ini mengukur seberapa baik model memprediksi probabilitas untuk dua kelas yang mungkin. Binary Cross-Entropy dihitung dengan rumus untuk menjaga kestabilan numerik, nilai prediksi **y_pred** dibatasi (clip) antara epsilon dan 1 - epsilon untuk menghindari masalah log(0).

a. **init()**

Konstruktor kelas ini menginisialisasi parameter epsilon, yang digunakan untuk mencegah masalah log(0) dengan membatasi nilai prediksi yang terlalu kecil atau besar.

b. **loss()**

Metode ini menghitung Binary Cross-Entropy loss dengan cara

mengklip nilai prediksi y_{pred} terlebih dahulu, lalu menghitung log-loss berdasarkan rumus BCE untuk setiap sampel dan menghitung rata-rata dari seluruh loss.

c. gradient()

Metode ini menghitung gradien dari BCE loss, yaitu turunan dari BCE terhadap y_{pred} . Gradien ini dihitung dengan rumus yang digunakan dalam pembaruan parameter model selama pelatihan.

3. CategoricalCrossEntropy

```
class Layer:
    """Base layer interface"""

    def forward(self, inputs: np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def backward(self, inputs: np.ndarray, gradient:
np.ndarray) -> np.ndarray:
        raise NotImplementedError

    def update_weights(self, learning_rate: float) -> None:
        raise NotImplementedError
```

Kelas **CategoricalCrossEntropy** mengimplementasikan fungsi loss Categorical Cross-Entropy, yang digunakan dalam masalah klasifikasi multi-kelas (lebih dari dua kelas). Categorical Cross-Entropy menghitung seberapa baik model memprediksi distribusi probabilitas untuk setiap kelas dalam masalah klasifikasi. Untuk menjaga kestabilan numerik, nilai prediksi y_{pred} juga dibatasi antara epsilon dan $1 - \text{epsilon}$. Kelas ini mendukung penggunaan label kelas dalam bentuk indeks (class indices) atau one-hot encoding.

a. init()

Konstruktor kelas ini menginisialisasi parameter epsilon untuk memastikan kestabilan saat melakukan operasi logaritma. Nilai prediksi kemudian dibatasi untuk menghindari $\log(0)$.

b. loss()

Metode ini menghitung Categorical Cross-Entropy loss dengan cara mengklip nilai prediksi y_{pred} terlebih dahulu. Kemudian, ia menghitung log-loss berdasarkan rumus CCE untuk one-hot encoded labels atau class indices. Jika **y_{true}** berupa indeks kelas, rumus ini menghitung log dari prediksi untuk kelas yang benar. Jika **y_{true}** berupa one-hot encoded labels, rumus ini menghitung log untuk setiap kelas dan menjumlahkan hasilnya.

c. gradient()

Metode ini menghitung gradien dari Categorical Cross-Entropy loss dengan cara menghitung selisih antara nilai prediksi dan target ($y_{\text{true_one_hot}}$). Gradien ini dihitung dengan rumus yang digunakan untuk memperbarui parameter model selama proses pelatihan.

3.1.4 neural_network

```
class NeuralNetwork:
    def __init__(self, layer_sizes, activation_functions,
weight_initialization,
                    use_batch_norm=False, dropout_rates=None,
seed=None, **init_params):
    # Validate inputs
    if len(layer_sizes) < 2:
        raise ValueError("At least input and output layers
are required")
    if len(activation_functions) != len(layer_sizes) - 1:
```

```

        raise ValueError("Number of activation functions must
match number of layers - 1")

    # Store model configuration
    self.layer_sizes = layer_sizes
    self.activation_functions = activation_functions
    self.use_batch_norm = use_batch_norm
    self.dropout_rates = dropout_rates if dropout_rates else
[0.0] * (len(layer_sizes) - 1)
    self.seed = seed

    # Adam optimizer parameters
    self.betal = 0.9
    self.beta2 = 0.999
    self.epsilon = 1e-8
    self.m = {}
    self.v = {}
    self.t = 0

    # Check dropout rates
    if len(self.dropout_rates) != len(layer_sizes) - 1:
        raise ValueError("Number of dropout rates must match
number of layers - 1")

    # Build network
    self.layers = self._create_layers(layer_sizes,
activation_functions)
    self._initialize_weights(weight_initialization,
seed=seed, **init_params)

    def _create_layers(self, layer_sizes, activation_functions):
        """Create network layers"""
        layers = []

        for i in range(len(layer_sizes) - 1):

```

```

        # Handle string activation names
        if isinstance(activation_functions[i], str):
            act_name = activation_functions[i].lower()
            if act_name == 'linear':
                activation = Linear()
            elif act_name == 'relu':
                activation = ReLU()
            elif act_name == 'sigmoid':
                activation = Sigmoid()
            elif act_name == 'tanh':
                activation = Tanh()
            elif act_name == 'softmax':
                activation = Softmax()
            elif act_name == 'softplus':
                activation = Softplus()
            elif act_name == 'elu':
                activation = ELU()
            else:
                raise ValueError(f"Unknown activation
function: {activation_functions[i]}")
        else:
            activation = activation_functions[i]

        # Create layer
        dense_layer = DenseLayer(
            input_size=layer_sizes[i],
            output_size=layer_sizes[i+1],
            activation=activation
        )
        layers.append(dense_layer)

    return layers

def _initialize_weights(self, method, **params):
    """Initialize weights for all layers"""

```

```

        for layer in self.layers:
            layer.initialize_weights(method, **params)

    def forward_propagation(self, X, training=True):
        """Perform forward pass through network"""
        # Handle invalid input values
        if np.any(np.isnan(X)) or np.any(np.isinf(X)):
            X = np.nan_to_num(X, nan=0.0, posinf=0.0, neginf=0.0)

        # Forward through layers
        A = X
        for layer in self.layers:
            if hasattr(layer, 'training_mode'):
                A = layer.forward(A, training=training)
            else:
                A = layer.forward(A)

        # Handle numerical instability
        if np.any(np.isnan(A)) or np.any(np.isinf(A)):
            A = np.nan_to_num(A, nan=0.0, posinf=0.0,
neginf=0.0)

        return A

    def backward_propagation(self, X, y, gradient):
        """Perform backward pass through network"""
        # Handle invalid gradient values
        if np.any(np.isnan(gradient)) or
np.any(np.isinf(gradient)):
            gradient = np.nan_to_num(gradient, nan=0.0,
posinf=0.0, neginf=0.0)

        # Backward through layers
        dA = gradient
        for i in range(len(self.layers) - 1, -1, -1):

```

```

        if i == 0:
            dA = self.layers[i].backward(X, dA)
        else:
            dA =
self.layers[i].backward(self.layers[i-1].output, dA)

def update_weights(self, learning_rate):
    """Update weights using Adam optimizer"""
    # Increment timestep
    self.t += 1

    # Initialize moment estimates if needed
    if not self.m:
        for i, layer in enumerate(self.layers):
            if hasattr(layer, 'weights') and layer.weights is
not None:
                self.m[f'w{i}'] =
np.zeros_like(layer.weights)
                self.m[f'b{i}'] = np.zeros_like(layer.bias)
                self.v[f'w{i}'] =
np.zeros_like(layer.weights)
                self.v[f'b{i}'] = np.zeros_like(layer.bias)

    # Update weights with Adam
    for i, layer in enumerate(self.layers):
        if hasattr(layer, 'weights') and
layer.weights_gradient is not None:
            # Get gradients
            dw = layer.weights_gradient
            db = layer.bias_gradient

            # Update momentum
            self.m[f'w{i}'] = self.betal * self.m[f'w{i}'] +
(1 - self.betal) * dw
            self.m[f'b{i}'] = self.betal * self.m[f'b{i}'] +

```

```

(1 - self.betal) * db

        # Update RMSprop
        self.v[f'w{i}'] = self.beta2 * self.v[f'w{i}'] +
(1 - self.beta2) * (dw**2)
        self.v[f'b{i}'] = self.beta2 * self.v[f'b{i}'] +
(1 - self.beta2) * (db**2)

        # Bias correction
        m_hat_w = self.m[f'w{i}'] / (1 -
self.betal**self.t)
        m_hat_b = self.m[f'b{i}'] / (1 -
self.betal**self.t)
        v_hat_w = self.v[f'w{i}'] / (1 -
self.beta2**self.t)
        v_hat_b = self.v[f'b{i}'] / (1 -
self.beta2**self.t)

        # Update parameters
        layer.weights -= learning_rate * m_hat_w /
(np.sqrt(v_hat_w) + self.epsilon)
        layer.bias -= learning_rate * m_hat_b /
(np.sqrt(v_hat_b) + self.epsilon)

    def apply_gradient_clipping(self, clip_value=5.0):
        """Clip gradients to prevent exploding gradients"""
        # Calculate gradient norm
        total_norm = 0
        for layer in self.layers:
            if hasattr(layer, 'weights_gradient') and
layer.weights_gradient is not None:
                total_norm +=
np.sum(np.square(layer.weights_gradient))
            if hasattr(layer, 'bias_gradient') and
layer.bias_gradient is not None:

```



```

        total_norm +=
np.sum(np.square(layer.bias_gradient))

    total_norm = np.sqrt(total_norm)

    # Apply clipping if needed
    if total_norm > clip_value:
        scale = clip_value / (total_norm + 1e-7)
        for layer in self.layers:
            if hasattr(layer, 'weights_gradient') and
layer.weights_gradient is not None:
                layer.weights_gradient *= scale
            if hasattr(layer, 'bias_gradient') and
layer.bias_gradient is not None:
                layer.bias_gradient *= scale

    def train_batch(self, X_batch, y_batch, learning_rate,
loss_function):
        """Train on a single batch"""
        try:
            # Forward pass
            y_pred = self.forward_propagation(X_batch,
training=True)

            # Handle NaN predictions
            if np.any(np.isnan(y_pred)):
                y_pred = np.nan_to_num(y_pred, nan=1e-8)

            # Calculate loss
            loss_value = loss_function.loss(y_batch, y_pred)

            # Add regularization
            reg_loss = 0
            total_weights = 0

```

```

        for layer in self.layers:
            if hasattr(layer, 'weights'):
                layer_weights = layer.weights.size
                total_weights += layer_weights

                if hasattr(layer, 'l1_lambda') and
layer.l1_lambda > 0:
                    reg_loss += layer.l1_lambda *
np.sum(np.abs(layer.weights))
                    if hasattr(layer, 'l2_lambda') and
layer.l2_lambda > 0:
                        reg_loss += 0.5 * layer.l2_lambda *
np.sum(np.square(layer.weights))

            # Normalize regularization loss
            if total_weights > 0:
                reg_loss = reg_loss / total_weights

            loss_value += reg_loss

            # Calculate gradient
            loss_gradient = loss_function.gradient(y_batch,
y_pred)

            # Handle NaN gradient
            if np.any(np.isnan(loss_gradient)):
                loss_gradient = np.nan_to_num(loss_gradient,
nan=0.0)

            # Backward pass
            self.backward_propagation(X_batch, y_batch,
loss_gradient)

            # Clip gradients
            self.apply_gradient_clipping(clip_value=1.0)

```

```

        # Update weights
        self.update_weights(learning_rate)

        return loss_value if not np.isnan(loss_value) else
0.0

    except Exception as e:
        print(f"Error in train_batch: {e}")
        return 0.0

    def train(self, X_train, y_train, learning_rate,
loss_function,
              X_val=None, y_val=None, batch_size=32, epochs=10,
verbose=1):
        """Train the model"""
        # Initialize history
        history = {
            'train_loss': [],
            'val_loss': []
        }

        # Calculate batches
        n_samples = X_train.shape[0]
        n_batches = int(np.ceil(n_samples / batch_size))

        # Training loop
        for epoch in range(epochs):
            epoch_loss = 0

            # Shuffle data
            indices = np.random.permutation(n_samples)
            X_shuffled = X_train[indices]
            y_shuffled = y_train[indices]

            # Batch training

```

```

        for batch in range(n_batches):
            start_idx = batch * batch_size
            end_idx = min((batch + 1) * batch_size,
n_samples)

            X_batch = X_shuffled[start_idx:end_idx]
            y_batch = y_shuffled[start_idx:end_idx]

            batch_loss = self.train_batch(X_batch, y_batch,
learning_rate, loss_function)
            epoch_loss += batch_loss

            # Show progress
            if verbose == 1:
                progress = (batch + 1) / n_batches
                progress_bar = '#' * int(progress * 20) + '-'
* (20 - int(progress * 20))
                print(f"\rEpoch {epoch+1}/{epochs}
[{progress_bar}] {progress*100:.1f}% - batch_loss:
{batch_loss:.4f}", end="")

            # Average loss
            epoch_loss /= n_batches
            history['train_loss'].append(epoch_loss)

            # Validation
            val_loss = None
            if X_val is not None and y_val is not None:
                y_val_pred = self.forward_propagation(X_val,
training=False)
                val_loss = loss_function.loss(y_val, y_val_pred)
                history['val_loss'].append(val_loss)

            # Print epoch results
            if verbose == 1:
                if val_loss is not None:

```

```

        print(f"\rEpoch {epoch+1}/{epochs} -
train_loss: {epoch_loss:.4f} - val_loss: {val_loss:.4f}")
    else:
        print(f"\rEpoch {epoch+1}/{epochs} -
train_loss: {epoch_loss:.4f}")

    return history

def predict(self, X):
    """Make predictions"""
    return self.forward_propagation(X, training=False)

def display_model(self):
    """Print model summary"""
    print("Neural Network Structure:")
    print("-----")
    print(f"Total layers: {len(self.layers) + 1} (including
input layer)")
    print(f"Layer sizes: {self.layer_sizes}")
    print()

    print("Layer Details:")
    print("-----")
    for i, layer in enumerate(self.layers):
        print(f"Layer {i+1}: {self.layer_sizes[i]} →
{self.layer_sizes[i+1]}")
        print(f"  Activation:
{layer.activation.__class__.__name__}")
        print(f"  Parameters: {layer.weights.size +
layer.bias.size}")
        print()

    total_params = sum(layer.weights.size + layer.bias.size
for layer in self.layers)
    print(f"Total parameters: {total_params}")

```

```

def save(self, filename):
    """Save model to file"""
    # Create directory if needed
    os.makedirs(os.path.dirname(filename) if
os.path.dirname(filename) else '.', exist_ok=True)

    # Prepare model data
    model_data = {
        'layer_sizes': self.layer_sizes,
        'layers': self.layers,
        'activation_functions': [layer.activation for layer
in self.layers]
    }

    # Save to file
    with open(filename, 'wb') as f:
        pickle.dump(model_data, f)

    print(f"Model saved to {filename}")

@classmethod
def load(cls, filename):
    """Load model from file"""
    with open(filename, 'rb') as f:
        model_data = pickle.load(f)

    # Create model instance
    nn = cls(
        layer_sizes=model_data['layer_sizes'],
        activation_functions=[None] *
(len(model_data['layer_sizes']) - 1),
        weight_initialization='zero'
    )

```

```

        # Restore layers
        nn.layers = model_data['layers']

        print(f"Model loaded from {filename}")
        return nn

    def summary(self):
        """Print a summary of the neural network architecture"""
        total_params = 0
        trainable_params = 0

        print("\nNeural Network Summary:")
        print("=" * 80)
        print(f"{'Layer (type)':<30}{'Output Shape':<20}{'Param  
#':<15}{'Trainable':<10}")
        print("=" * 80)

        # Input layer
        if hasattr(self, 'input_size'):
            print(f"{'Input':<30}{'(None,  
self.input_size)!s:<20}{'0':<15}{'N/A':<10}")

        # Hidden layers and output layer
        for i, layer in enumerate(self.layers):
            layer_name = f"{type(layer).__name__} ({i})"

            # Get output shape
            output_shape = "Unknown"
            if hasattr(layer, 'output_size'):
                output_shape = (None, layer.output_size)

            # Calculate number of parameters
            params = 0
            trainable = "Yes"

```

```

        if hasattr(layer, 'weights') and layer.weights is not
None:
            weights_params = layer.weights.size
            bias_params = layer.bias.size if hasattr(layer,
'bias') and layer.bias is not None else 0
            params = weights_params + bias_params
            trainable_params += params
        else:
            trainable = "No"

        total_params += params

        # Print layer info

print(f"{layer_name:<30}{output_shape!s:<20}{params:<15}{trainabl
e:<10}")

        print("=" * 80)
        print(f"Total params: {total_params:,}")
        print(f"Trainable params: {trainable_params:,}")
        print(f"Non-trainable params: {total_params -
trainable_params:,}")
        print("=" * 80)

        return None

```

Kelas NeuralNetwork adalah implementasi dari jaringan saraf penuh (*fully connected neural network*) yang menyediakan berbagai fitur untuk melatih dan mengevaluasi model. Kelas ini memungkinkan pengguna untuk membangun, melatih, dan menyimpan model jaringan saraf dengan berbagai jenis lapisan dan fungsi aktivasi. Pada saat inisialisasi, pengguna perlu memberikan beberapa parameter penting, seperti `layer_sizes` (daftar yang menunjukkan ukuran masing-masing lapisan dalam jaringan), `activation_functions` (daftar fungsi aktivasi yang digunakan pada setiap

lapisan, kecuali lapisan output), dan `weight_initialization` (metode untuk menginisialisasi bobot). Selain itu, terdapat opsi untuk menggunakan batch normalization (`use_batch_norm`), dropout (`dropout_rates`), serta pengaturan acak (`seed`). Selama pelatihan, kelas ini mendukung penggunaan optimizer Adam untuk pembaruan bobot dan menyediakan opsi regularisasi L1 dan L2 untuk mengurangi risiko overfitting.

1. Atribut-Atribut

a. Layer_sizes

Menyimpan ukuran setiap lapisan dalam jaringan saraf.

b. Activation_functions

Menyimpan fungsi aktivasi untuk setiap lapisan jaringan (kecuali lapisan output).

c. use_batch_norm dan dropout_rates

Parameter opsional yang memengaruhi bagaimana lapisan diproses selama pelatihan, dengan batch normalization dan dropout digunakan untuk memperbaiki stabilitas dan mencegah overfitting.

d. m, v, dan t

Merupakan parameter yang digunakan oleh algoritma Adam untuk pembaruan bobot, dengan `m` mewakili estimasi momentum, `v` untuk estimasi kuadrat gradien, dan `t` untuk waktu langkah (iterasi) yang digunakan dalam perbaikan bias untuk momentum dan RMSprop.

2. Metode-Metode

a. init()

Digunakan untuk memvalidasi input, menyimpan parameter model, serta membangun dan menginisialisasi lapisan-lapisan dalam jaringan.

b. `_create_layers()`

Bertanggung jawab untuk membuat setiap lapisan jaringan berdasarkan ukuran dan fungsi aktivasi yang ditentukan.

c. `_initialize_weights()`

Menginisialisasi bobot dan bias setiap lapisan dengan metode yang telah ditentukan sebelumnya.

d. `forward_propagation()`

Mengimplementasikan proses *forward pass* untuk menghitung output dari jaringan berdasarkan input yang diberikan.

e. `backward_propagation()`

Menghitung gradien selama *backward pass*, yang akan digunakan untuk pembaruan bobot.

f. `update_weights()` Menangani pembaruan bobot menggunakan algoritma Adam, yang menggabungkan prinsip momentum dan RMSprop untuk meningkatkan konvergensi pelatihan.

g. `apply_gradient_clipping()`

Mencegah masalah *gradient explosion* dengan memotong gradien jika normanya terlalu besar, menjaga kestabilan pelatihan.

h. `train_batch()`

Melatih model pada satu batch data, menghitung nilai loss, melakukan *backward pass*, dan memperbarui bobot.

i. `train()`

Loop pelatihan utama yang melatih model untuk beberapa epoch, memproses data dalam batch, serta mengevaluasi kinerja model pada data pelatihan dan validasi jika tersedia.

j. predict()

Digunakan untuk menghasilkan prediksi berdasarkan input yang diberikan.

k. display_model()

Menyajikan ringkasan struktur model dengan mencetak informasi tentang setiap lapisan, termasuk ukuran input dan output serta jumlah parameter yang dapat dilatih.

l. save()

Menyimpan model ke dalam file menggunakan pickle, memungkinkan penyimpanan model yang sudah dilatih untuk digunakan kembali.

m. load()

Memungkinkan pemulihan model yang telah disimpan sebelumnya dari file.

n. summary()

Menyediakan ringkasan lengkap tentang arsitektur model, termasuk jumlah total parameter, parameter yang dapat dilatih, dan detail lebih lanjut tentang setiap lapisan.

3.1.5 weight_initializers

1. zero_init

```
def zero_init(shape: Tuple[int, ...], **kwargs):  
    """Initialize weights with zeros"""  
    return np.zeros(shape, dtype=np.float32)
```

Fungsi ini menginisialisasi bobot dengan nilai nol. Meskipun metode ini mudah diterapkan, sering kali tidak efektif untuk jaringan saraf. Menggunakan nol untuk bobot dapat menyebabkan masalah dalam proses pembelajaran, seperti gradien yang menjadi nol selama *backpropagation*, yang menghambat proses pembelajaran. Fungsi ini

menerima parameter shape untuk menentukan dimensi bobot yang akan diinisialisasi dan mengembalikan array yang berisi nilai nol di seluruh elemen.

2. `random_uniform_init`

```
def random_uniform_init(shape: Tuple[int, ...], lower=-0.1,
                        upper=0.1, seed=None, **kwargs):
    """Initialize weights using uniform distribution"""
    rng = np.random.default_rng(seed)
    return rng.uniform(lower, upper,
                       size=shape).astype(np.float32)
```

Fungsi ini menginisialisasi bobot dengan distribusi seragam. Nilai bobot dihasilkan secara acak dalam rentang tertentu yang bisa diatur melalui parameter `lower` dan `upper`. Parameter `seed` memungkinkan pengaturan nilai acak yang dapat direproduksi, sementara `shape` menentukan ukuran array bobot. Metode ini banyak digunakan pada model dengan lapisan yang tidak terlalu dalam.

3. `random_normal_init`

```
def random_normal_init(shape: Tuple[int, ...], mean=0.0,
                       std=0.1, seed=None, **kwargs):
    """Initialize weights using normal distribution"""
    rng = np.random.default_rng(seed)
    return rng.normal(mean, std,
                     size=shape).astype(np.float32)
```

Fungsi ini menginisialisasi bobot menggunakan distribusi normal (Gaussian), dengan nilai bobot yang dihasilkan berdasarkan rata-rata (`mean`) dan standar deviasi (`std`) yang bisa disesuaikan. Dengan parameter `seed`, kita bisa mengontrol acakan secara konsisten. Ini adalah salah satu metode yang paling sering digunakan

dalam jaringan saraf untuk memberikan variasi pada bobot di awal pelatihan.

4. xavier_init

```
def xavier_init(shape: Tuple[int, ...], input_size,
output_size, seed=None, **kwargs):
    """Xavier/Glorot initialization"""
    rng = np.random.default_rng(seed)
    limit = np.sqrt(6 / (input_size + output_size))
    return rng.uniform(-limit, limit,
size=shape).astype(np.float32)
```

Fungsi ini mengimplementasikan inisialisasi Xavier (atau Glorot), yang dirancang untuk menjaga kestabilan varians antara lapisan-lapisan dalam jaringan selama pelatihan. Bobot diinisialisasi dalam rentang yang ditentukan oleh rumus:

5. he_init

```
def he_init(shape: Tuple[int, ...], input_size, seed=None,
**kwargs):
    """He initialization"""
    rng = np.random.default_rng(seed)
    std = np.sqrt(2 / input_size)
    return rng.normal(0, std,
size=shape).astype(np.float32)
```

Fungsi ini mengimplementasikan inisialisasi He, yang merupakan variasi dari Xavier yang lebih cocok digunakan dengan fungsi aktivasi *ReLU*. Dalam metode ini, standar deviasi dihitung berdasarkan jumlah neuron di lapisan sebelumnya menggunakan rumus:

6. initialize_weights

```
def initialize_weights(method, shape, **params):
```

```

"""Initialize weights using specified method"""
methods = {
    "zero": zero_init,
    "random_uniform": random_uniform_init,
    "random_normal": random_normal_init,
    "xavier": xavier_init,
    "he": he_init,
}

# Check if method exists
if method not in methods:
    raise ValueError(f"Unknown initialization method:
{method}")

# Check required parameters
if method == "xavier":
    if "input_size" not in params or "output_size" not
in params:
        raise ValueError("Xavier initialization
requires input_size and output_size")
    elif method == "he":
        if "input_size" not in params:
            raise ValueError("He initialization requires
input_size")

# Get only relevant parameters for the method
filtered_params = _filter_params(method, params)

return methods[method](shape, **filtered_params)

```

Fungsi **initialize_weights()** bertanggung jawab untuk mengatur proses inisialisasi bobot dalam jaringan. Fungsi ini menerima parameter yang menunjukkan metode inisialisasi yang diinginkan (seperti "zero", "random_uniform", "xavier", dll.), serta parameter lain yang relevan seperti input_size dan output_size

yang diperlukan untuk metode tertentu seperti Xavier dan He. Fungsi ini akan memetakan metode inisialisasi yang dipilih dengan fungsi yang sesuai menggunakan kamus yang menghubungkan nama metode dengan implementasi fungsi inisialisasi yang bersangkutan. Sebelum memanggil metode inisialisasi, fungsi ini juga memvalidasi input dan memastikan semua parameter yang diperlukan tersedia. Untuk mengelola parameter dengan lebih efisien, fungsi ini menggunakan metode **_filter_params()** untuk memastikan hanya parameter yang diperlukan yang diteruskan ke fungsi inisialisasi yang tepat.

7. **_filter_params**

```
def _filter_params(method, params):
    """Filter parameters for specific initialization
    method"""
    # Parameters needed for each method
    param_map = {
        "zero": [],
        "random_uniform": ["lower", "upper", "seed"],
        "random_normal": ["mean", "std", "seed"],
        "xavier": ["input_size", "output_size", "seed"],
        "he": ["input_size", "seed"]
    }

    # Get list of parameters for this method
    needed_params = param_map.get(method, [])

    # Return all params if method not found
    if not needed_params:
        return params

    # Filter parameters
    result = {}
```

```

for key, value in params.items():
    if key in needed_params:
        result[key] = value

return result

```

Fungsi **_filter_params()** digunakan untuk memfilter parameter yang diperlukan dari argumen yang diberikan sesuai dengan metode inisialisasi yang dipilih. Fungsi ini memetakan setiap metode inisialisasi ke daftar parameter yang dibutuhkan. Sebagai contoh, untuk metode inisialisasi "xavier", diperlukan `input_size` dan `output_size`, sementara untuk "random_uniform", hanya dibutuhkan parameter `lower`, `upper`, dan `seed`. Dengan cara ini, fungsi ini memastikan bahwa hanya parameter yang relevan dan valid yang diteruskan ke metode inisialisasi, mengurangi kemungkinan kesalahan akibat parameter yang tidak sesuai.

3.1.6 metrics

1. accuracy_score

```

def accuracy_score(y_true, y_pred):
    """Calculate prediction accuracy"""
    # Convert probabilities to class predictions
    if y_pred.ndim > 1 and y_pred.shape[-1] > 1:
        y_pred = np.argmax(y_pred, axis=-1)
        if y_true.ndim > 1 and y_true.shape[-1] > 1:
            y_true = np.argmax(y_true, axis=-1)
    else:
        y_pred = (y_pred > 0.5).astype(int)

    return np.mean(y_pred == y_true)

```

Fungsi **accuracy_score** digunakan untuk menghitung akurasi prediksi model, yaitu persentase prediksi yang benar dibandingkan

dengan total prediksi yang dilakukan. Fungsi ini mengatasi kemungkinan model menghasilkan output probabilistik, baik untuk klasifikasi biner maupun multi-kelas. Jika model memberikan probabilitas, fungsi ini mengonversinya menjadi prediksi kelas: untuk multi-kelas menggunakan *argmax*, dan untuk klasifikasi biner menggunakan threshold 0.5. Setelah itu, akurasi dihitung dengan membandingkan prediksi model dengan nilai yang benar (*y_true*), menghitung persentase kecocokan, dan mengembalikan nilai rata-rata akurasi.

2. **precision_score**

```
def precision_score(y_true, y_pred, average='binary',
per_class=True):
    """Calculate precision score"""
    # Convert to class indices
    if y_pred.ndim > 1 and y_pred.shape[-1] > 1:
        y_pred = np.argmax(y_pred, axis=-1)
        if y_true.ndim > 1 and y_true.shape[-1] > 1:
            y_true = np.argmax(y_true, axis=-1)
    else:
        y_pred = (y_pred > 0.5).astype(int)

    # Binary classification case
    if average == 'binary' or (np.max(y_pred) <= 1 and
np.max(y_true) <= 1):
        tp = np.sum((y_pred == 1) & (y_true == 1))
        fp = np.sum((y_pred == 1) & (y_true == 0))
        return tp / (tp + fp) if (tp + fp) > 0 else 0.0

    # Multi-class case
    num_classes = max(np.max(y_pred) + 1, np.max(y_true) +
1)
    precisions = np.zeros(num_classes)
```

```

    for cls in range(num_classes):
        # Number of true positives (correctly predicted as
        this class)
        tp = np.sum((y_pred == cls) & (y_true == cls))

        # Number of predictions for this class
        pred_as_cls = np.sum(y_pred == cls)

        # Calculate precision for this class
        if pred_as_cls > 0:
            precisions[cls] = tp / pred_as_cls
        # Class with no predictions gets precision of 0

    # Return per-class precision if requested
    if per_class:
        return precisions

    # Different averaging methods
    if average is None or average == 'none':
        return precisions
    elif average == 'macro':
        # Simple mean of class-wise precision
        return np.mean(precisions)
    elif average == 'micro':
        # Calculate global TP and FP
        tp_sum = sum(np.sum((y_pred == cls) & (y_true ==
cls)) for cls in range(num_classes))
        fp_sum = sum(np.sum((y_pred == cls) & (y_true !=
cls)) for cls in range(num_classes))
        return tp_sum / (tp_sum + fp_sum) if (tp_sum +
fp_sum) > 0 else 0.0
    else:
        # Weight by class frequency
        class_counts = np.bincount(y_true,
minlength=num_classes)

```

```

sample_count = len(y_true)
if sample_count > 0:
    weights = class_counts / sample_count
    return np.sum(precisions * weights)
return 0.0

```

precision_score mengukur presisi model, yaitu seberapa akurat prediksi positif yang dibuat oleh model dibandingkan dengan seluruh prediksi positif yang ada. Fungsi ini mendukung baik klasifikasi biner maupun multi-kelas. Untuk klasifikasi biner, presisi dihitung sebagai rasio antara *true positives* (TP) dengan jumlah prediksi positif (TP + FP). Untuk klasifikasi multi-kelas, presisi dihitung untuk setiap kelas secara terpisah dengan menggunakan *np.argmax* untuk mengonversi output probabilitas menjadi kelas yang sesuai. Fungsi ini juga memungkinkan beberapa metode untuk menghitung rata-rata presisi, seperti *macro*, *micro*, dan *weighted*.

3. recall_score

```

def recall_score(y_true, y_pred, average='binary',
per_class=True):
    """Calculate recall score"""
    # Convert to class indices
    if y_pred.ndim > 1 and y_pred.shape[-1] > 1:
        y_pred = np.argmax(y_pred, axis=-1)
        if y_true.ndim > 1 and y_true.shape[-1] > 1:
            y_true = np.argmax(y_true, axis=-1)
    else:
        y_pred = (y_pred > 0.5).astype(int)

    # Binary classification case
    if average == 'binary' or (np.max(y_pred) <= 1 and
np.max(y_true) <= 1):
        tp = np.sum((y_pred == 1) & (y_true == 1))

```

```

        fn = np.sum((y_pred == 0) & (y_true == 1))
        return tp / (tp + fn) if (tp + fn) > 0 else 0.0

    # Multi-class case
    num_classes = max(np.max(y_pred) + 1, np.max(y_true) +
1)
    recalls = np.zeros(num_classes)

    for cls in range(num_classes):
        # Number of true positives (correctly predicted as
this class)
        tp = np.sum((y_pred == cls) & (y_true == cls))

        # Number of true instances of this class
        actual_cls = np.sum(y_true == cls)

        # Calculate recall for this class
        if actual_cls > 0:
            recalls[cls] = tp / actual_cls
        # Class with no instances gets recall of 0

    # Return per-class recall if requested
    if per_class:
        return recalls

    # Different averaging methods
    if average is None or average == 'none':
        return recalls
    elif average == 'macro':
        # Simple mean of class-wise recall
        return np.mean(recalls)
    elif average == 'micro':
        # Calculate global TP and FN
        tp_sum = sum(np.sum((y_pred == cls) & (y_true ==
cls)) for cls in range(num_classes))

```

```

        fn_sum = sum(np.sum((y_pred != cls) & (y_true ==
cls)) for cls in range(num_classes))
        return tp_sum / (tp_sum + fn_sum) if (tp_sum +
fn_sum) > 0 else 0.0
    else: # weighted
        # Weight by class frequency
        class_counts = np.bincount(y_true,
minlength=num_classes)
        sample_count = len(y_true)
        if sample_count > 0:
            weights = class_counts / sample_count
            return np.sum(recalls * weights)
        return 0.0

```

Fungsi **recall_score** menghitung recall, yaitu seberapa baik model dalam mengidentifikasi semua contoh positif dari data yang benar. Untuk klasifikasi biner, recall dihitung sebagai rasio *true positives* (TP) terhadap jumlah total kelas positif yang benar (TP + FN). Untuk klasifikasi multi-kelas, recall dihitung untuk setiap kelas secara terpisah. Sama seperti presisi, recall juga dapat dihitung dengan beberapa metode penghitungan rata-rata, termasuk *macro*, *micro*, atau *weighted*, tergantung pada cara menggabungkan hasil dari setiap kelas.

4. f1_score

```

def f1_score(y_true, y_pred, average='binary',
per_class=True):
    """Calculate F1 score"""
    # Get per-class precision and recall
    precisions = precision_score(y_true, y_pred,
average=None)
    recalls = recall_score(y_true, y_pred, average=None)

```

```

# Calculate per-class F1
f1_scores = np.zeros_like(precisions)
for i in range(len(precisions)):
    if precisions[i] + recalls[i] > 0:
        f1_scores[i] = 2 * (precisions[i] * recalls[i])
/ (precisions[i] + recalls[i])

# Return per-class F1 if requested
if per_class:
    return f1_scores

# Average F1 scores based on method
if average is None or average == 'none':
    return f1_scores
elif average == 'macro':
    return np.mean(f1_scores)
elif average == 'micro':
    # Recalculate using micro-averaged precision and
recall
    prec = precision_score(y_true, y_pred,
average='micro')
    rec = recall_score(y_true, y_pred, average='micro')
    return 2 * (prec * rec) / (prec + rec) if (prec +
rec) > 0 else 0.0
else: # weighted
    # Weight by class frequency
    if y_true.ndim > 1 and y_true.shape[1] > 1:
        y_true_indices = np.argmax(y_true, axis=1)
    else:
        y_true_indices = y_true

    num_classes = len(f1_scores)
    class_counts = np.bincount(y_true_indices,
minlength=num_classes)
    weights = class_counts / len(y_true_indices)

```

```
return np.sum(f1_scores * weights)
```

Fungsi **f1_score** menggabungkan presisi dan recall menjadi satu metrik untuk memberikan gambaran seimbang antara keduanya. Fungsi ini juga memungkinkan perhitungan F1 score per kelas untuk masalah multi-kelas, dan mendukung penghitungan rata-rata menggunakan metode *macro*, *micro*, atau *weighted*. F1 score sangat berguna terutama ketika ada ketidakseimbangan antara kelas, karena memberikan bobot yang lebih seimbang antara presisi dan recall.

5. confusion_matrix

```
def confusion_matrix(y_true, y_pred):
    """Generate confusion matrix where rows are true labels
    and columns are predicted labels"""
    # Convert to class indices
    if y_pred.ndim > 1 and y_pred.shape[-1] > 1:
        y_pred = np.argmax(y_pred, axis=-1)
    if y_true.ndim > 1 and y_true.shape[-1] > 1:
        y_true = np.argmax(y_true, axis=-1)
    else:
        y_pred = (y_pred > 0.5).astype(int)

    # Create matrix
    num_classes = max(np.max(y_pred) + 1, np.max(y_true) +
1)
    matrix = np.zeros((num_classes, num_classes),
dtype=int)

    # Fill matrix: rows are true labels, columns are
predicted labels
    for i in range(len(y_true)):
        matrix[y_true[i], y_pred[i]] += 1

    return matrix
```

confusion_matrix menghasilkan matriks kebingungannya, yang memberikan gambaran yang lebih mendalam tentang bagaimana model mengklasifikasikan data dengan membandingkan prediksi model dengan label yang benar. Matriks ini menunjukkan jumlah *true positives* (TP), *false positives* (FP), *false negatives* (FN), dan *true negatives* (TN) untuk setiap kelas. Matriks kebingungannya sangat berguna untuk memahami jenis kesalahan yang dilakukan model, misalnya apakah model cenderung salah mengklasifikasikan satu kelas menjadi kelas lain.

6. **classification_report**

```
def classification_report(y_true, y_pred, digits=3):
    """Generate a text report showing the main
    classification metrics"""
    # Convert to class indices
    if y_pred.ndim > 1 and y_pred.shape[-1] > 1:
        y_pred_indices = np.argmax(y_pred, axis=-1)
        if y_true.ndim > 1 and y_true.shape[-1] > 1:
            y_true_indices = np.argmax(y_true, axis=-1)
        else:
            y_true_indices = y_true
    else:
        y_pred_indices = (y_pred > 0.5).astype(int)
        y_true_indices = y_true

    # Calculate accuracy first - this should match your
    reported accuracy
    acc = np.mean(y_pred_indices == y_true_indices)

    # Get metrics
    precisions = precision_score(y_true_indices,
    y_pred_indices, average=None)
    recalls = recall_score(y_true_indices, y_pred_indices,
    average=None)
```



```

    f1_scores = f1_score(y_true_indices, y_pred_indices,
average=None)

    # Get support (number of samples per class)
    num_classes = len(precisions)
    support = np.bincount(y_true_indices,
minlength=num_classes)

    # Create header
    header = f"{'':>8} {'precision':>10} {'recall':>10}
{'f1-score':>10} {'support':>10}"
    report = [header, "-" * len(header)]

    # Add per-class metrics
    for i in range(num_classes):
        report.append(f"{i:>8}
{precisions[i]:{digits+6}.{digits}f}
{recalls[i]:{digits+6}.{digits}f} "
                    f"{f1_scores[i]:{digits+6}.{digits}f}
{support[i]:>10}")

    # Add averages
    report.append("-" * len(header))

    # Macro average (equal weight to all classes)
    macro_prec = np.mean(precisions)
    macro_rec = np.mean(recalls)
    macro_f1 = np.mean(f1_scores)
    report.append(f"{'macro avg':>8}
{macro_prec:{digits+6}.{digits}f}
{macro_rec:{digits+6}.{digits}f} "
                f"{macro_f1:{digits+6}.{digits}f}
{sum(support):>10}")

    # Weighted average (weight by class frequency)

```

```

weights = support / sum(support)
weighted_prec = np.sum(precisions * weights)
weighted_rec = np.sum(recalls * weights)
weighted_f1 = np.sum(f1_scores * weights)
report.append(f"{'weighted avg':>8}
{weighted_prec:{digits+6}.{digits}f}
{weighted_rec:{digits+6}.{digits}f} "
              f"{weighted_f1:{digits+6}.{digits}f}
{sum(support):>10}")

# Accuracy
report.append(f"\nAccuracy: {acc:.{digits}f}")

return "\n".join(report)

```

Fungsi **classification_report** menghasilkan laporan teks yang menyajikan metrik-metrik utama seperti presisi, recall, dan F1 score untuk setiap kelas. Laporan ini juga mencakup nilai rata-rata *macro* dan *weighted* untuk keseluruhan model. Selain itu, laporan ini menunjukkan jumlah contoh untuk setiap kelas (*support*) dan akurasi keseluruhan model. Laporan ini memberikan gambaran menyeluruh tentang kinerja model pada masing-masing kelas serta rata-rata kinerja model secara keseluruhan. Fungsi ini juga memungkinkan penentuan jumlah digit yang ditampilkan untuk setiap metrik menggunakan parameter *digits*.

3.1.7 trainer

1. Trainer

```

class Trainer:
    """Helper class for training neural networks"""

    def __init__(self, model, learning_rate, loss_function,

```

```

batch_size=32, epochs=10, verbose=1):
    self.model = model
    self.learning_rate = learning_rate
    self.loss_function = loss_function
    self.batch_size = batch_size
    self.epochs = epochs
    self.verbose = verbose
    self.history = None

    def train(self, X_train, y_train, X_val=None,
y_val=None, callbacks=None):
        """Train the model"""
        if callbacks is None:
            callbacks = []

        # Use model's train method
        self.history = self.model.train(
            X_train=X_train,
            y_train=y_train,
            learning_rate=self.learning_rate,
            loss_function=self.loss_function,
            X_val=X_val,
            y_val=y_val,
            batch_size=self.batch_size,
            epochs=self.epochs,
            verbose=self.verbose
        )

        # Run callbacks
        for callback in callbacks:
            callback(self.model, self.history)

        return self.history

```

Kelas Trainer adalah kelas pendukung yang dirancang untuk mempermudah proses pelatihan model jaringan saraf. Kelas ini menyatukan beberapa elemen penting dalam pelatihan seperti pengaturan *learning rate*, fungsi *loss*, dan ukuran *batch*. Dengan menggunakan kelas ini, pengguna bisa mengorganisir dan mengelola berbagai parameter yang dibutuhkan selama pelatihan model.

Atribut-atribut utama yang dimiliki kelas Trainer adalah:

a. Model

Menyimpan instance model jaringan saraf yang akan dilatih.

b. Learning_rate

Menentukan seberapa besar perubahan yang terjadi pada parameter model setelah setiap iterasi.

c. Loss_function

Fungsi yang digunakan untuk menghitung seberapa besar kesalahan antara prediksi dan nilai yang sebenarnya.

d. Batch_size

Mengontrol jumlah sampel yang diproses dalam satu kali iterasi.

e. Epochs

Menetapkan jumlah iterasi pelatihan yang akan dilakukan.

f. Verbose

Mengatur tingkat informasi yang ditampilkan selama pelatihan.

g. History

Menyimpan riwayat metrik pelatihan, seperti *loss*, selama proses pelatihan.

Metode utama yang ada dalam kelas ini adalah `train()`, yang menjalankan proses pelatihan pada model. Metode ini menerima data pelatihan (`X_train`, `y_train`) dan data validasi opsional (`X_val`, `y_val`). Selama pelatihan, metode ini akan memanggil fungsi `train()` dari model

untuk menjalankan proses pelatihan, dan kemudian mengelola callback yang diteruskan. Callback ini dapat berisi berbagai tindakan yang ingin dijalankan selama pelatihan, seperti pengaturan learning rate atau penghentian pelatihan lebih awal.

2. Train_test_split

```
def train_test_split(X, y, test_size=0.2,
                    random_state=None, stratify=None):
    """Split data into train and test sets"""
    if random_state is not None:
        np.random.seed(random_state)

    n_samples = X.shape[0]

    if stratify is not None:
        # Stratified split to maintain class proportions
        unique_classes = np.unique(stratify)
        test_indices = []
        train_indices = []

        for cls in unique_classes:
            cls_indices = np.where(stratify == cls)[0]
            n_cls_samples = len(cls_indices)

            # Shuffle indices for this class
            np.random.shuffle(cls_indices)

            # Split by test size ratio
            n_test_samples = int(n_cls_samples * test_size)

            test_indices.extend(cls_indices[:n_test_samples])

            train_indices.extend(cls_indices[n_test_samples:])

    # Check test size
```

```

        actual_test_size = len(test_indices) / n_samples
        if abs(actual_test_size - test_size) > 0.05:
            print(f"Warning: Actual test size
({actual_test_size:.2f}) differs from requested
({test_size:.2f})")
        else:
            # Random split
            indices = np.random.permutation(n_samples)
            test_count = int(n_samples * test_size)
            test_indices = indices[:test_count]
            train_indices = indices[test_count:]

    # Create train and test sets
    X_train, X_test = X[train_indices], X[test_indices]
    y_train, y_test = y[train_indices], y[test_indices]

    return X_train, X_test, y_train, y_test

```

Fungsi **train_test_split()** digunakan untuk membagi dataset menjadi dua bagian: data untuk pelatihan dan data untuk pengujian. Fungsi ini menerima data fitur (**X**) dan target (**y**), serta beberapa parameter tambahan seperti ukuran data pengujian (**test_size**), seed untuk kontrol acak, dan parameter **stratify** yang memastikan distribusi kelas dalam data pelatihan dan pengujian tetap seimbang. Jika parameter **stratify** diberikan, pembagian dilakukan dengan mempertahankan proporsi kelas yang ada, sedangkan jika tidak, pembagian dilakukan secara acak. Fungsi ini akan mengembalikan dua set data: satu untuk pelatihan (**X_train, y_train**) dan satu untuk pengujian (**X_test, y_test**).

3. EarlyStopping

```

class EarlyStopping:
    """Early stopping callback to prevent overfitting"""

```

```

def __init__(self, patience=5, min_delta=0.0):
    self.patience = patience
    self.min_delta = min_delta
    self.wait = 0
    self.best_loss = float('inf')
    self.stopped_epoch = 0

def __call__(self, model, history):
    """Check if training should stop"""
    # Get loss value (prefer validation loss)
    if 'val_loss' in history and history['val_loss']:
        current_loss = history['val_loss'][-1]
    elif 'train_loss' in history and
history['train_loss']:
        current_loss = history['train_loss'][-1]
    else:
        return False

    # Check for improvement
    if current_loss < self.best_loss - self.min_delta:
        self.best_loss = current_loss
        self.wait = 0
    else:
        self.wait += 1
        if self.wait >= self.patience:
            # Get epoch count
            if 'train_loss' in history:
                self.stopped_epoch =
len(history['train_loss'])
            elif 'val_loss' in history:
                self.stopped_epoch =
len(history['val_loss'])
            else:
                self.stopped_epoch = 0

```

```
        print(f"\nEarly stopping at epoch  
{self.stopped_epoch}")  
        model.stop_training = True  
        return True  
  
    return False
```

Kelas EarlyStopping adalah sebuah *callback* yang digunakan untuk menghentikan pelatihan lebih awal jika tidak ada peningkatan yang signifikan dalam metrik tertentu, seperti *loss*. Ini berguna untuk mencegah model overfit dengan menghentikan pelatihan sebelum model terlalu banyak "belajar" dari data pelatihan.

Atribut utama dalam kelas ini adalah:

a. Patience

Menentukan berapa banyak epoch berturut-turut yang diizinkan tanpa adanya peningkatan sebelum pelatihan dihentikan.

b. Min_delta

Menetapkan batas minimum perubahan yang dianggap sebagai peningkatan.

c. Wait

Melacak jumlah epoch yang telah berlalu tanpa adanya peningkatan.

d. Best_loss

Menyimpan nilai *loss* terbaik yang tercapai selama pelatihan.

Ketika model tidak menunjukkan peningkatan setelah sejumlah epoch tertentu, pelatihan akan dihentikan. Metode **call()** memungkinkan kelas ini digunakan sebagai *callback* selama pelatihan

untuk memeriksa apakah pelatihan harus dihentikan berdasarkan kondisi saat itu.

4. LearningRateScheduler

```
class LearningRateScheduler:
    """Learning rate scheduler with exponential decay"""

    def __init__(self, initial_lr=0.01, decay_rate=0.95,
decay_steps=1):
        self.initial_lr = initial_lr
        self.decay_rate = decay_rate
        self.decay_steps = decay_steps
        self.current_lr = initial_lr
        self.epochs = 0

    def get_lr(self):
        """Get current learning rate"""
        self.current_lr = self.initial_lr *
(self.decay_rate ** (self.epochs // self.decay_steps))
        return self.current_lr

    def step(self):
        """Increment epoch counter and return new learning
rate"""
        self.epochs += 1
        return self.get_lr()

    def __call__(self, model, history):
        """Callback interface for use with Trainer"""
        self.epochs = len(history.get('train_loss', []))
        new_lr = self.get_lr()
        model.learning_rate = new_lr
        return new_lr
```

Kelas `LearningRateScheduler` adalah sebuah *callback* yang mengatur perubahan *learning rate* selama pelatihan, menggunakan pembaruan eksponensial.

Atribut utama dalam kelas ini adalah:

a. `Initial_lr`

Menyimpan nilai *learning rate* awal.

b. `Decay_rate`

Menentukan laju penurunan *learning rate*.

c. `Decay_steps`

Mengontrol frekuensi pembaruan *learning rate* setelah beberapa epoch.

d. `Current_lr`

Menyimpan nilai *learning rate* yang sedang berlaku.

e. `Epochs`

Melacak jumlah epoch yang telah dijalani.

Metode **`get_lr()`** menghitung nilai *learning rate* saat ini berdasarkan pembaruan eksponensial, sementara metode **`step()`** menambah jumlah epoch dan mengembalikan *learning rate* yang telah diperbarui. Metode **`call()`** memungkinkan kelas ini untuk digunakan sebagai *callback* dalam pelatihan, di mana setiap kali dipanggil, *learning rate* akan diperbarui dan diterapkan ke model.

3.1.8 visualization

1. `Visualize_model_performance`

```
def visualize_model_performance(model, history,
                               title="Model Performance"):
    """Plot training history and weight distributions"""
    # Create figure with loss plot and weight visualization
    fig = plt.figure(figsize=(12, 5))
```

```

# Left subplot: loss curves
ax1 = fig.add_subplot(1, 2, 1)
ax1.plot(history["train_loss"], label="Training Loss",
color='steelblue')

if "val_loss" in history:
    ax1.plot(history["val_loss"], linestyle="--",
label="Validation Loss", color='indianred')

ax1.set_title(f"{title}: Loss History", fontsize=12)
ax1.set_xlabel("Epoch", fontsize=10)
ax1.set_ylabel("Loss", fontsize=10)
ax1.grid(True, alpha=0.3)
ax1.legend(fontsize=9)

# Right subplot: weight distributions
ax2 = fig.add_subplot(1, 2, 2)

# Find layers with weights
dense_layers = [i for i, layer in
enumerate(model.layers) if hasattr(layer, 'weights')]

if dense_layers:
    for layer_idx in dense_layers:
        weights =
model.layers[layer_idx].weights.flatten()
        sns.kdeplot(weights, label=f"Layer
{layer_idx}", ax=ax2)

ax2.set_title("Weight Distributions", fontsize=12)
ax2.set_xlabel("Weight Value", fontsize=10)
ax2.set_ylabel("Density", fontsize=10)
ax2.legend(fontsize=9)

plt.tight_layout()

```

```
return fig
```

Fungsi **visualize_model_performance** bertujuan untuk memvisualisasikan kinerja model selama pelatihan dengan menampilkan dua grafik utama:

a. Kurva loss pelatihan dan validasi

Grafik pertama ini menunjukkan bagaimana nilai loss berubah seiring waktu (epoch) untuk data pelatihan dan validasi. Hal ini memberikan gambaran tentang seberapa baik model mengadaptasi diri terhadap data pelatihan dan seberapa baik model dapat menggeneralisasi pada data validasi.

b. Distribusi bobot model

Grafik kedua ini menggambarkan distribusi bobot untuk setiap lapisan dalam model. Visualisasi ini membantu memeriksa apakah ada masalah dalam distribusi bobot, seperti gradien yang sangat kecil atau besar. Grafik distribusi bobot ini menggunakan *Kernel Density Estimate* (KDE) yang digambar dengan bantuan `sns.kdeplot`.

Fungsi ini dirancang untuk lapisan yang memiliki bobot, memberikan pemahaman yang lebih baik tentang bagaimana bobot model berkembang selama pelatihan.

2. Visualize_model_weights_and_gradients

```
def visualize_model_weights_and_gradients(model,
max_layers=8):
    """Show weight and gradient distributions for layers"""
    # Find layers with weights
    layer_indices = [i for i, layer in
enumerate(model.layers)
                    if hasattr(layer, 'weights')]
```

```

# Limit number of displayed layers
if len(layer_indices) > max_layers:
    step = len(layer_indices) // max_layers
    if step < 1:
        step = 1
    layer_indices = layer_indices[::step][:max_layers]

n_layers = len(layer_indices)
if n_layers == 0:
    print("No layers with weights found.")
    return None

# Create figure with weights and gradients side by side
fig, axs = plt.subplots(n_layers, 2, figsize=(10,
3*n_layers))

# Handle single layer case
if n_layers == 1:
    axs = axs.reshape(1, 2)

for i, layer_idx in enumerate(layer_indices):
    layer = model.layers[layer_idx]

    # Plot weights
    weights = layer.weights.flatten()
    sns.histplot(weights, kde=True, ax=axs[i, 0],
color='steelblue', bins=30)
    axs[i, 0].set_title(f'Layer {layer_idx}: Weights',
fontsize=10)

    # Add stats
    w_mean = np.mean(weights)
    w_std = np.std(weights)
    axs[i, 0].text(0.95, 0.95, f"μ: {w_mean:.5f}\nσ:
{w_std:.5f}",

```

```

        transform=axes[i, 0].transAxes,
        ha='right', va='top', fontsize=8,
        bbox=dict(boxstyle='round',
facecolor='white', alpha=0.7))

    # Plot gradients if available
    if hasattr(layer, 'weights_gradient') and
layer.weights_gradient is not None:
        gradients = layer.weights_gradient.flatten()
        sns.histplot(gradients, kde=True, ax=axes[i, 1],
color='indianred', bins=30)
        axes[i, 1].set_title(f'Layer {layer_idx}:
Gradients', fontsize=10)

        # Add gradient stats
        g_mean = np.mean(gradients)
        g_std = np.std(gradients)
        axes[i, 1].text(0.95, 0.95, f"μ:
{g_mean:.6f}\nσ: {g_std:.6f}",
        transform=axes[i, 1].transAxes,
        ha='right', va='top', fontsize=8,
        bbox=dict(boxstyle='round',
facecolor='white', alpha=0.7))
    else:
        axes[i, 1].text(0.5, 0.5, "No gradient data
available",
        ha='center', va='center',
fontsize=10)
        axes[i, 1].set_title(f'Layer {layer_idx}: No
Gradients', fontsize=10)

plt.tight_layout()
return fig

```

Fungsi **visualize_model_weights_and_gradients** menyediakan visualisasi yang lebih rinci tentang distribusi bobot dan gradien pada setiap lapisan yang memiliki bobot. Untuk setiap lapisan, bobot dan gradien dihitung dan divisualisasikan menggunakan histogram serta distribusi *Kernel Density Estimate* (KDE). Fungsi ini membantu menganalisis apakah ada masalah seperti bobot yang terlalu besar atau kecil, serta bagaimana distribusi gradien pada lapisan-lapisan tertentu.

Fungsi ini juga menyertakan statistik penting, seperti rata-rata (mean) dan standar deviasi (std) untuk bobot dan gradien, yang ditampilkan di dalam grafik. Pengguna dapat memilih lapisan tertentu (hingga maksimal 8 lapisan) untuk divisualisasikan, memungkinkan analisis mendalam terhadap perkembangan bobot dan gradien dalam pelatihan.

3. Compare_models_performance

```
def compare_models_performance(models_dict, figsize=(12, 8)):  
    """Compare training curves and weights across models"""  
    # Extract models and histories  
    models = {}  
    histories = {}  
  
    for name, result in models_dict.items():  
        if isinstance(result, dict) and "model" in result:  
            models[name] = result["model"]  
            histories[name] = result["history"]  
        elif isinstance(result, tuple) and len(result) >=  
2:  
            models[name] = result[0]  
            histories[name] = result[1]  
        else:  
            raise ValueError(f"Could not extract
```

```

model/history for {name}")

    # Create loss comparison figure
    loss_fig, loss_ax = plt.subplots(figsize=(figsize[0],
figsize[1]//2))

    for name, history in histories.items():
        loss_ax.plot(history["train_loss"], label=f"{name}
(Train)")
        if "val_loss" in history:
            loss_ax.plot(history["val_loss"],
linestyle="--", label=f"{name} (Val)")

    loss_ax.set_title("Training Loss Comparison",
fontsize=14)
    loss_ax.set_xlabel("Epoch", fontsize=12)
    loss_ax.set_ylabel("Loss", fontsize=12)
    loss_ax.grid(True, alpha=0.3)
    loss_ax.legend(fontsize=10)

    # Find common layers to compare weights
    common_layers = min([len(model.layers) for model in
models.values()])
    dense_layers = []

    # Find layers with weights across all models
    for i in range(common_layers):
        if all(hasattr(model.layers[i], 'weights') for
model in models.values()):
            dense_layers.append(i)

    if not dense_layers:
        return loss_fig, None

    # Select representative layers

```



```

if len(dense_layers) > 3:
    dense_layers = [
        dense_layers[0],
        dense_layers[len(dense_layers)//2],
        dense_layers[-1]
    ]

# Create weight comparison figure
n_layers = len(dense_layers)
weights_fig, w_axs = plt.subplots(n_layers, 1,
figsize=(figsize[0], figsize[1]//2))

if n_layers == 1:
    w_axs = [w_axs]

# Plot weights for each model by layer
for i, layer_idx in enumerate(dense_layers):
    for name, model in models.items():
        weights =
model.layers[layer_idx].weights.flatten()
        sns.kdeplot(weights, ax=w_axs[i], label=name)

        w_axs[i].set_title(f"Layer {layer_idx} Weights",
fontsize=12)
        w_axs[i].set_xlabel("Weight Value", fontsize=10)
        w_axs[i].set_ylabel("Density", fontsize=10)
        w_axs[i].legend(fontsize=9)

plt.tight_layout()
return loss_fig, weights_fig

```

Fungsi **compare_models_performance** memungkinkan perbandingan kinerja beberapa model dengan membandingkan kurva loss dan distribusi bobot di antara berbagai model dalam satu

grafik. Fungsi ini menerima sebuah dictionary yang berisi nama model dan hasil pelatihannya, kemudian menghasilkan dua grafik:

- a. Perbandingan loss pelatihan dan validasi antara berbagai model.
- b. Perbandingan distribusi bobot pada lapisan yang sama di berbagai model.

Jika ada lebih dari tiga lapisan yang memiliki bobot di semua model, fungsi ini akan memilih lapisan pertama, tengah, dan terakhir untuk dibandingkan. Fungsi ini sangat berguna untuk menganalisis bagaimana berbagai model berperilaku selama pelatihan dan bagaimana bobot masing-masing model berkembang..

4. Compare_per_class_accuracy

```
def compare_per_class_accuracy(custom_model=None,
                               sklearn_model=None, X_test=None, y_test=None,
                               custom_history=None,
                               sklearn_history=None, custom_preds=None,
                               model_names=None):
    if model_names is None:
        model_names = ["Custom Model", "Sklearn Model"]

    # Create loss comparison figure
    if custom_history is not None and sklearn_history is
    not None:
        loss_fig, loss_ax = plt.subplots(figsize=(12, 6))

        # Custom model loss
        loss_ax.plot(custom_history["train_loss"],
                     label=f"{model_names[0]} (Train)", color='steelblue')
        if "val_loss" in custom_history:
            loss_ax.plot(custom_history["val_loss"],
                         linestyle="--",
                         label=f"{model_names[0]} (Val)",
```

```

color='lightblue')

    # Sklearn model loss
    if 'loss' in sklearn_history:
        loss_ax.plot(sklearn_history["loss"],
label=f"{model_names[1]}", color='indianred')

        loss_ax.set_title("Training Loss Comparison",
fontsize=14)
        loss_ax.set_xlabel("Epoch", fontsize=12)
        loss_ax.set_ylabel("Loss", fontsize=12)
        loss_ax.grid(True, alpha=0.3)
        loss_ax.legend()
        plt.tight_layout()
    else:
        loss_fig = None

    # Get custom predictions
    if custom_preds is None and custom_model is not None
and X_test is not None:
        custom_preds = custom_model.predict(X_test)

    # Get sklearn predictions
    if sklearn_model is not None and X_test is not None:
        if hasattr(sklearn_model, 'predict_proba'):
            sklearn_preds =
sklearn_model.predict_proba(X_test)
        else:
            sklearn_preds = np.zeros((X_test.shape[0],
y_test.shape[1]))
            preds = sklearn_model.predict(X_test)
            for i, p in enumerate(preds):
                sklearn_preds[i, p] = 1
    else:
        sklearn_preds = None

```

```

# Compare per-class accuracy
if custom_preds is not None and sklearn_preds is not
None and y_test is not None:
    # Convert to class indices
    y_true = np.argmax(y_test, axis=1)
    pred1 = np.argmax(custom_preds, axis=1)
    pred2 = np.argmax(sklearn_preds, axis=1)

    # Calculate per-class accuracies
    num_classes = y_test.shape[1]
    acc1 = np.zeros(num_classes)
    acc2 = np.zeros(num_classes)

    for i in range(num_classes):
        class_samples = np.where(y_true == i)[0]
        if len(class_samples) > 0:
            acc1[i] = np.mean(pred1[class_samples] ==
i)
            acc2[i] = np.mean(pred2[class_samples] ==
i)

    # Create bar chart
    acc_fig, ax = plt.subplots(figsize=(12, 6))
    x = np.arange(num_classes)
    width = 0.35

    ax.bar(x - width/2, acc1, width,
label=model_names[0], color='steelblue', alpha=0.7)
    ax.bar(x + width/2, acc2, width,
label=model_names[1], color='indianred', alpha=0.7)

    ax.set_xlabel('Class', fontsize=12)
    ax.set_ylabel('Accuracy', fontsize=12)
    ax.set_title('Per-Class Accuracy Comparison',

```

```

fontsize=14)
    ax.set_xticks(x)
    ax.set_xticklabels([str(i) for i in
range(num_classes)])
    ax.legend()
    ax.grid(True, alpha=0.3, axis='y')
    plt.tight_layout()

    # Calculate overall accuracy
    custom_acc = np.mean(pred1 == y_true)
    sklearn_acc = np.mean(pred2 == y_true)

    # Create summary
    summary = {
        "custom_accuracy": custom_acc,
        "sklearn_accuracy": sklearn_acc,
        "custom_per_class": acc1,
        "sklearn_per_class": acc2,
        "custom_confusion": confusion_matrix(y_true,
pred1),
        "sklearn_confusion": confusion_matrix(y_true,
pred2)
    }
    else:
        acc_fig = None
        summary = {}

    # Return figures and data
    return {"loss": loss_fig, "accuracy": acc_fig}, summary

```

Fungsi **compare_per_class_accuracy** digunakan untuk membandingkan akurasi per kelas antara dua model, misalnya antara model kustom dan model yang menggunakan pustaka seperti sklearn. Fungsi ini menerima dua model yang telah dilatih, data uji, dan hasil prediksi dari kedua model, kemudian membandingkan akurasi

masing-masing kelas secara terpisah. Hasilnya berupa grafik batang yang menunjukkan perbandingan akurasi antara kedua model untuk setiap kelas.

Selain itu, fungsi ini juga menghitung dan menampilkan akurasi keseluruhan dari kedua model, serta matriks kebingungannya (confusion matrix) untuk menganalisis kesalahan klasifikasi di tingkat kelas. Jika riwayat *loss* tersedia, fungsi ini juga akan menampilkan grafik perbandingan *loss* antara kedua model, memberikan analisis lebih dalam tentang bagaimana model-model tersebut berkinerja dalam hal akurasi dan kerugian selama pelatihan.

3.2 Forward Propagation

Forward propagation adalah langkah pertama dalam proses pelatihan jaringan saraf, di mana data input dimasukkan ke dalam jaringan untuk menghasilkan prediksi. Pada tahap ini, input pertama kali diproses oleh lapisan pertama jaringan. Di setiap lapisan, input dikalikan dengan bobot, kemudian bias ditambahkan, menghasilkan nilai yang disebut *z*. Nilai *z* ini kemudian diproses oleh fungsi aktivasi, yang memberikan elemen non-linearitas pada model, seperti ReLU, Sigmoid, Tanh, atau Softmax. Fungsi aktivasi ini memungkinkan model menangkap pola yang lebih kompleks dalam data.

Setelah hasil dari lapisan pertama dihitung, output tersebut menjadi input untuk lapisan berikutnya. Proses ini berulang hingga mencapai lapisan output, di mana model menghasilkan prediksi akhir. Output dari forward propagation ini selanjutnya dibandingkan dengan label yang sebenarnya untuk menghitung kesalahan (*loss*).

Pada kode yang ada, fungsi **forward_propagation()** di kelas `NeuralNetwork` bertanggung jawab untuk melaksanakan forward pass. Di dalamnya, data input

diproses melalui beberapa lapisan, dengan setiap lapisan melakukan transformasi linear (perkalian bobot dan penambahan bias) dan kemudian diikuti oleh aplikasi fungsi aktivasi. Hasil akhirnya adalah prediksi model yang digunakan untuk menghitung error atau loss.

3.3 Backward Propagation and Update Weight

Setelah forward propagation, langkah berikutnya adalah **backward propagation**, yang bertujuan untuk menghitung gradien dari fungsi loss terhadap parameter jaringan (seperti bobot dan bias). Tujuan utama dari backward propagation adalah untuk menentukan seberapa besar pengaruh masing-masing bobot terhadap kesalahan prediksi, dan untuk memperbarui bobot-bobot tersebut agar model bisa lebih akurat.

Proses ini dimulai dengan menghitung gradien error pada lapisan output, yaitu selisih antara prediksi model dan nilai sebenarnya, yang biasanya dihitung menggunakan fungsi loss seperti Mean Squared Error (MSE) atau Cross-Entropy. Gradien ini kemudian diteruskan ke lapisan-lapisan sebelumnya, melalui aturan rantai, untuk menghitung gradien pada setiap lapisan dari output ke input.

Pada kode yang ada, fungsi **backward_propagation()** di kelas `NeuralNetwork` menangani backward pass ini. Fungsi ini pertama-tama menerima gradien yang dihitung pada lapisan output dan kemudian menghitung gradien untuk lapisan-lapisan sebelumnya. Setiap lapisan menghitung gradiennya berdasarkan turunan dari fungsi aktivasi yang digunakan pada hasil transformasi linear (z), serta gradien yang diterima dari lapisan berikutnya. Gradien untuk bobot dihitung dengan cara mengalikan input dari lapisan sebelumnya dengan gradien pada lapisan saat ini. Sementara gradien untuk bias dihitung dengan menjumlahkan gradien pada seluruh batch.

Setelah gradien dihitung untuk semua bobot dan bias, langkah selanjutnya adalah **pembaruan bobot**. Tujuan dari pembaruan ini adalah untuk meminimalkan error atau loss. Pembaruan bobot dilakukan dengan cara mengurangi bobot sesuai

dengan gradien yang dihitung, dengan faktor pembaruan yang dikenal sebagai *learning rate*. Pada kode ini, pembaruan bobot dilakukan menggunakan algoritma **Adam**, yang menggabungkan teknik *momentum* dan estimasi *RMSprop*. Pendekatan ini membantu mempercepat konvergensi dan meningkatkan stabilitas selama pelatihan. Fungsi **update_weights()** di kelas `NeuralNetwork` mengimplementasikan algoritma Adam, yang mengupdate bobot dan bias dengan memanfaatkan estimasi *momentum* (m) dan *RMSprop* (v), serta melakukan perbaikan bias berdasarkan jumlah langkah pelatihan (t).

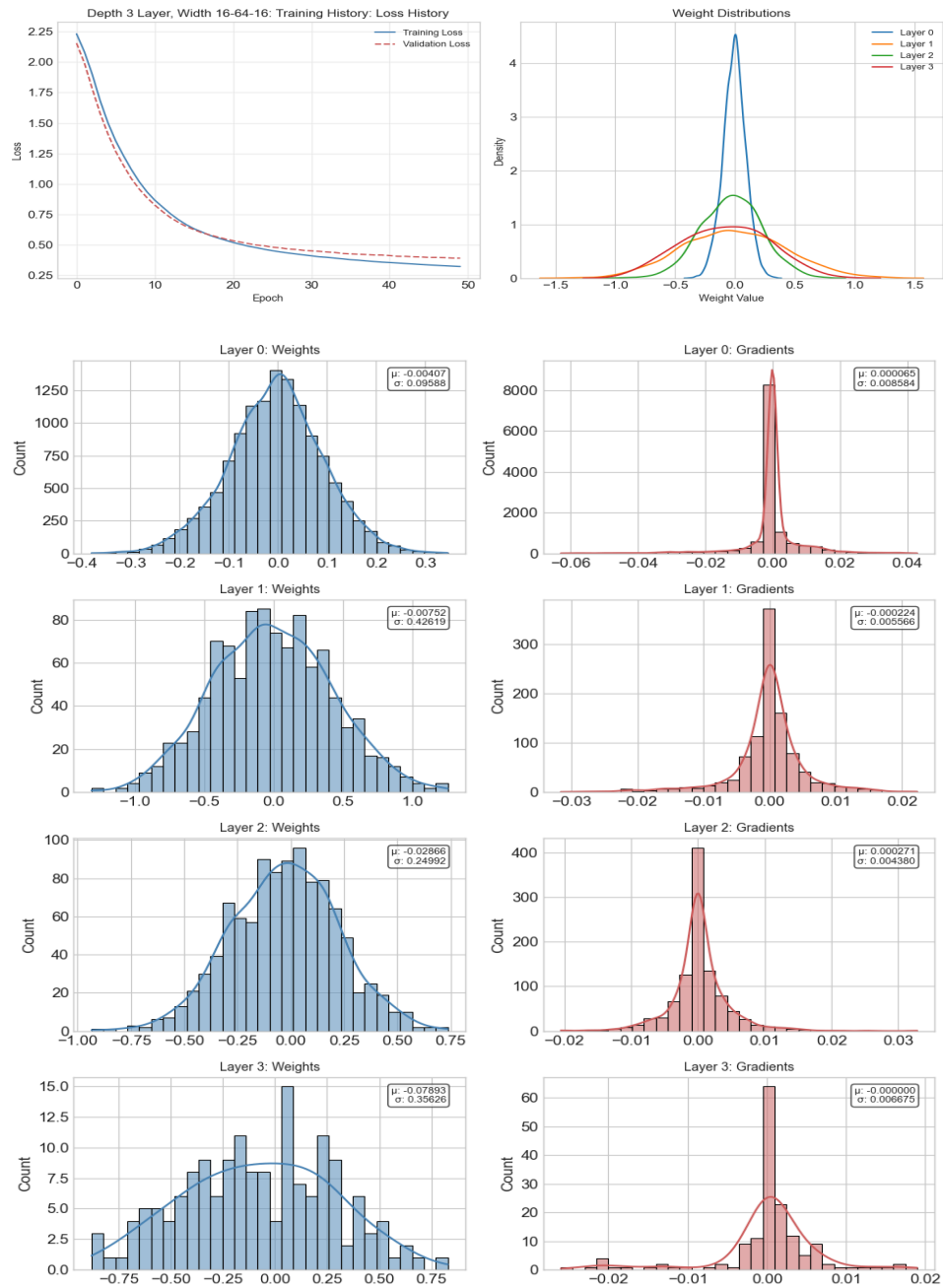
Learning rate yang digunakan untuk pembaruan bobot ini bisa disesuaikan, baik secara manual maupun menggunakan scheduler untuk pengaturan otomatis.

Bab IV

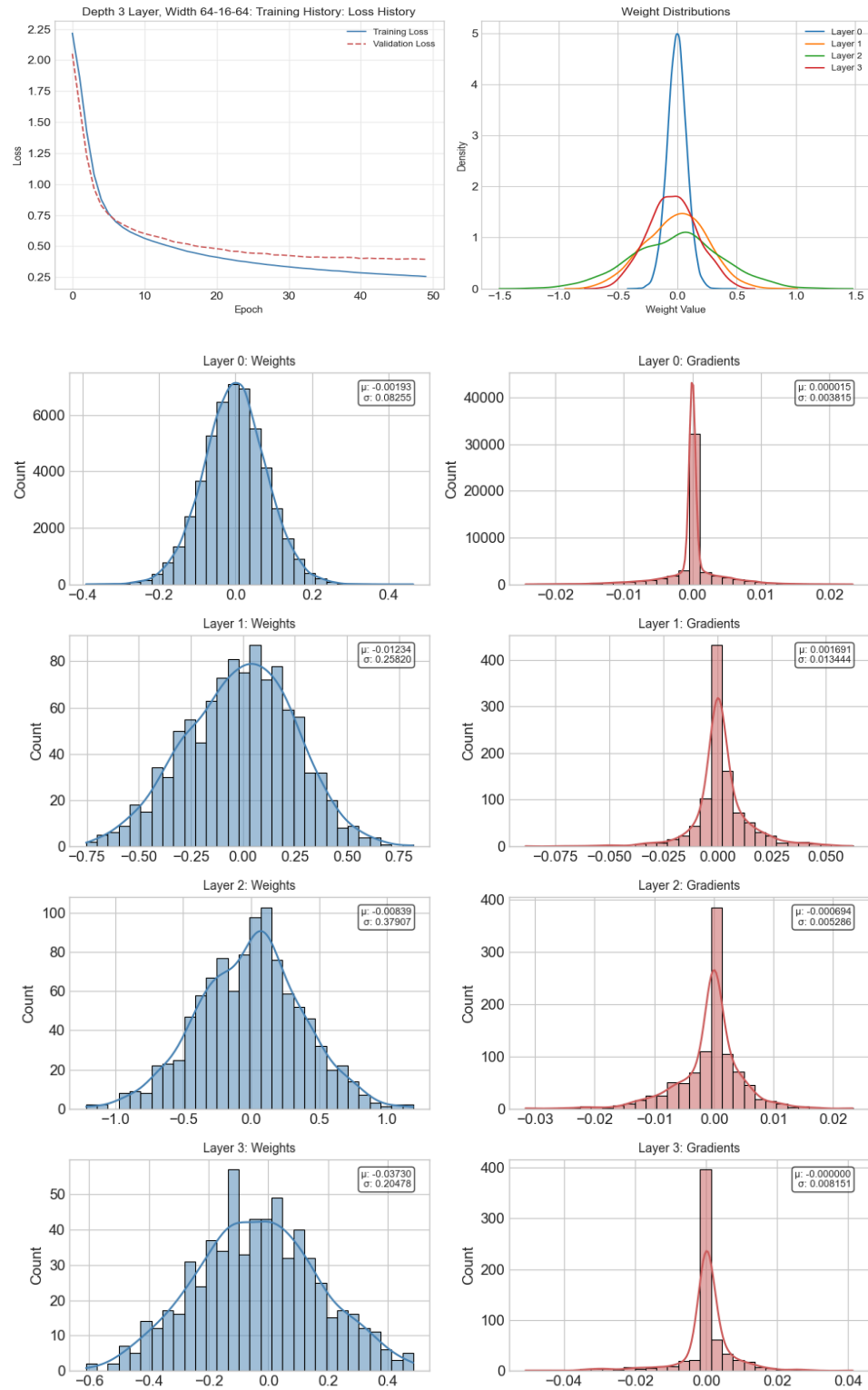
Hasil dan Pembahasan

4.1 Layers

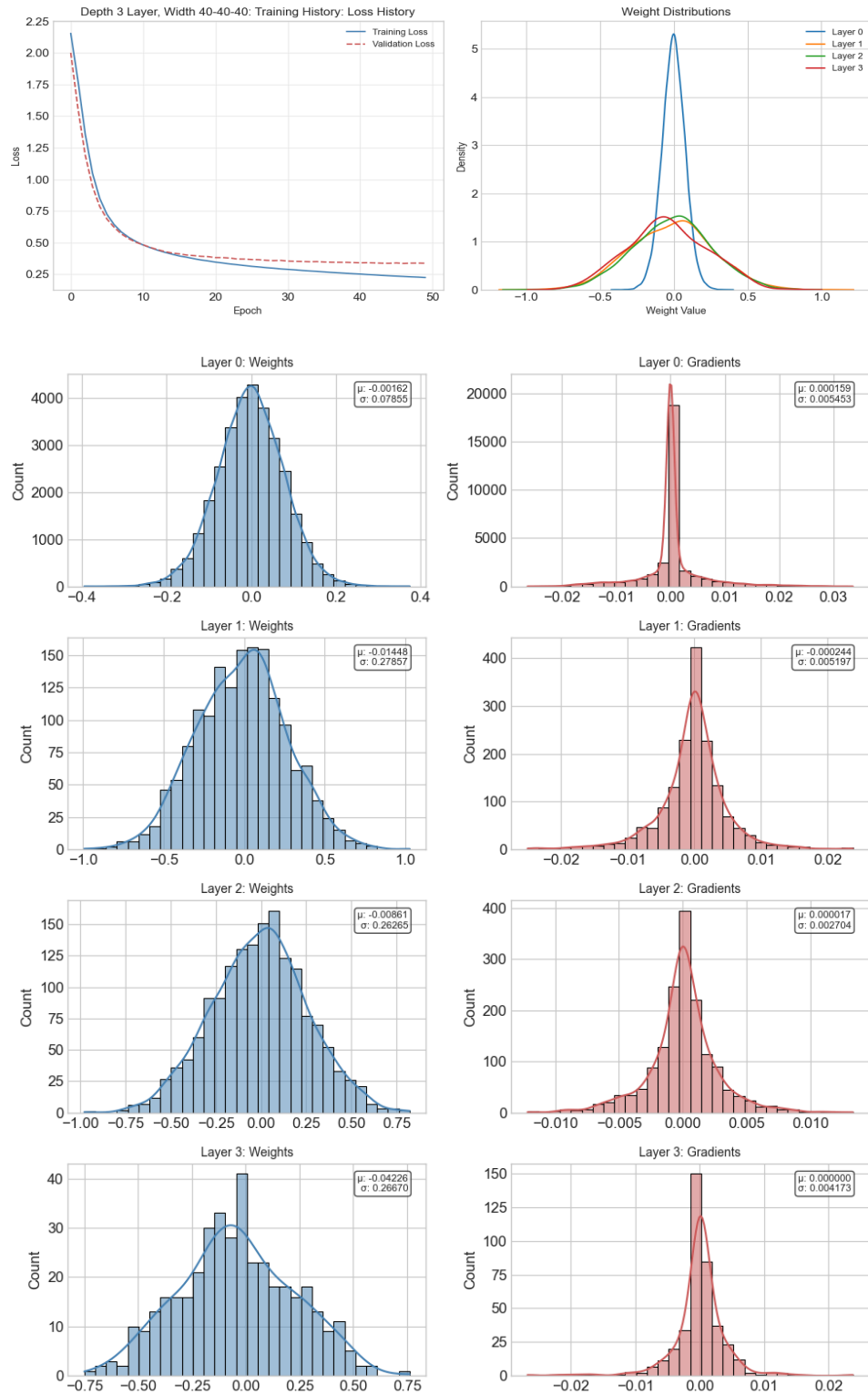
4.1.1 Depth1Width1



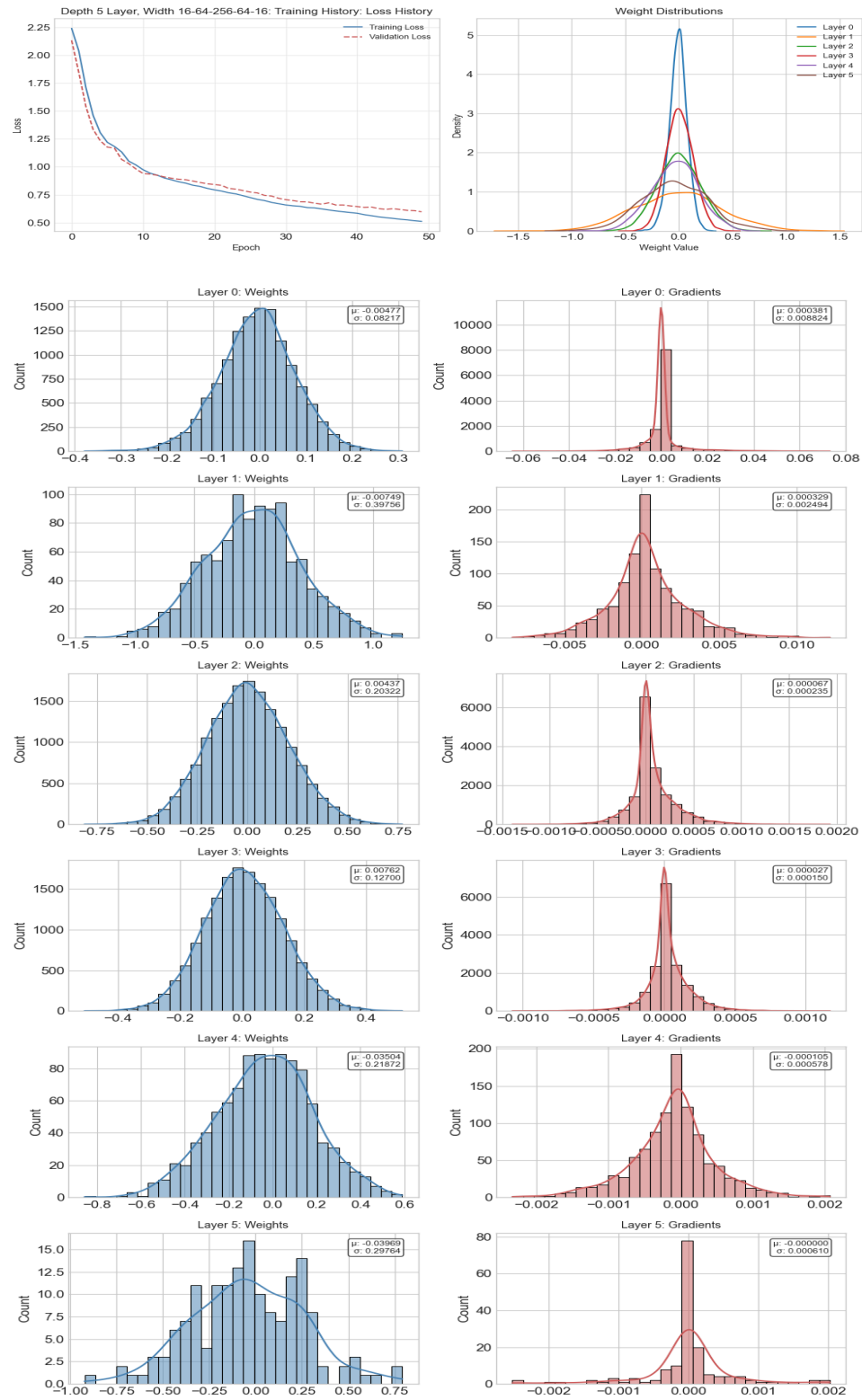
4.1.2 Depth 1 Width 2



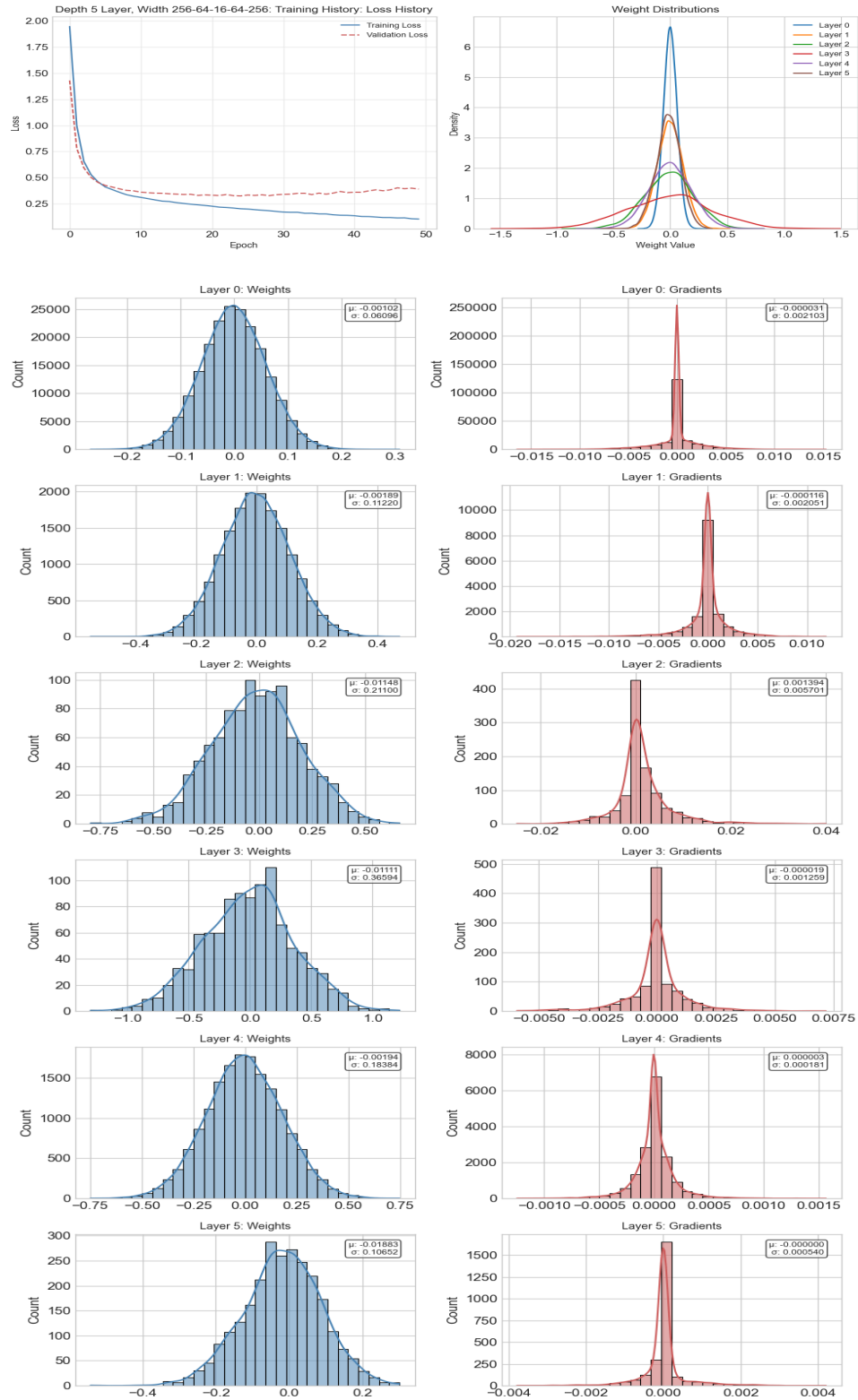
4.1.3 Depth 1 Width 3



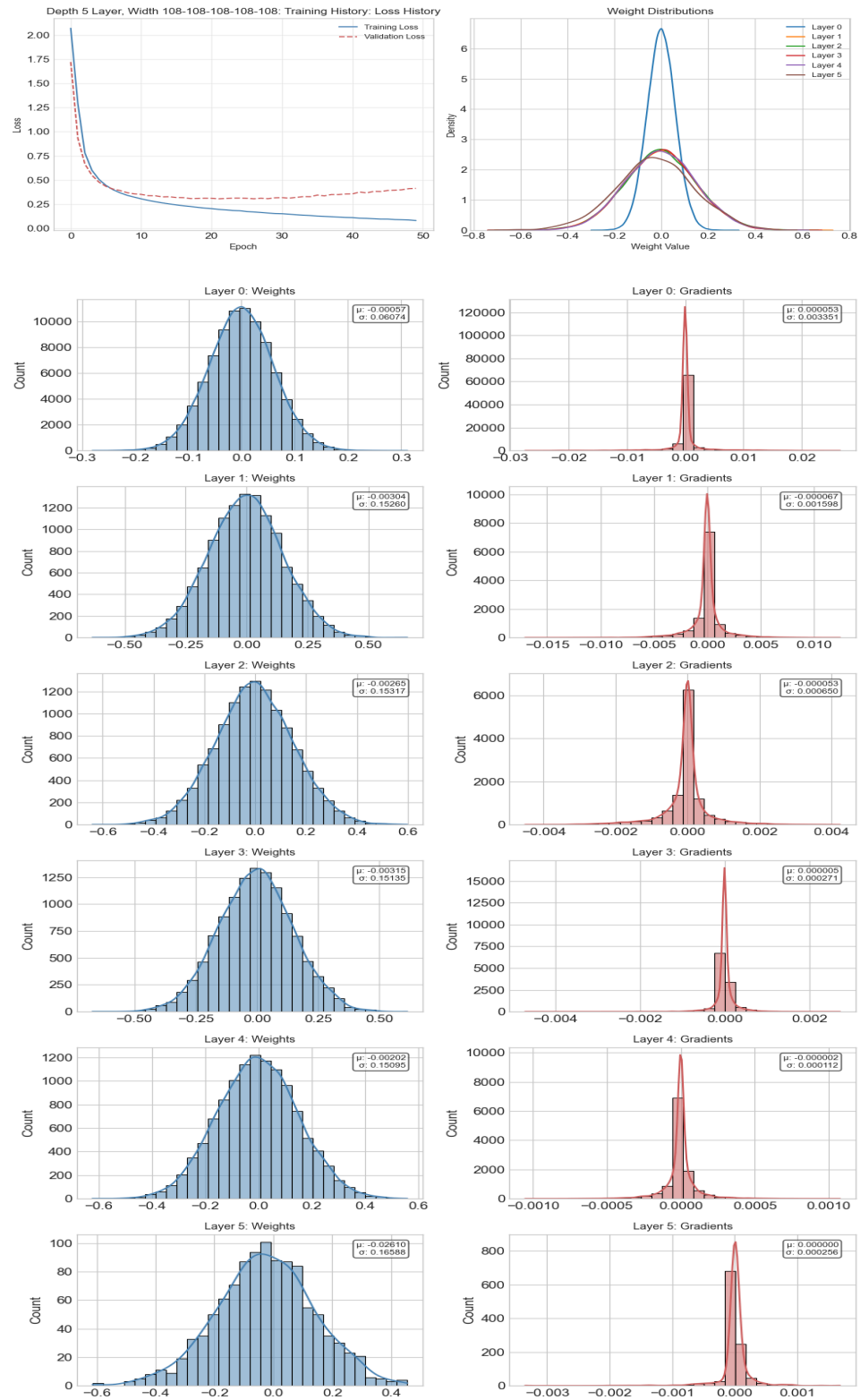
4.1.4 Depth 2 Width 1



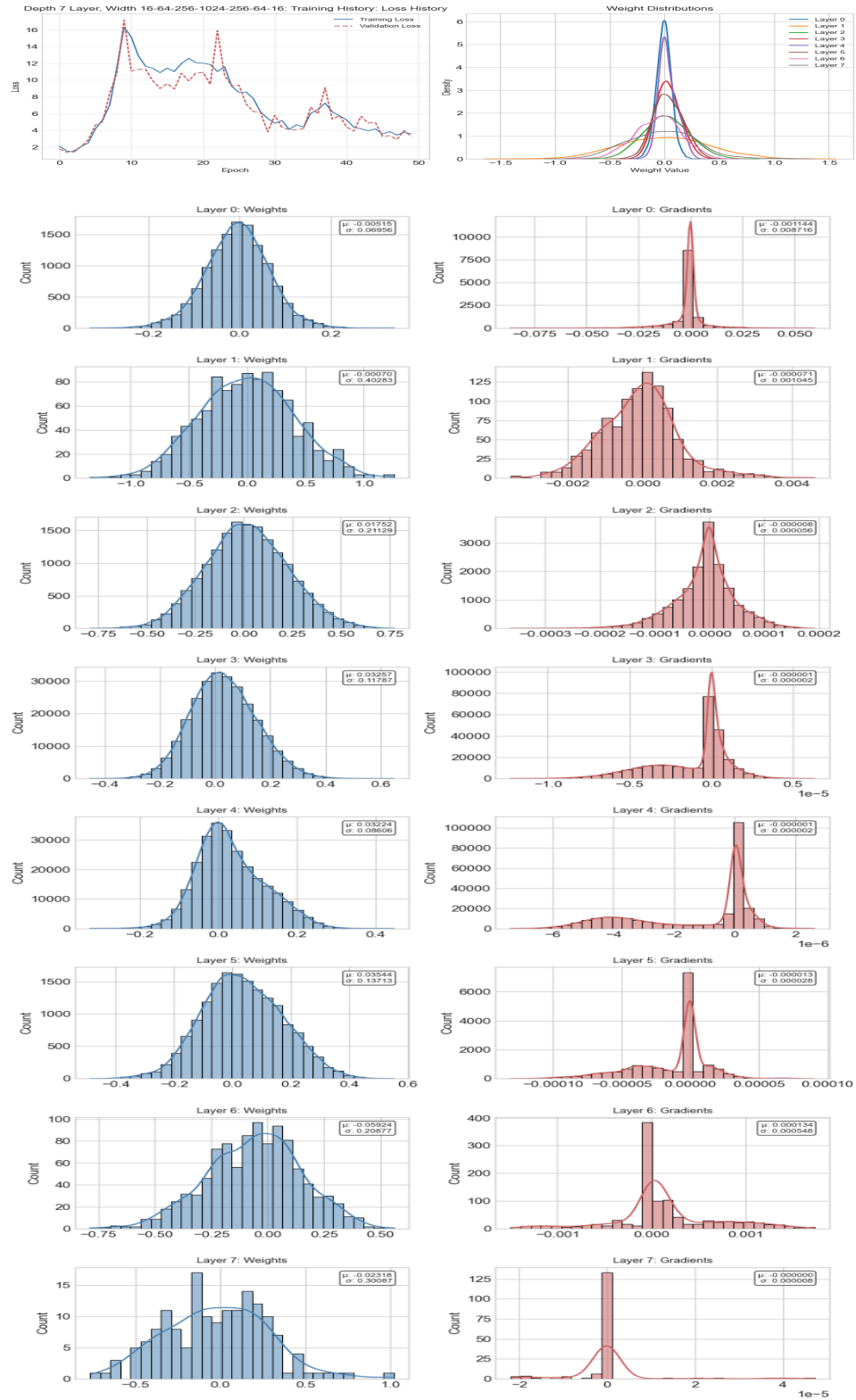
4.1.5 Depth 2 Width 2



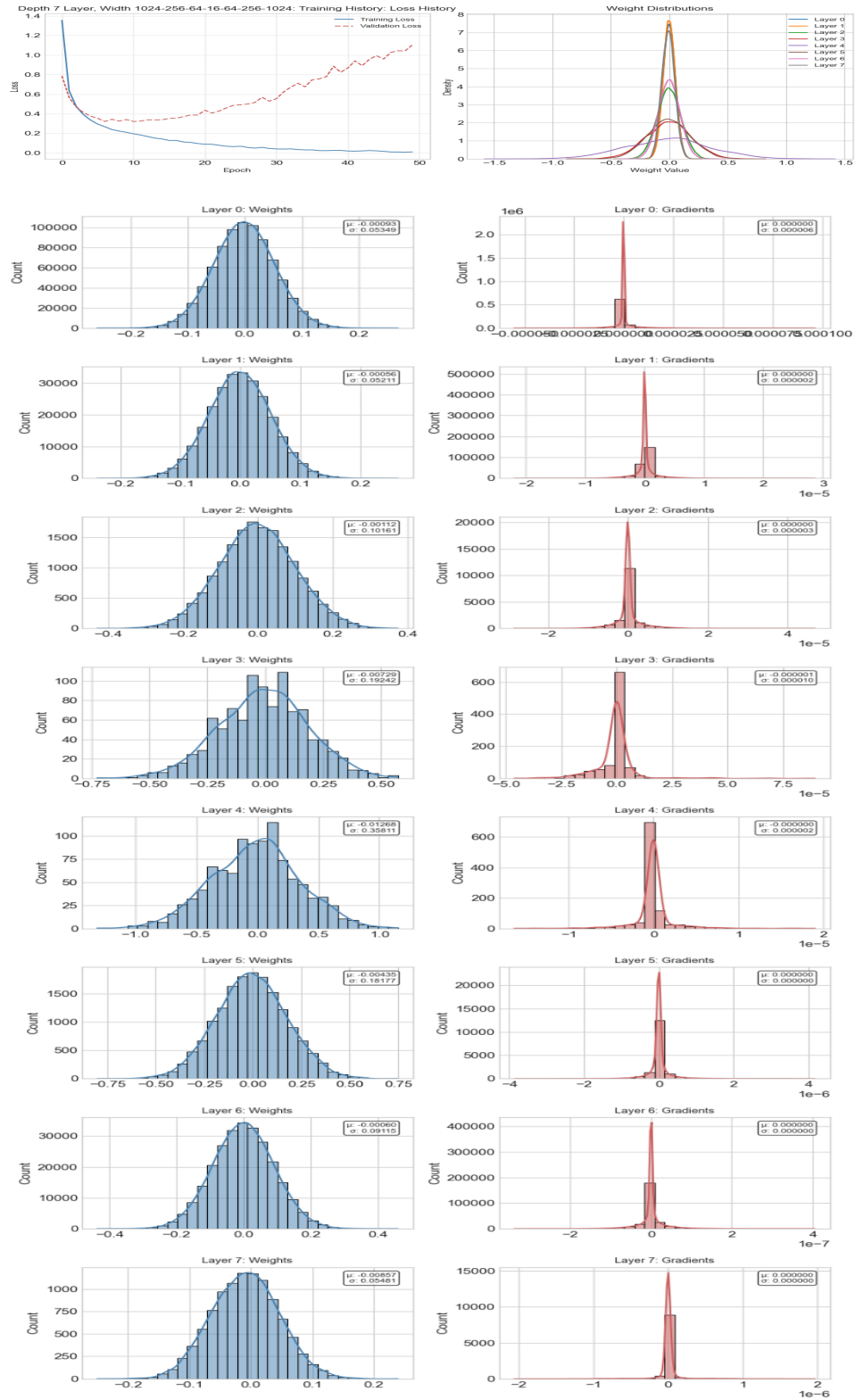
4.1.6 Depth 2 Width 3



4.1.7 Depth 3 Width 1

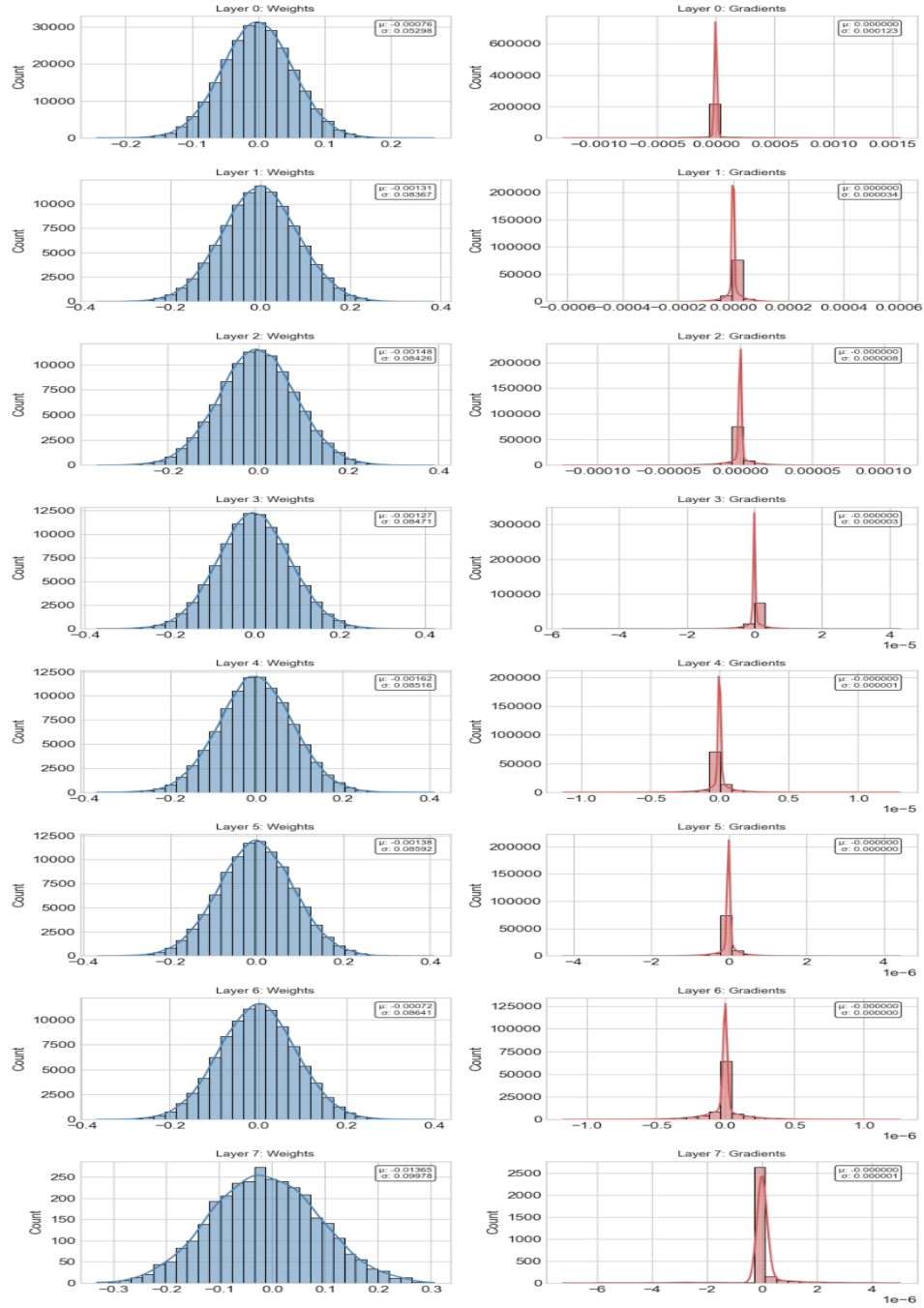


4.1.8 Depth 3 Width 2

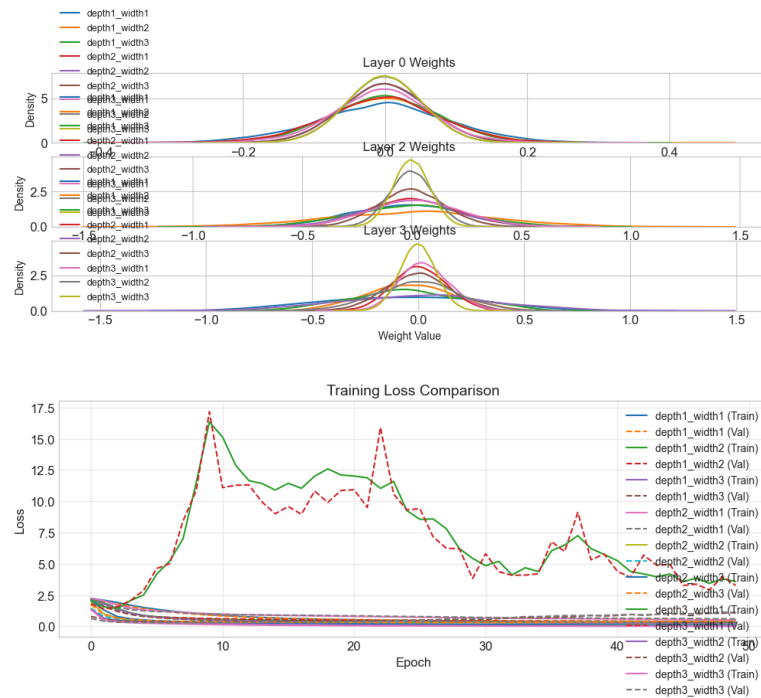


4.1.9 Depth 3 Width 3





4.1.10 Comparison Hidden Layer Depth and Width



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Hidden Layer Depth and Width, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

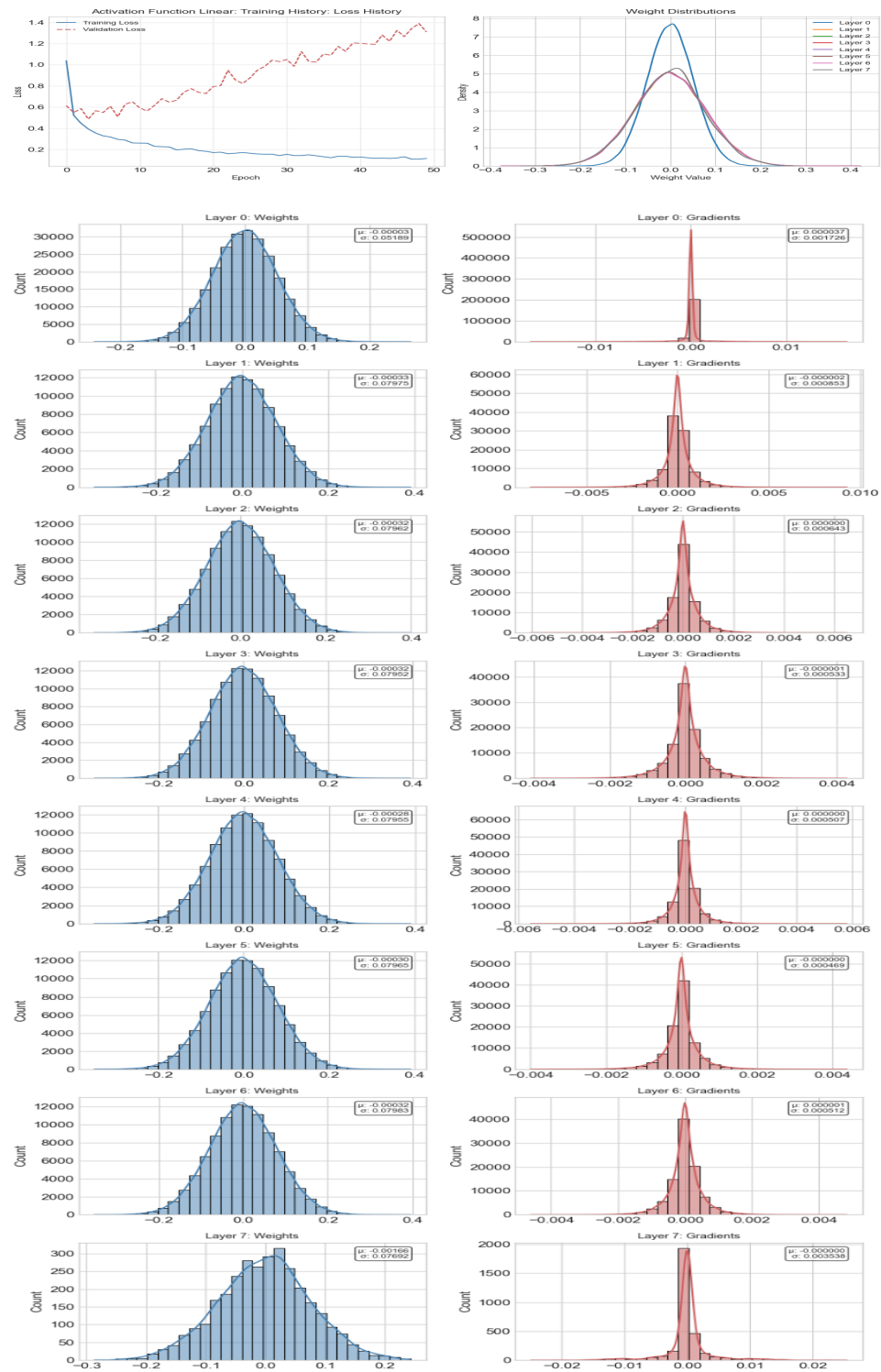
- | | | |
|-----------|-----------|-----------|
| 1. 0.8864 | 4. 0.8486 | 7. 0.4679 |
| 2. 0.9007 | 5. 0.9079 | 8. 0.9071 |
| 3. 0.9050 | 6. 0.9100 | 9. 0.9200 |

Berdasarkan seluruh gambar perbandingan Hidden Layer Depth and Width di atas, hasil menunjukkan bahwa:

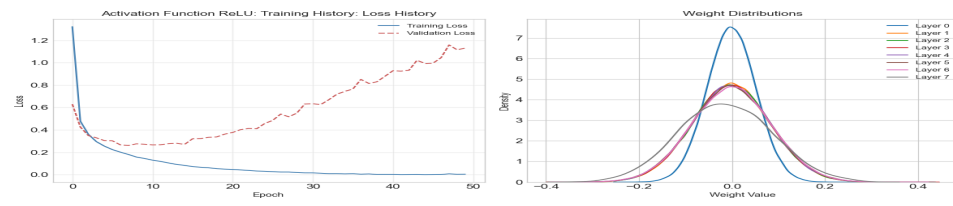
1. Model dengan kedalaman dan lebar optimal (Depth 3, Width 3) mencapai akurasi tertinggi
2. Peningkatan kedalaman memungkinkan model untuk mempelajari pola yang lebih kompleks
3. Peningkatan lebar meningkatkan kapasitas model untuk merepresentasikan fitur

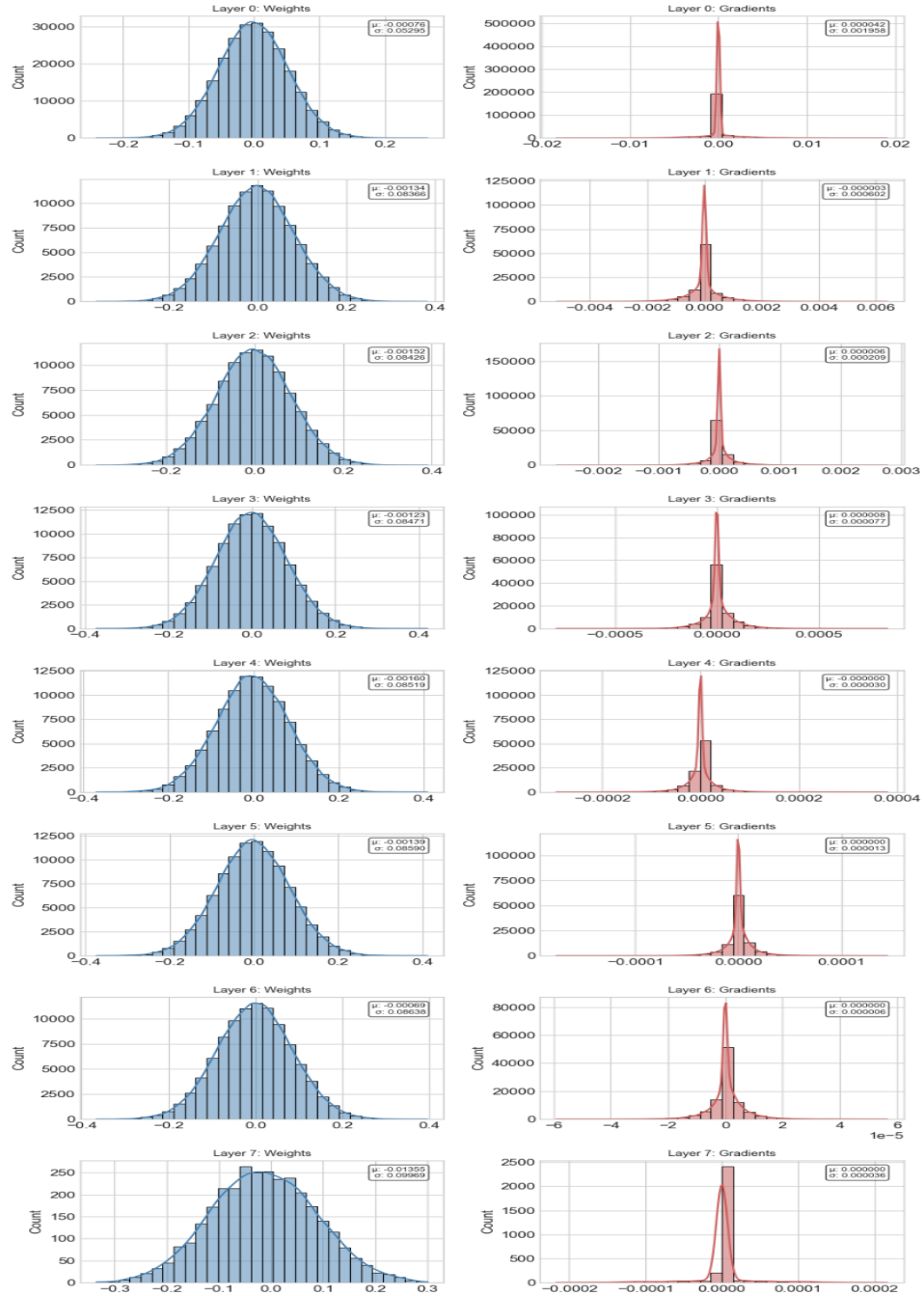
4.2 Hidden Layer Activations Function

4.2.1 Linear

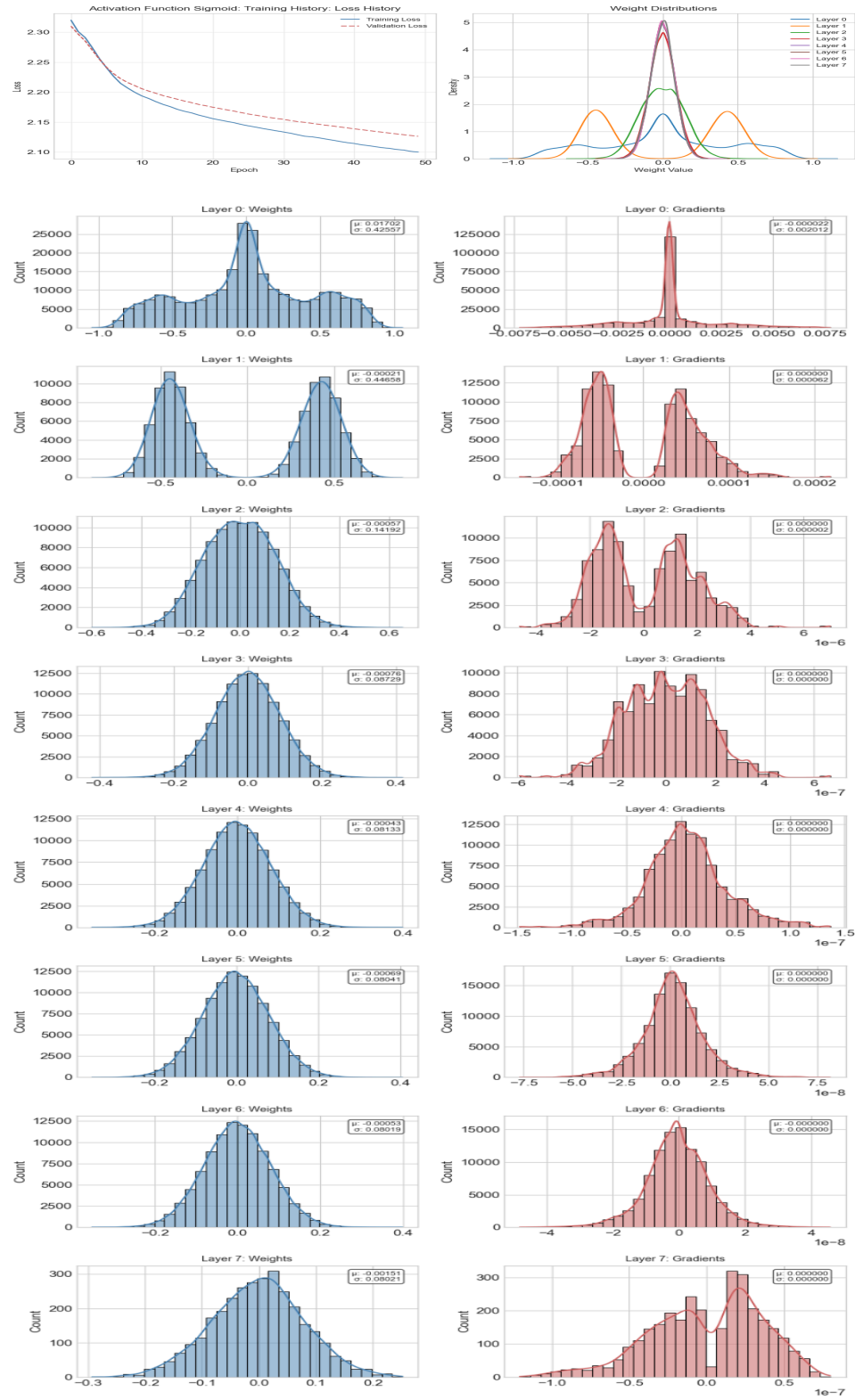


4.2.2 ReLU

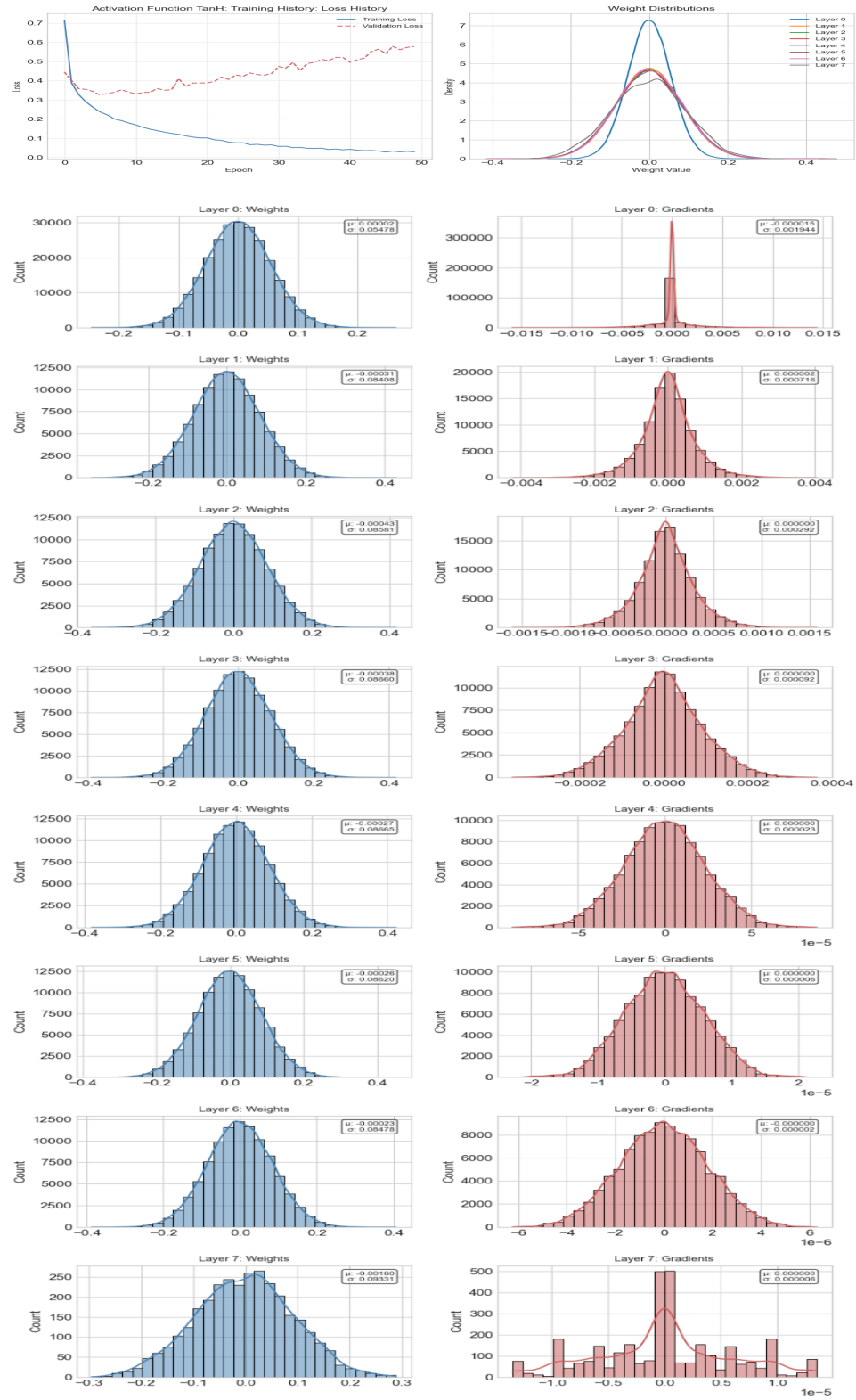




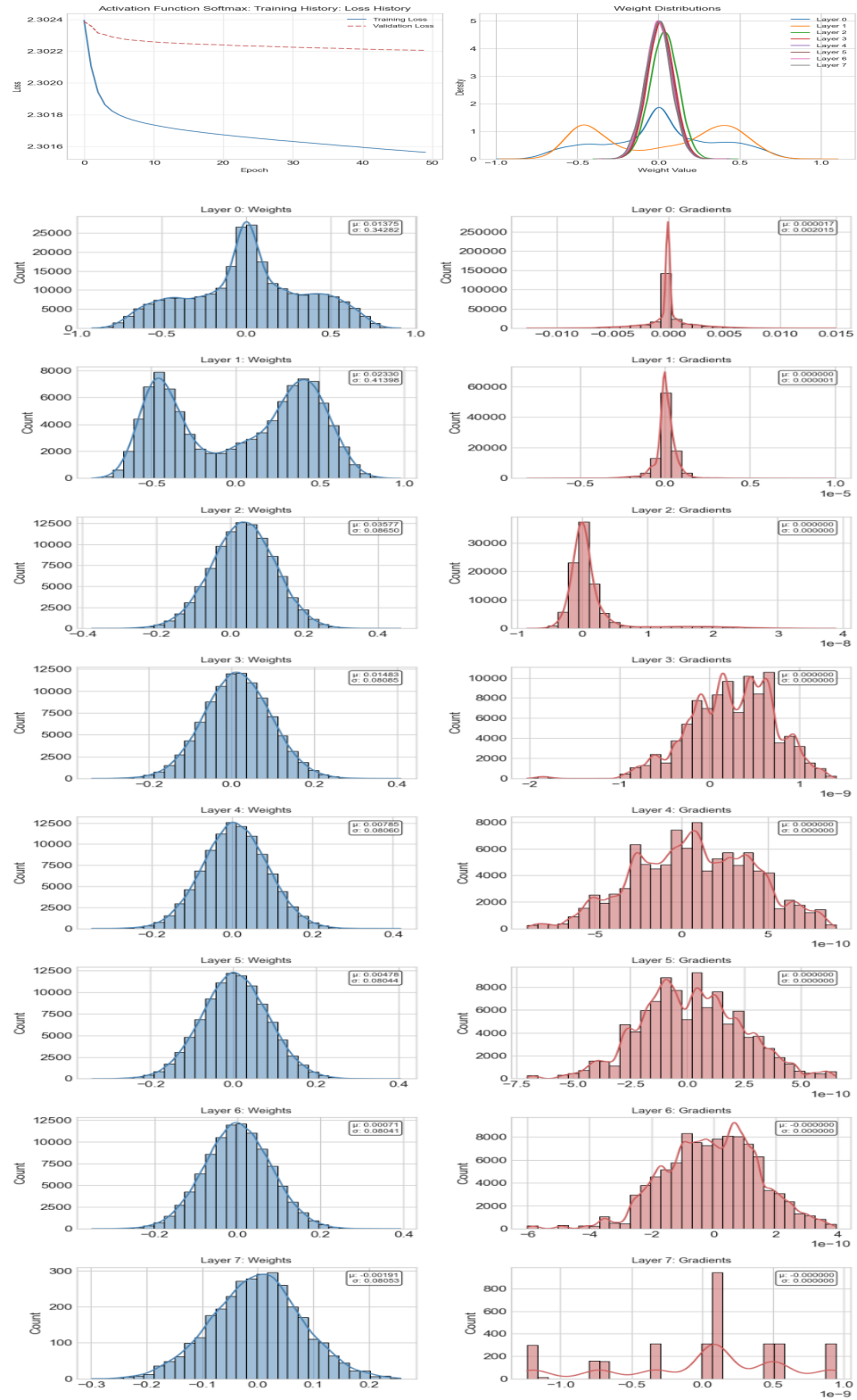
4.2.3 Sigmoid



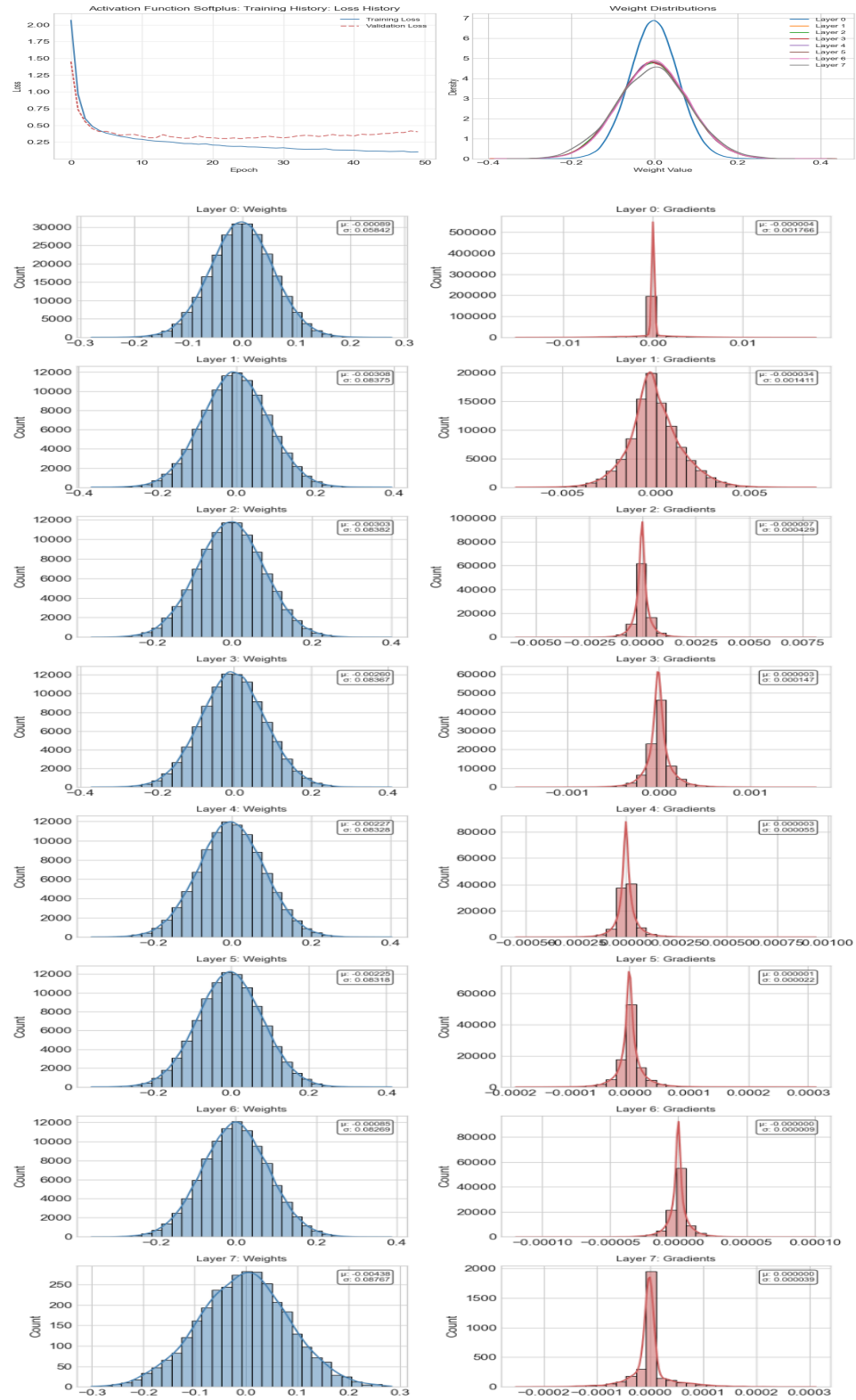
4.2.4 TanH



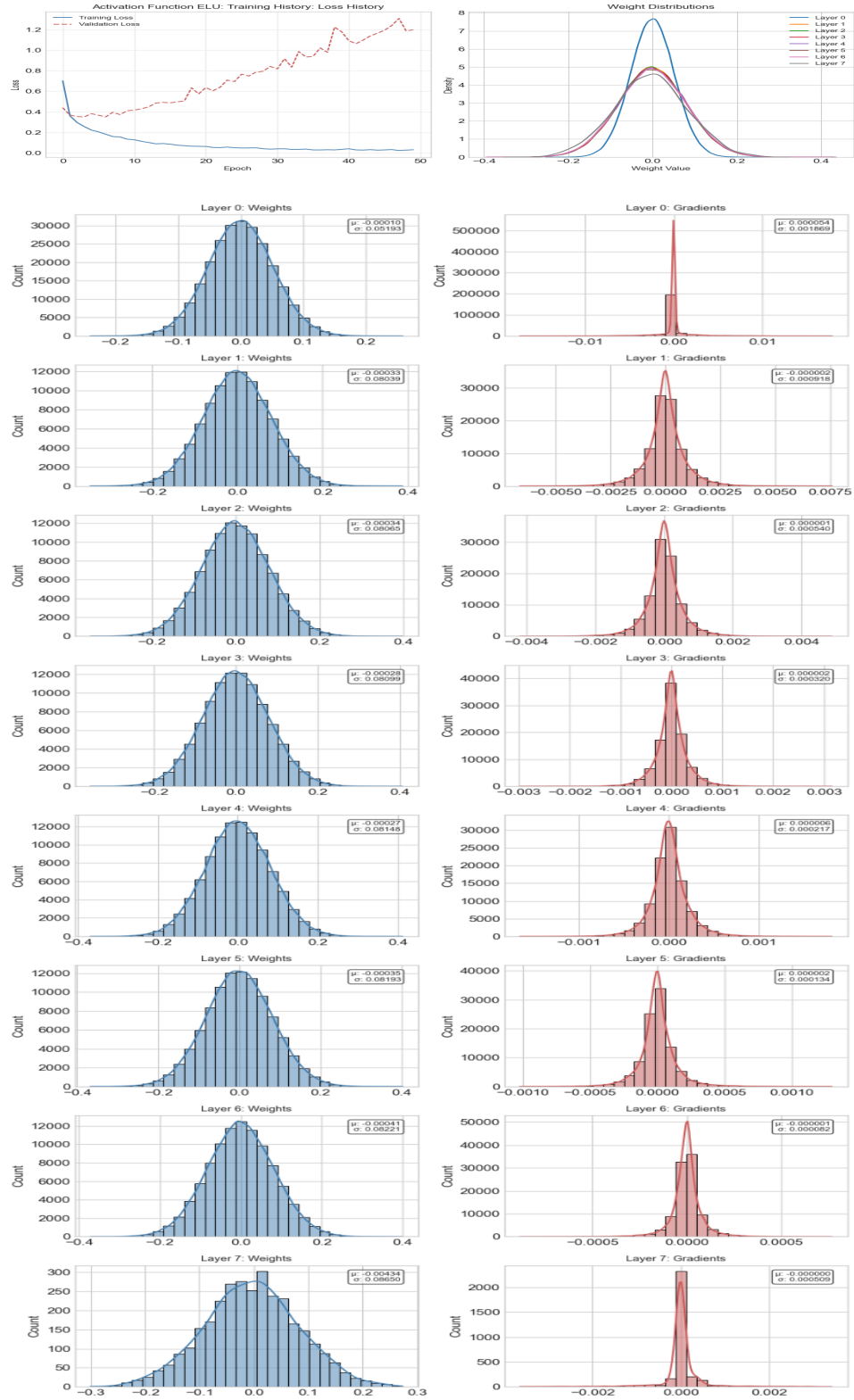
4.2.5 Softmax



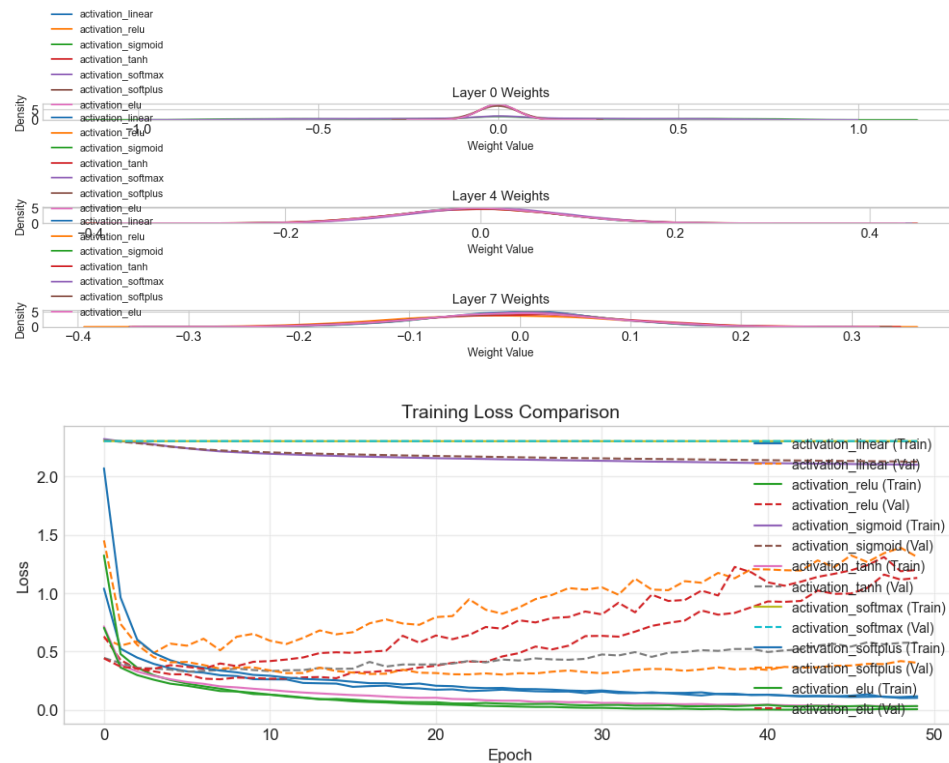
4.2.6 Softplus



4.2.7 ELU



4.2.8 Comparison Hidden Layer Activation Function



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Hidden Layer Activation Function, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

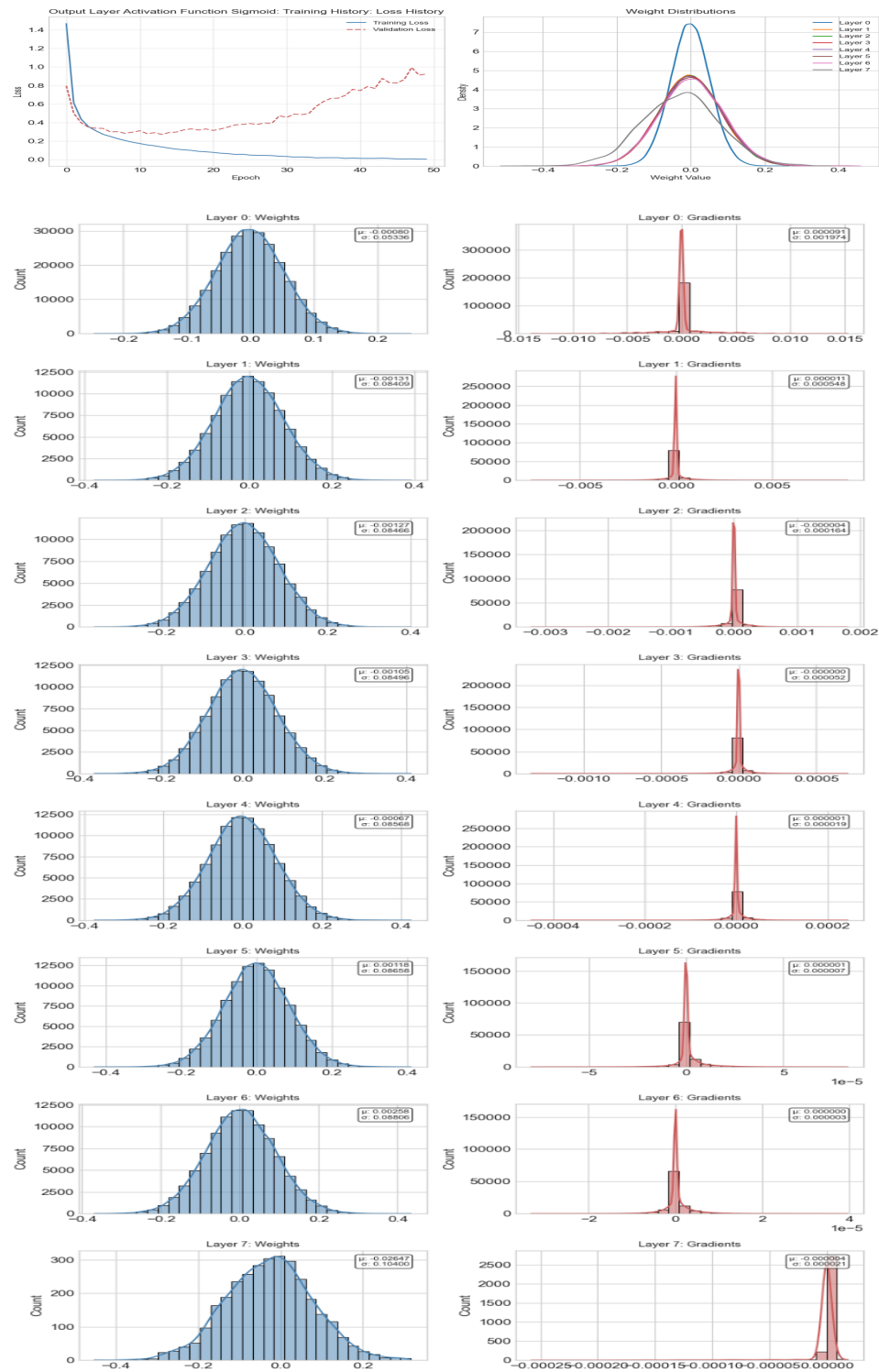
- | | | |
|-----------|-----------|-----------|
| 1. 0.8786 | 4. 0.9064 | 7. 0.9014 |
| 2. 0.9193 | 5. 0.1029 | |
| 3. 0.1743 | 6. 0.9179 | |

Berdasarkan seluruh gambar Hidden Layer Activation Function di atas, hasil menunjukkan bahwa:

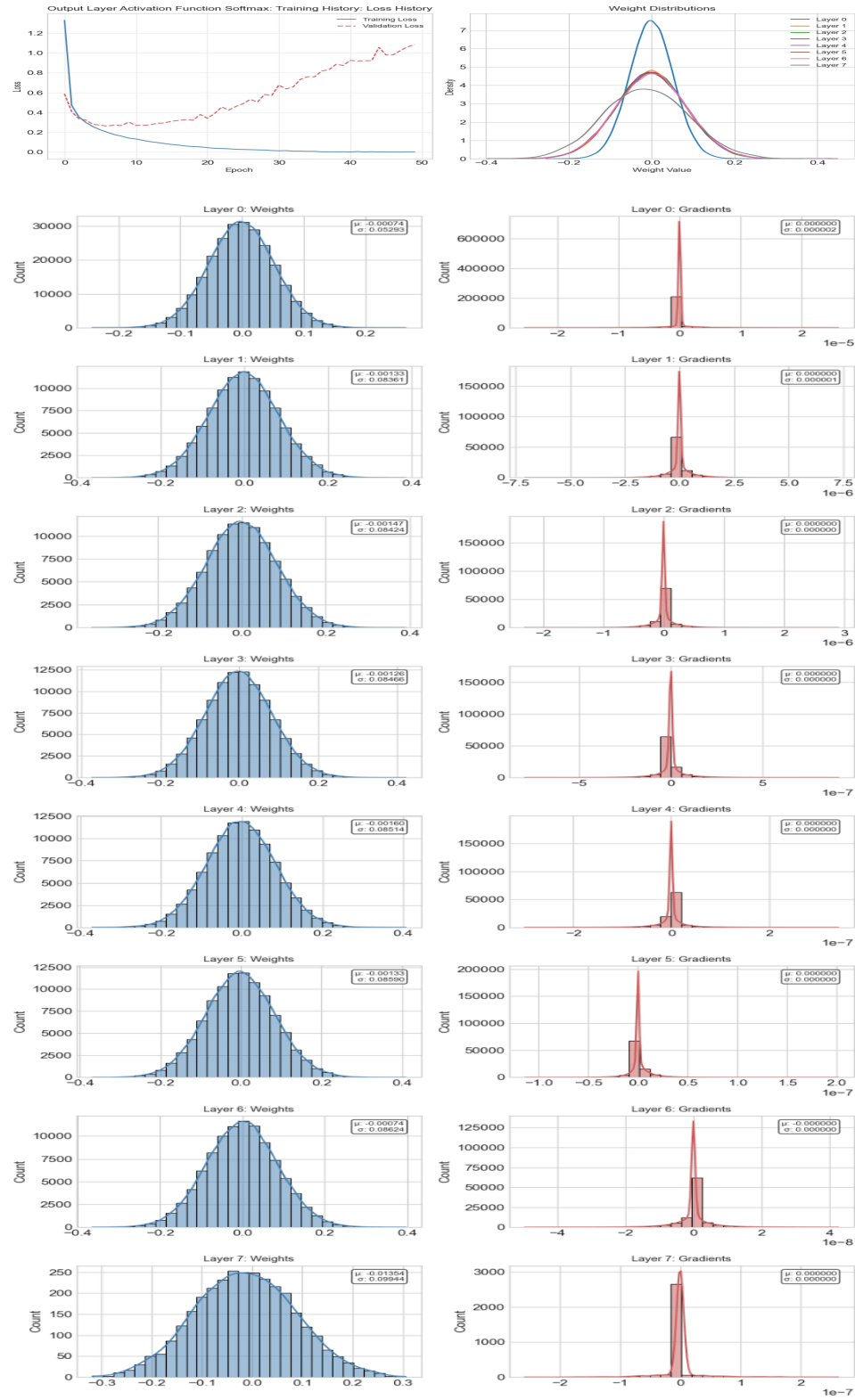
1. ReLU memberikan performa terbaik dengan konvergensi tercepat
2. Sigmoid dan TanH mengalami masalah vanishing gradient pada jaringan dalam
3. ELU menunjukkan performa yang baik tetapi tidak sebaik ReLU

4.3 Output Layer Activation Function

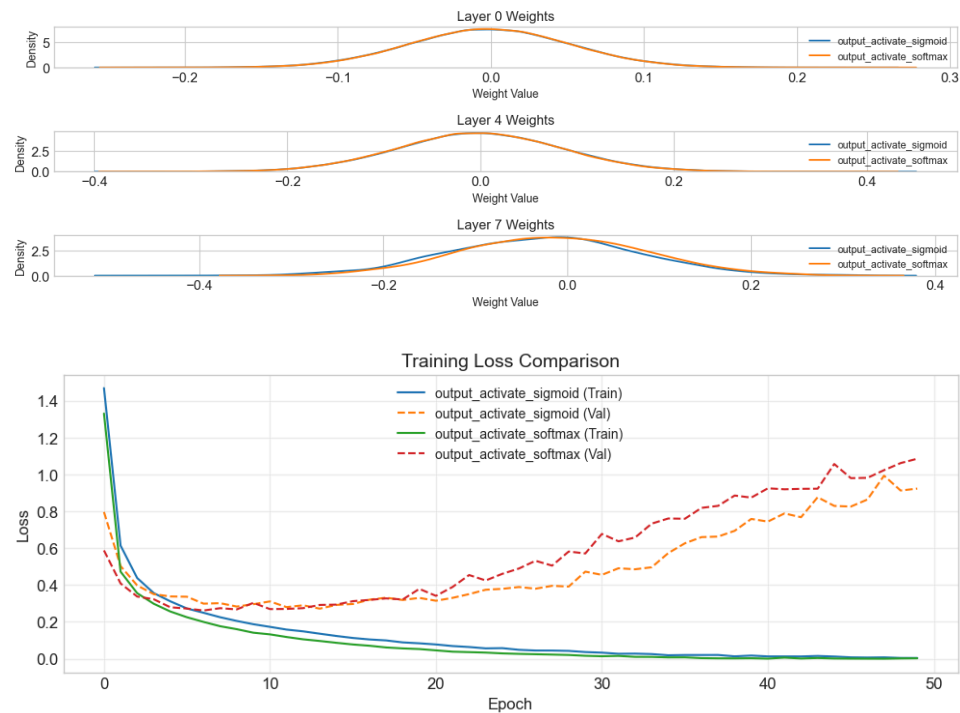
4.3.1 Sigmoid



4.3.2 Softmax



4.3.3 Comparison Output Layer Activation Function



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Output Layer Activation Function, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

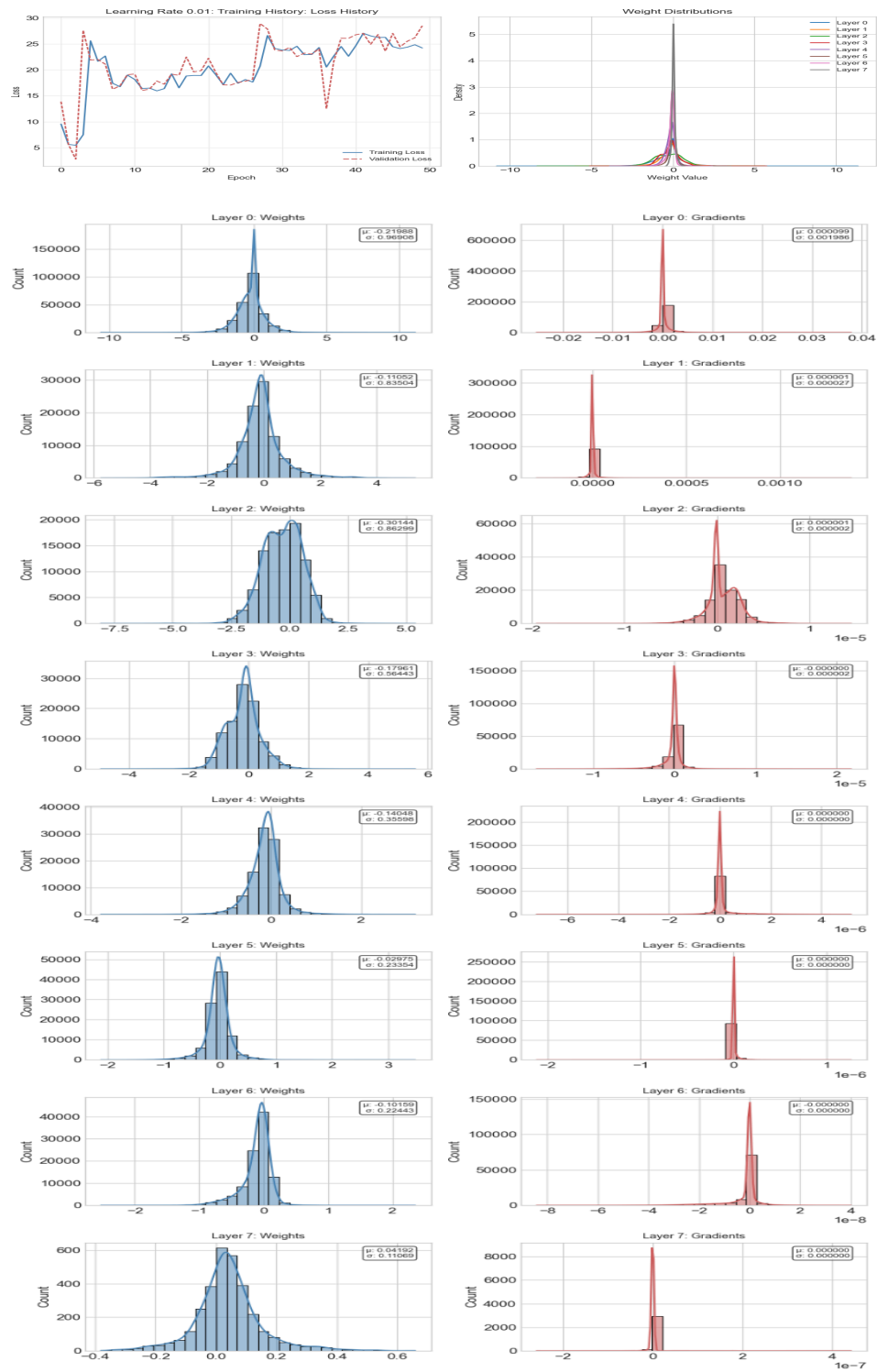
1. 0.9186
2. 0.9200

Berdasarkan seluruh gambar Output Layer Activation Function di atas, hasil menunjukkan bahwa:

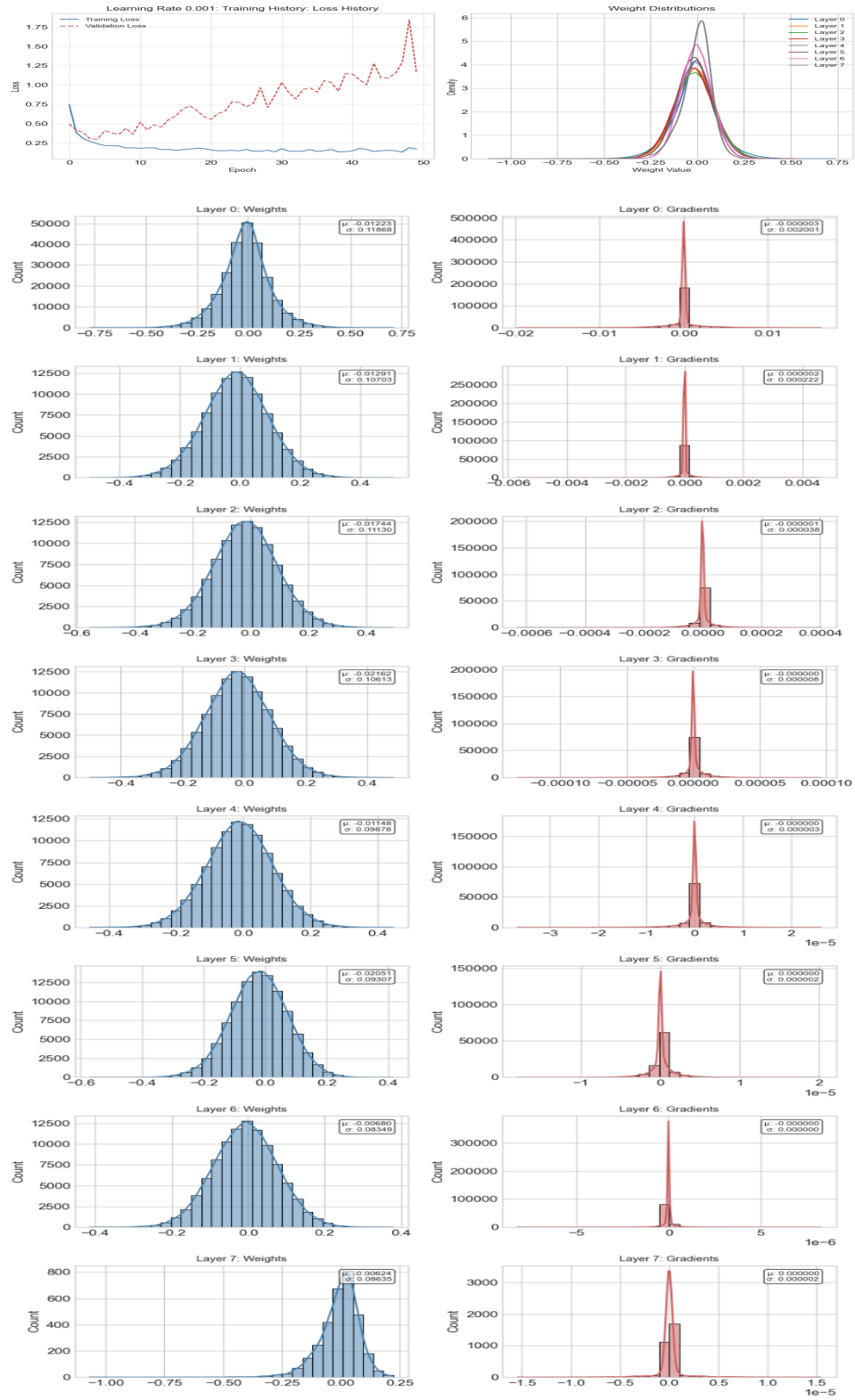
1. Softmax memberikan performa lebih baik untuk klasifikasi multi-kelas seperti MNIST
2. Sigmoid kurang optimal karena tidak mempertimbangkan hubungan antar kelas

4.4 Learning Rate

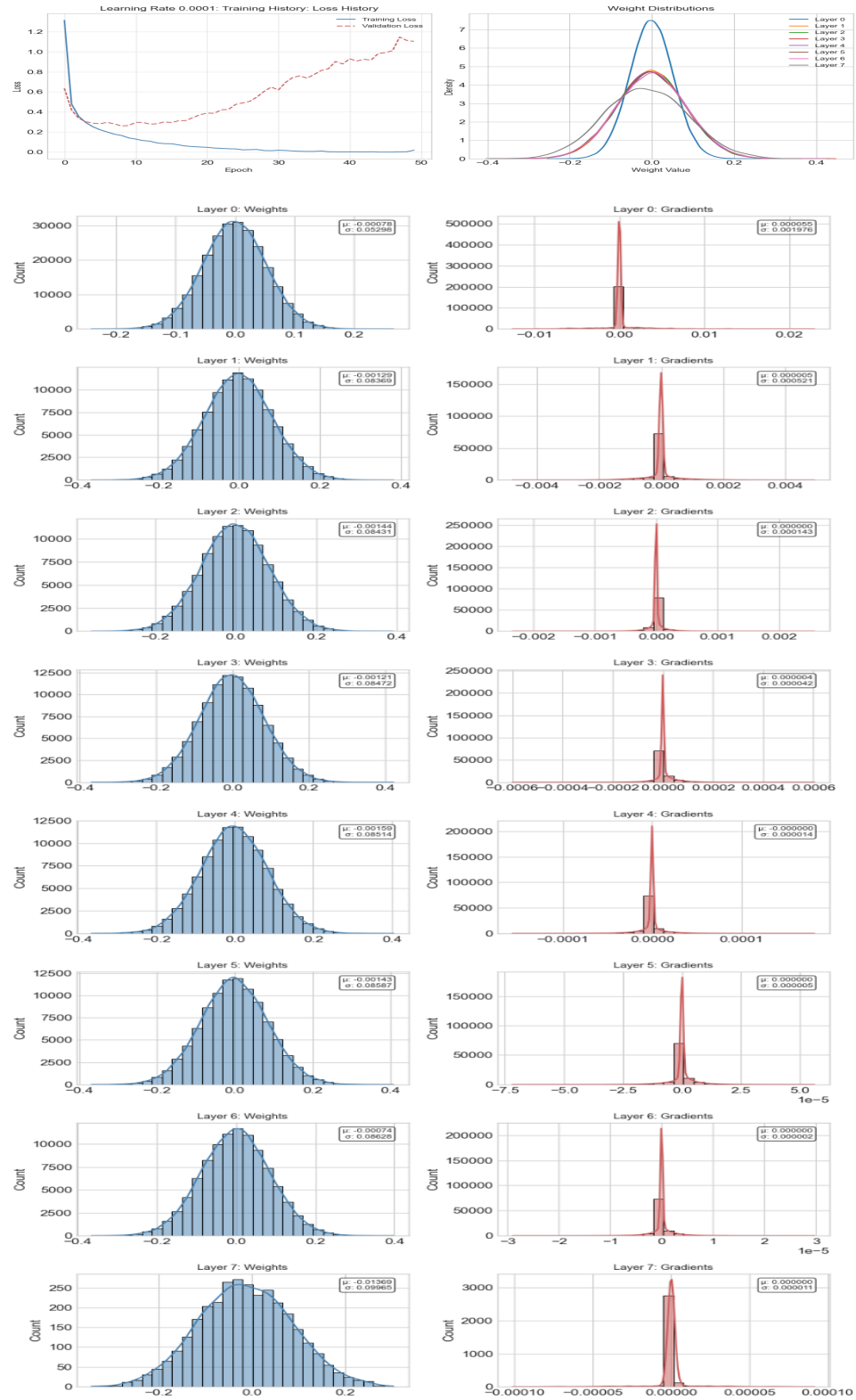
4.4.1 0.01



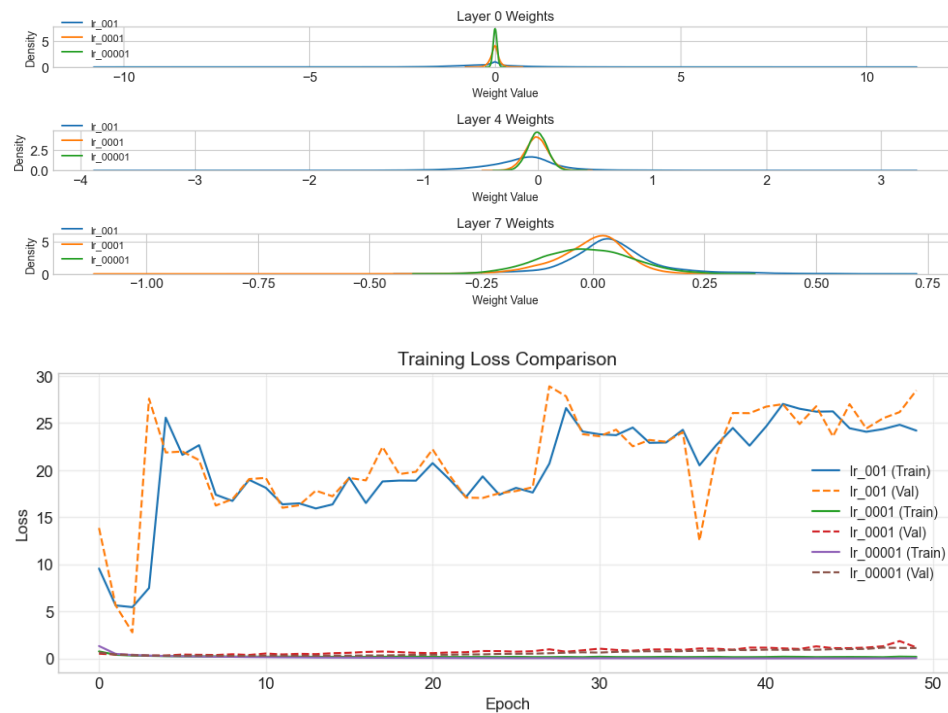
4.4.2 0.001



4.4.3 0.0001



4.4.4 Comparison Learning Rate



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Learning Rate, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

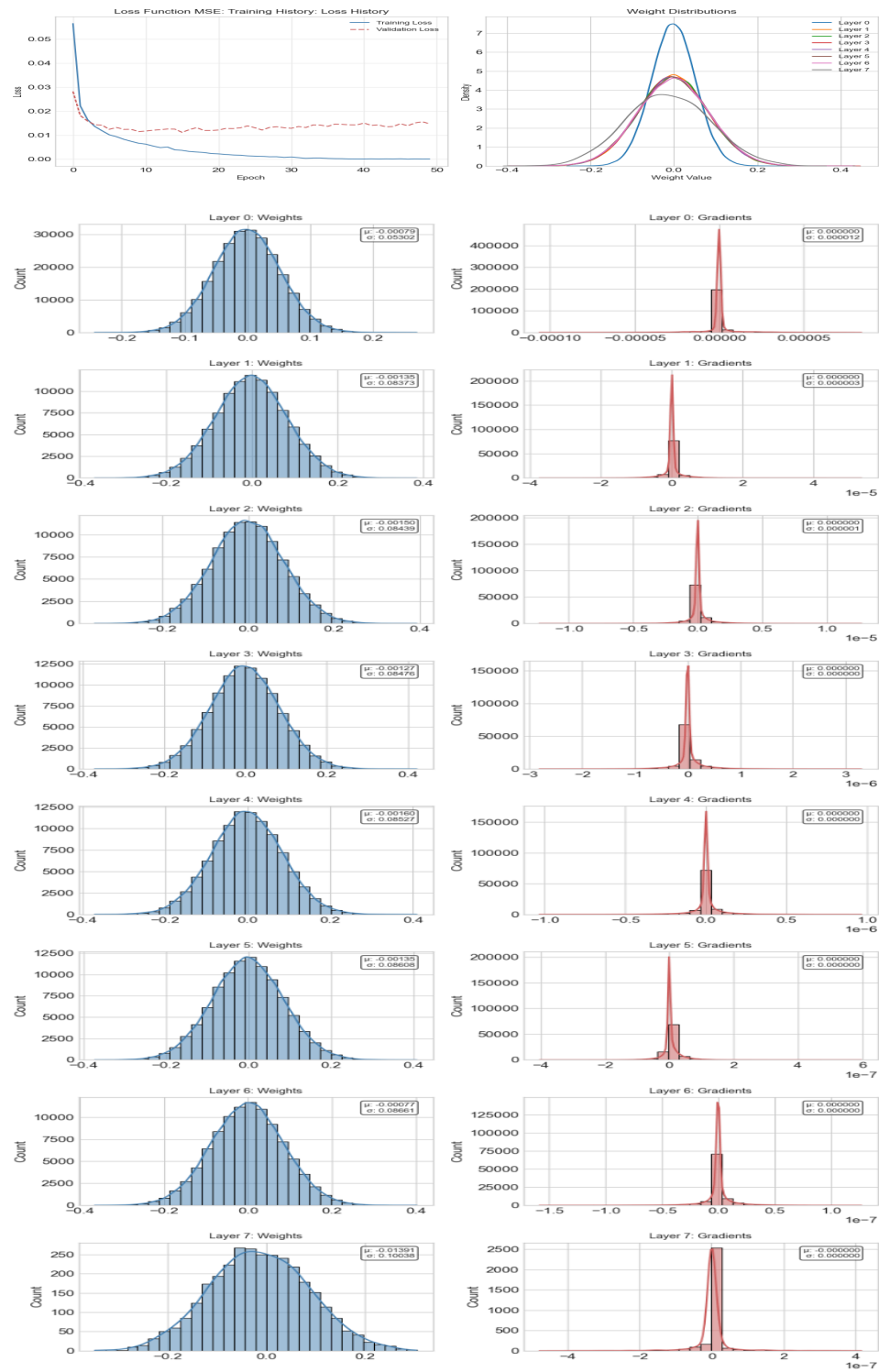
1. 0.1750
2. 0.8929
3. 0.9229

Berdasarkan seluruh gambar perbandingan Learning Rate di atas, hasil menunjukkan bahwa:

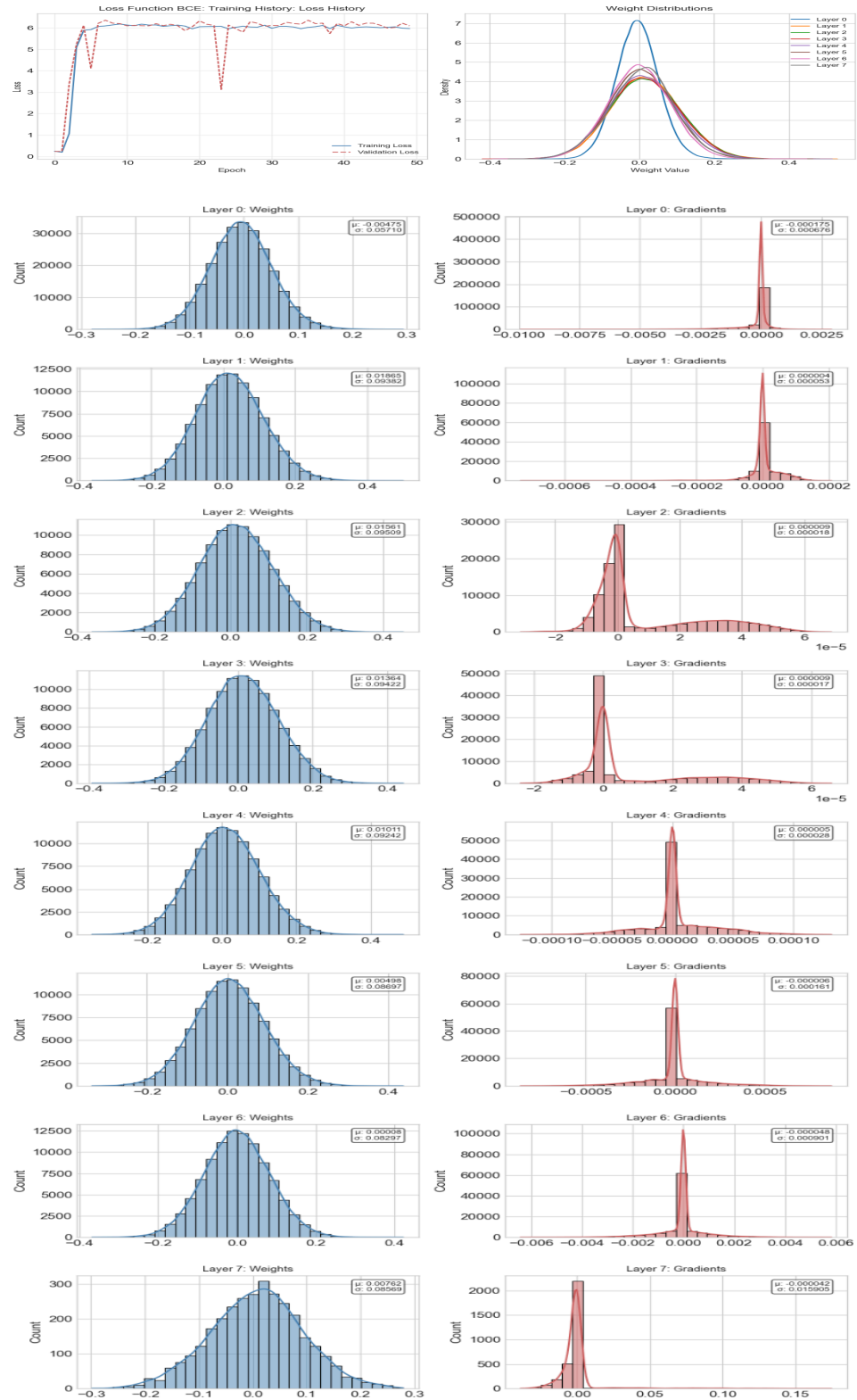
1. Learning rate 0.001 memberikan keseimbangan terbaik antara kecepatan konvergensi dan stabilitas
2. Learning rate 0.01 terlalu tinggi, menyebabkan osilasi dan kesulitan konvergensi
3. Learning rate 0.0001 terlalu lambat, membutuhkan lebih banyak epoch untuk konvergensi. Namun, memberikan hasil terbaik.

4.5 Loss Function

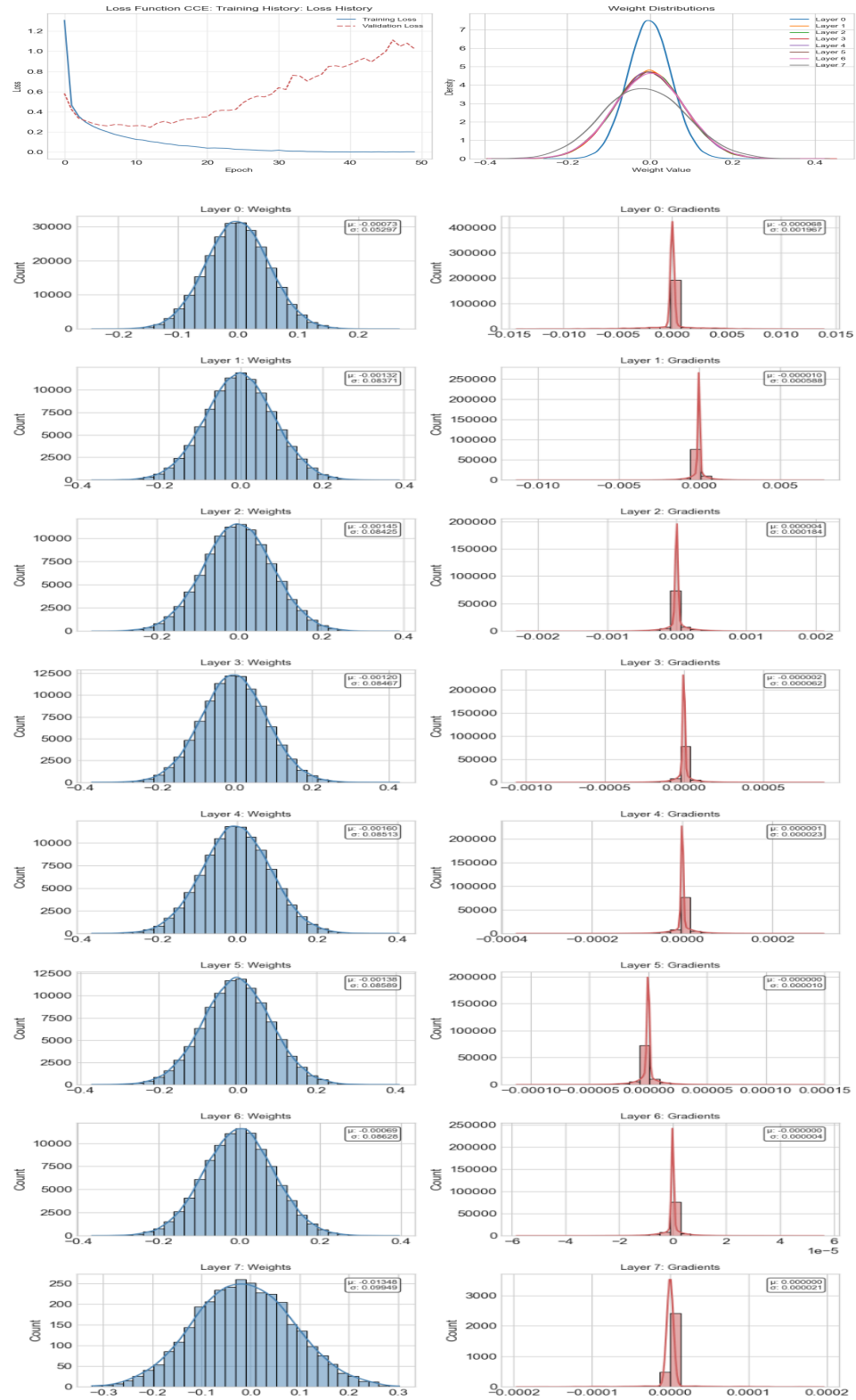
4.5.1 MSE



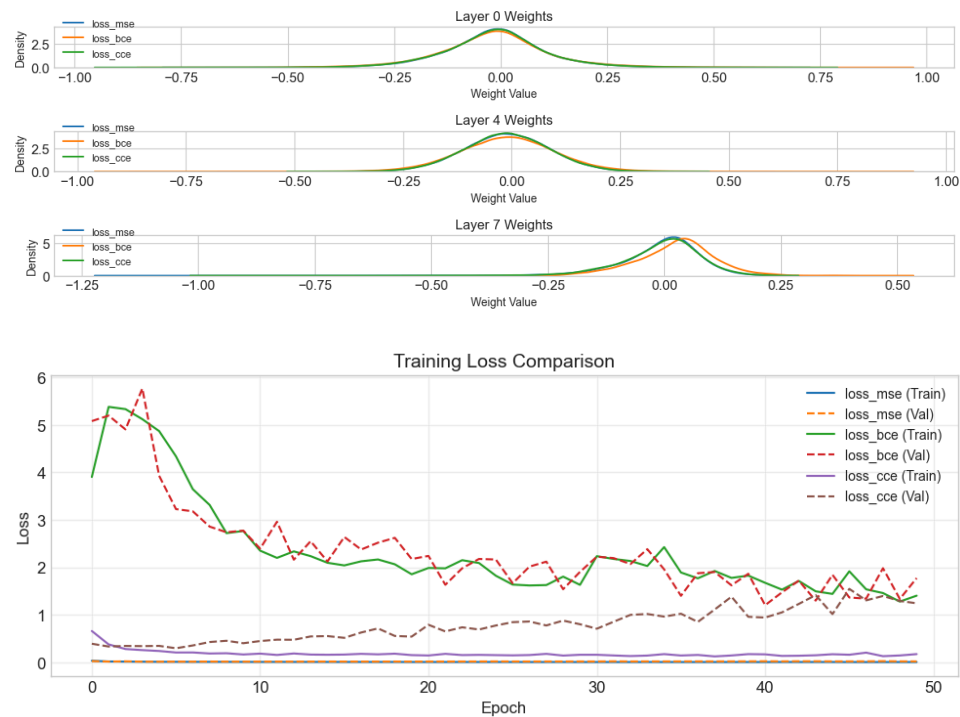
4.5.2 BCE



4.5.3 CCE



4.5.4 Comparison Loss Function



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Loss Function, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

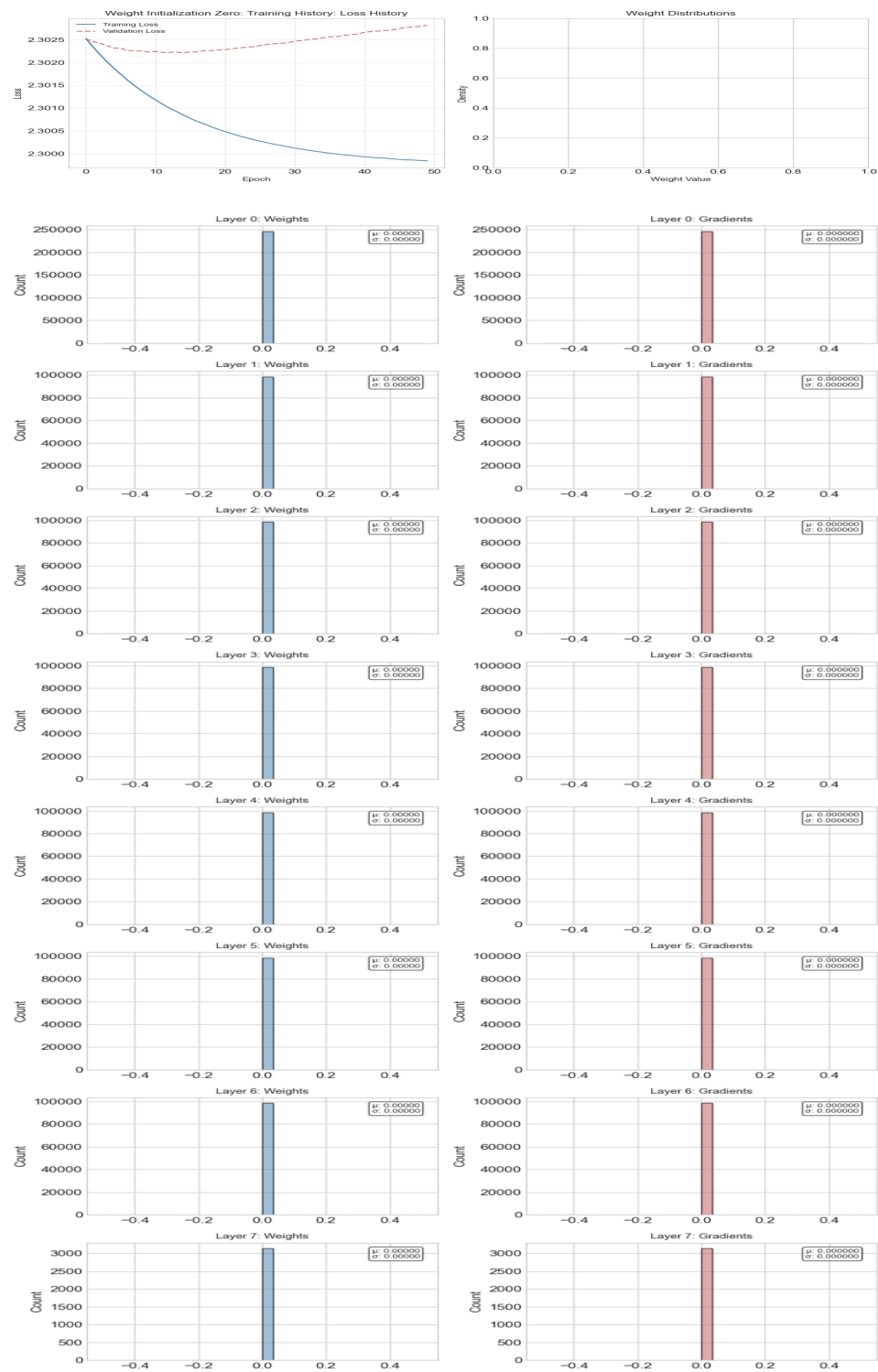
1. 0.9229
2. 0.1171
3. 0.9236

Berdasarkan seluruh gambar perbandingan Loss Function di atas, hasil menunjukkan bahwa:

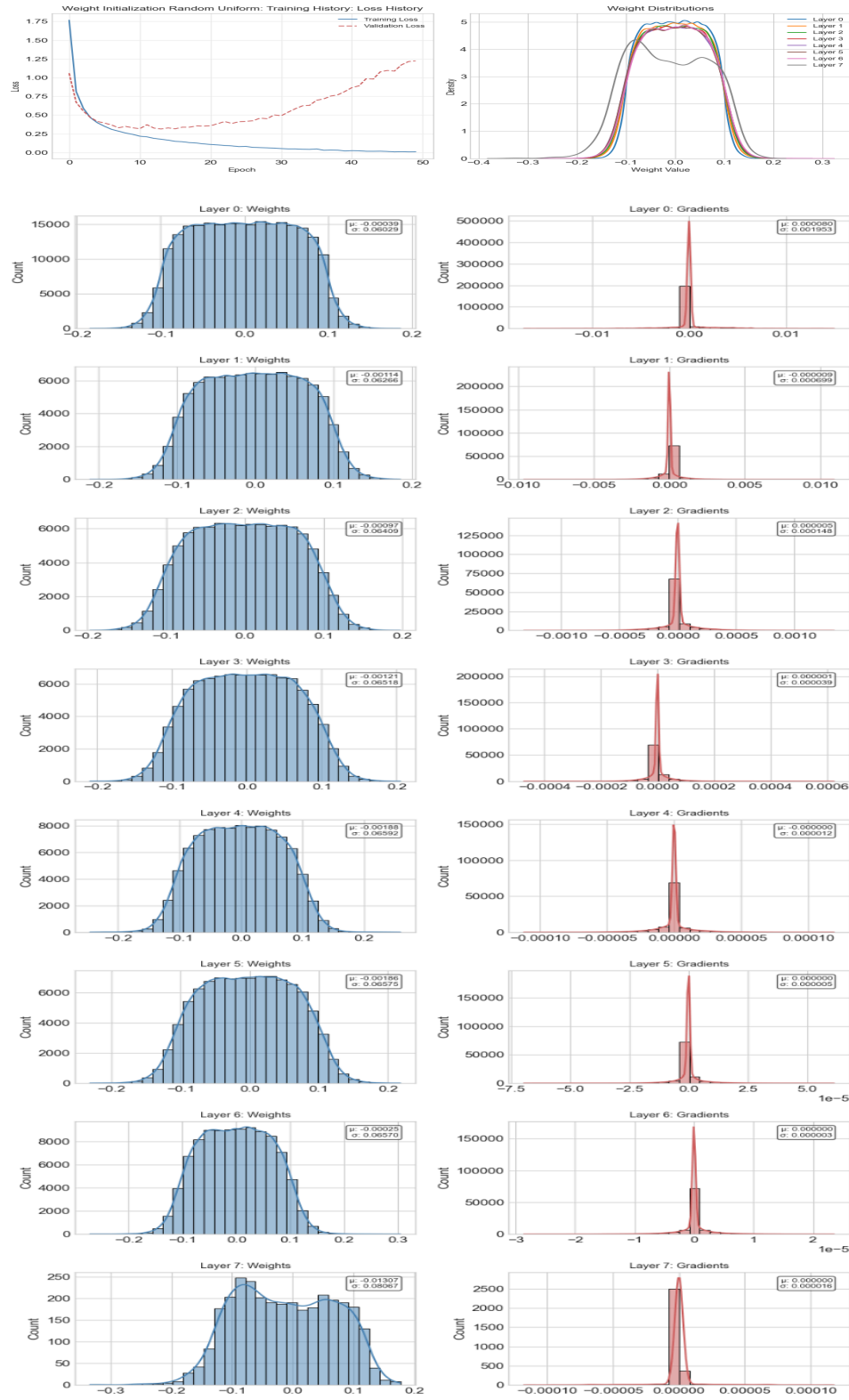
1. CCE memberikan performa terbaik untuk dataset MNIST, sesuai dengan ekspektasi untuk masalah klasifikasi multi-kelas
2. MSE kurang optimal untuk klasifikasi, konvergensi lebih lambat
3. BCE tidak tepat untuk masalah multi-kelas

4.6 Weight Initialization

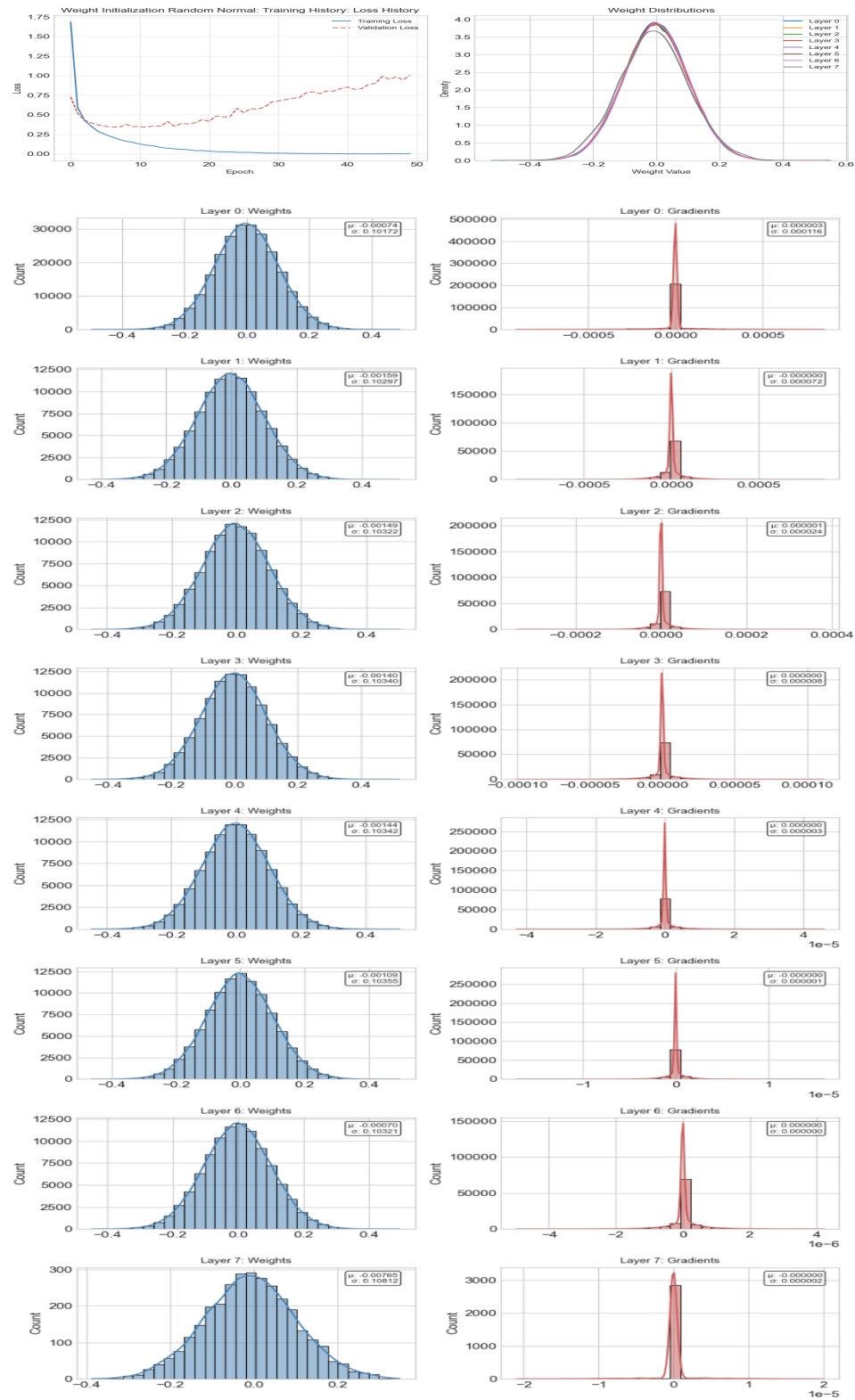
4.6.1 Zero



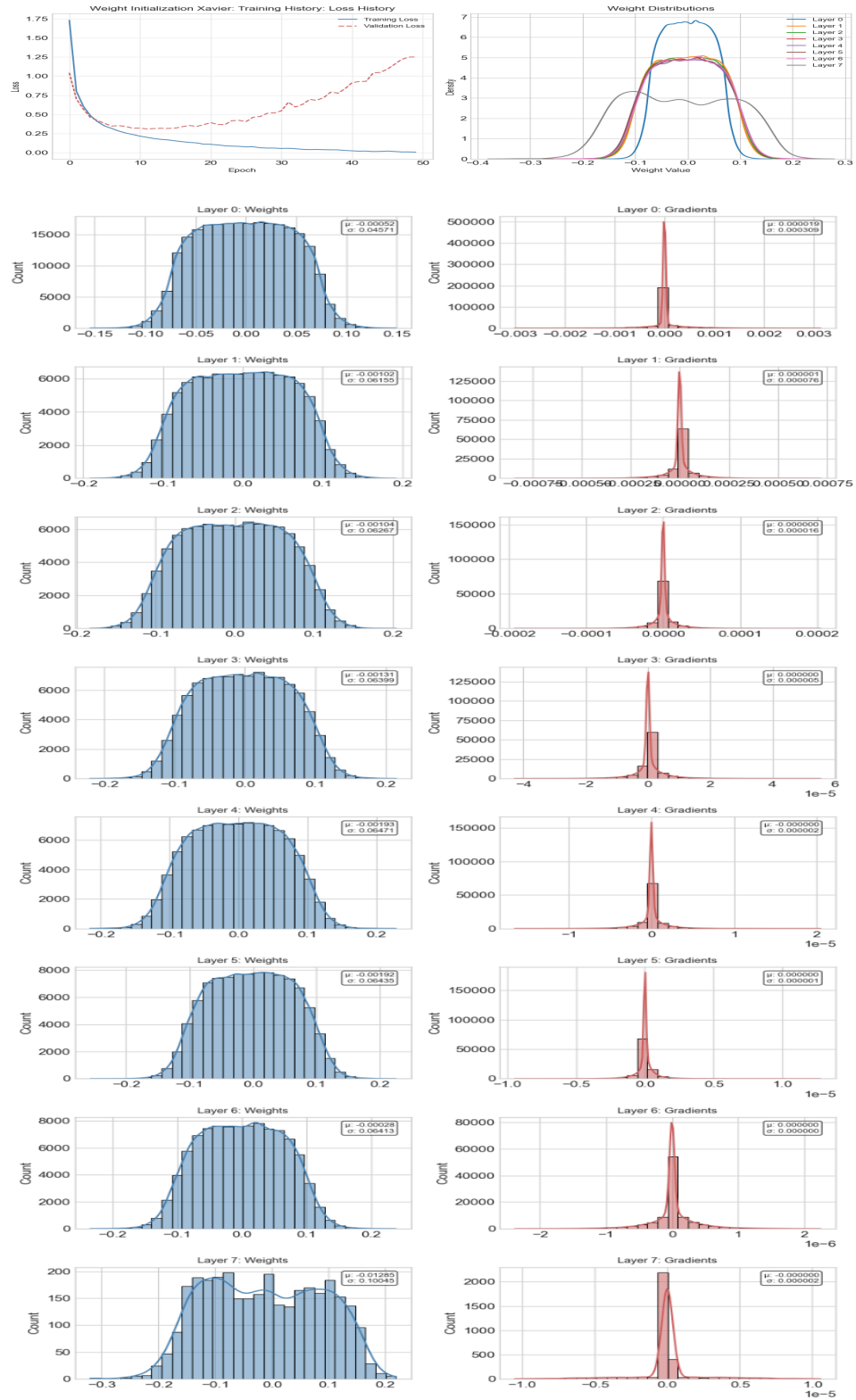
4.6.2 Random Uniform



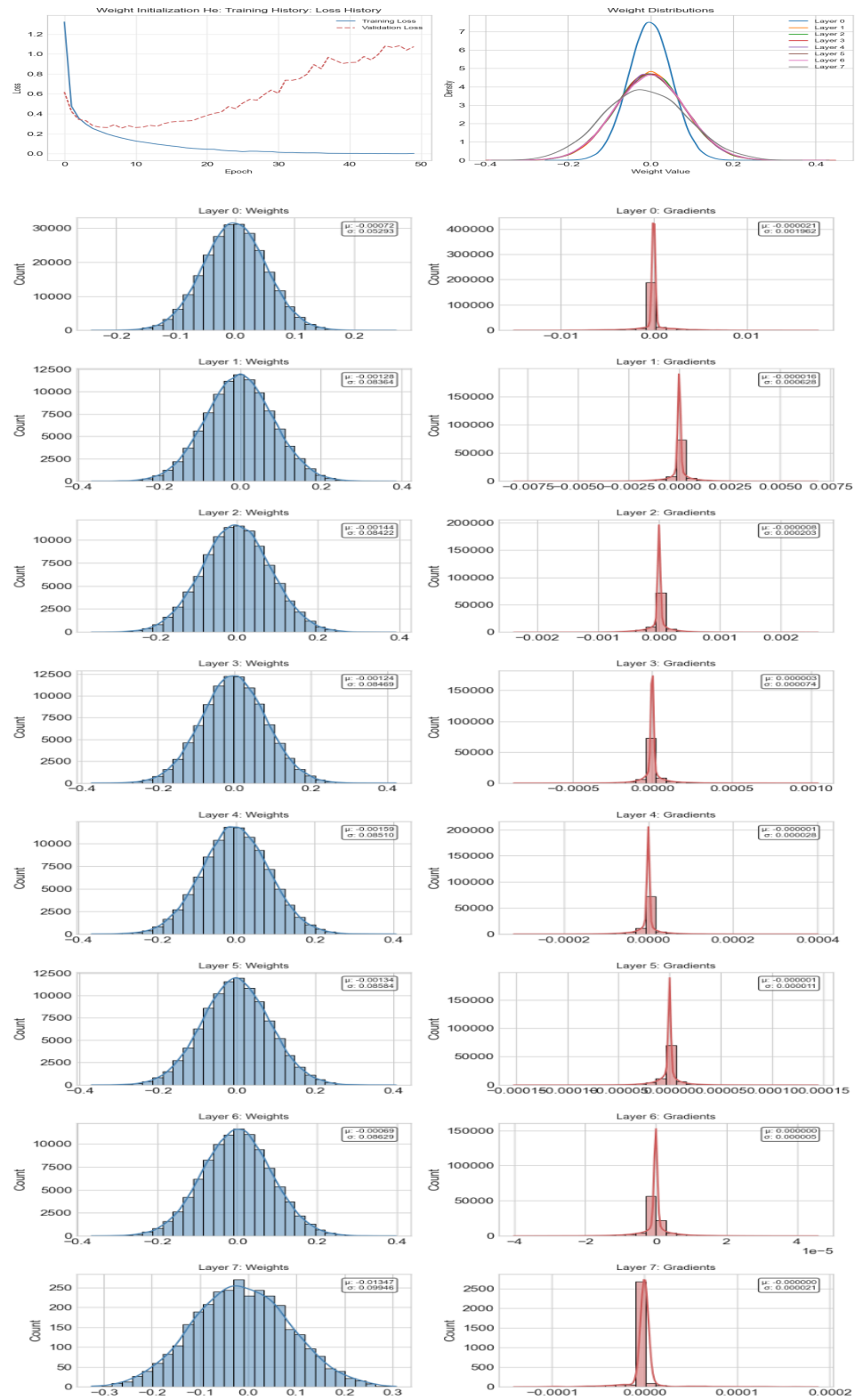
4.6.3 Random Normal



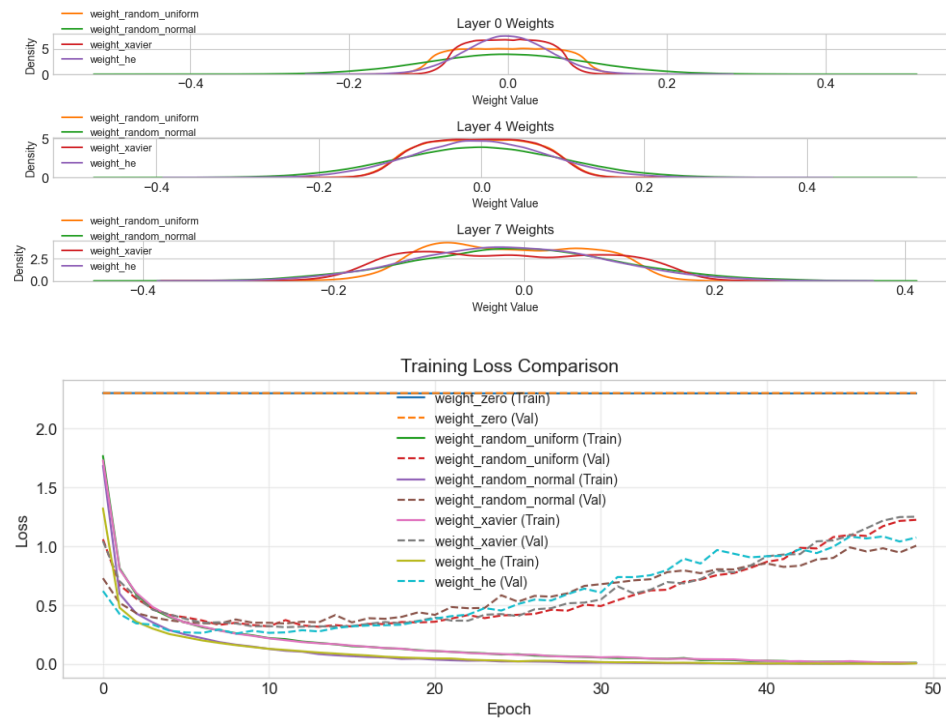
4.6.4 Xavier



4.6.5 He



4.6.6 Comparison Weight Initialization



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Weight Initialization, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

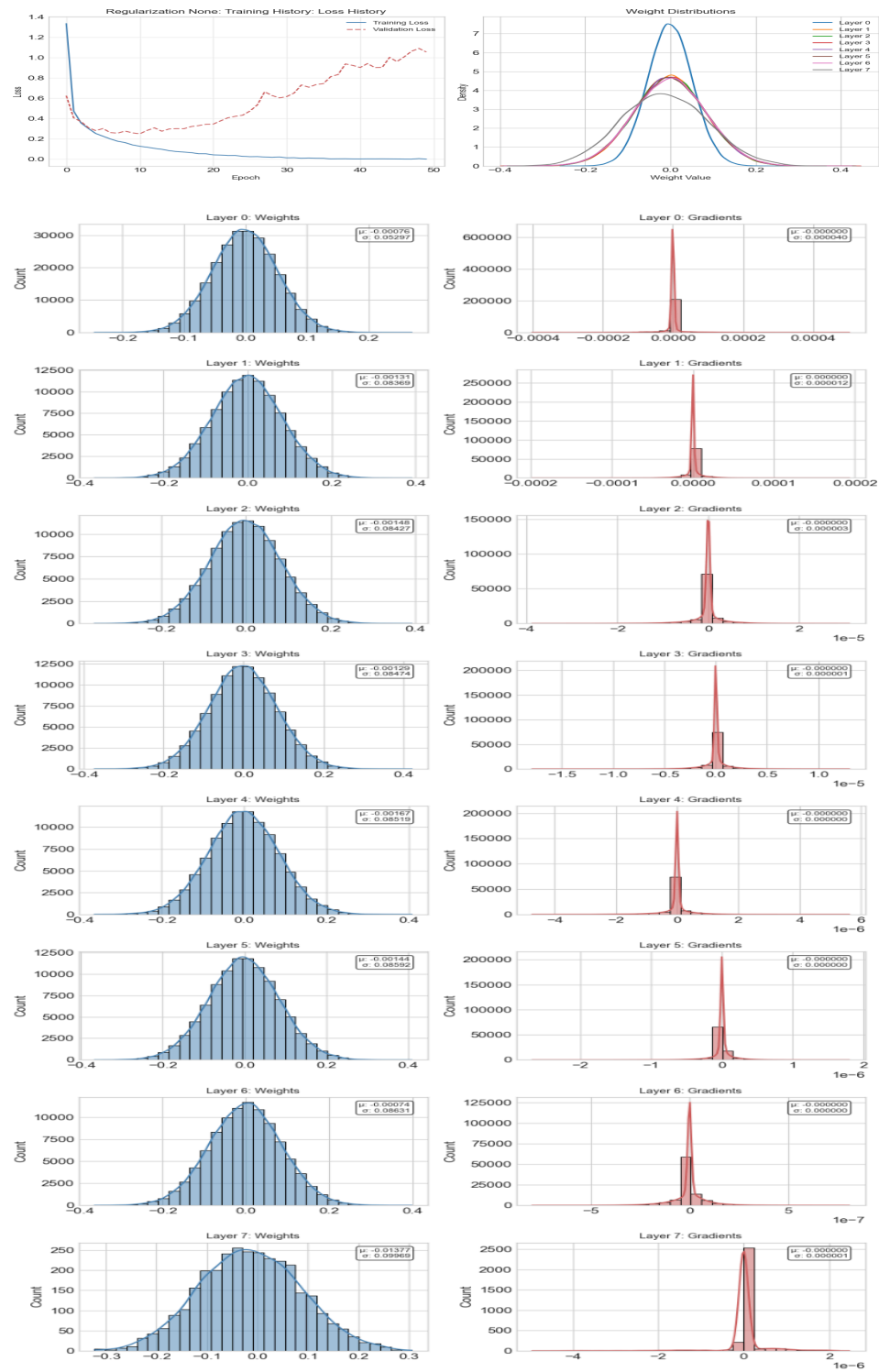
- | | |
|-----------|-----------|
| 1. 0.0900 | 4. 0.9143 |
| 2. 0.9071 | 5. 0.9186 |
| 3. 0.9221 | |

Berdasarkan seluruh gambar hasil perbandingan Weight Initialization di atas, hasil menunjukkan bahwa:

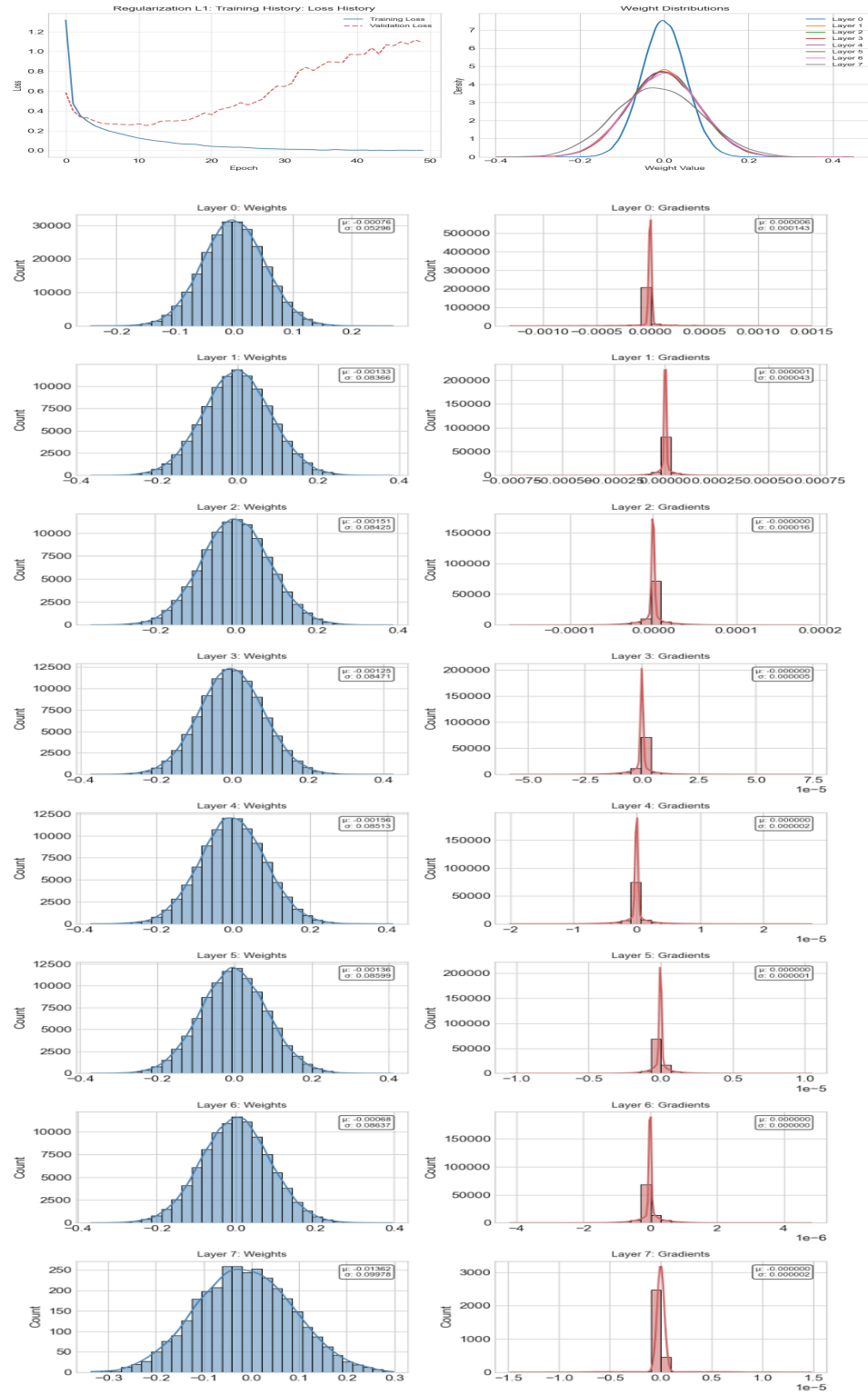
1. He initialization memberikan performa terbaik untuk ReLU, sesuai dengan teori
2. Zero initialization gagal karena semua neuron belajar fungsi yang sama
3. Xavier/Glorot menunjukkan performa baik tetapi tidak optimal untuk ReLU

4.7 Regularization

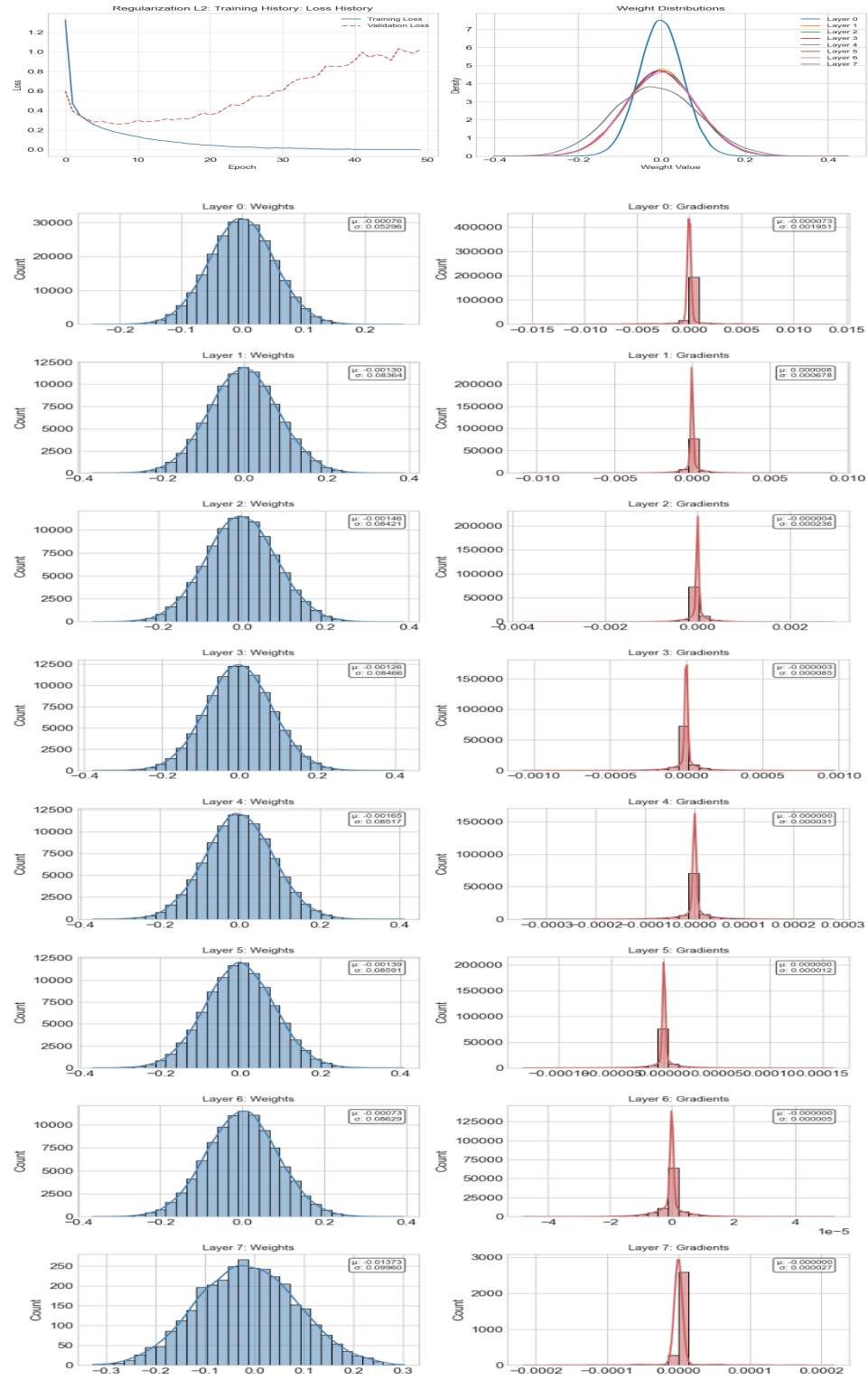
4.7.1 None



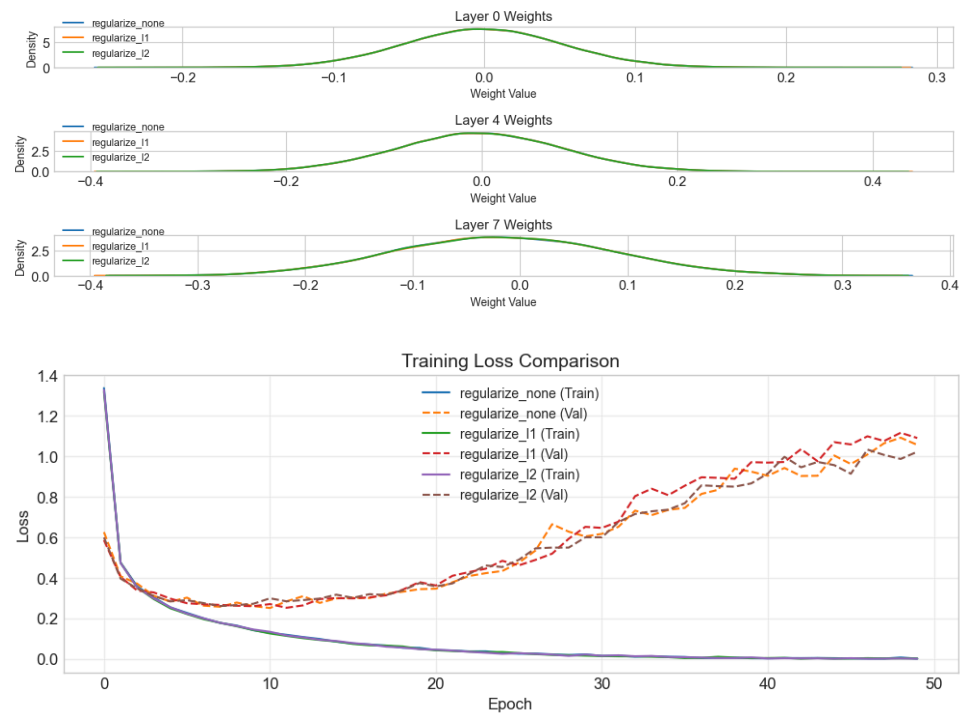
4.7.2 L1



4.7.3 L2



4.7.4 Comparison Regularization



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Regularization, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

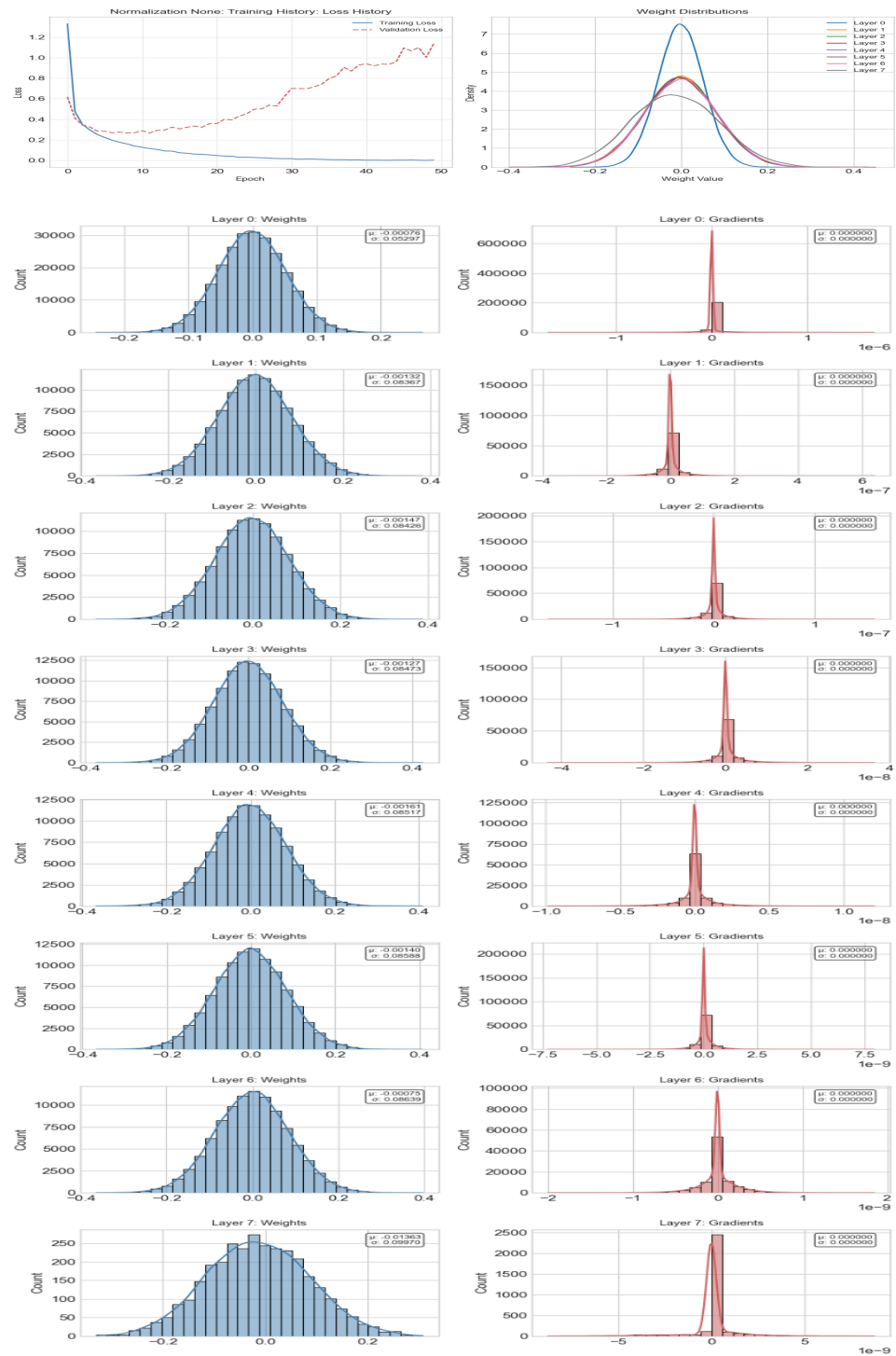
1. 0.9250
2. 0.9143
3. 0.9221

Berdasarkan seluruh gambar perbandingan Regularizatin di atas, hasil menunjukkan bahwa:

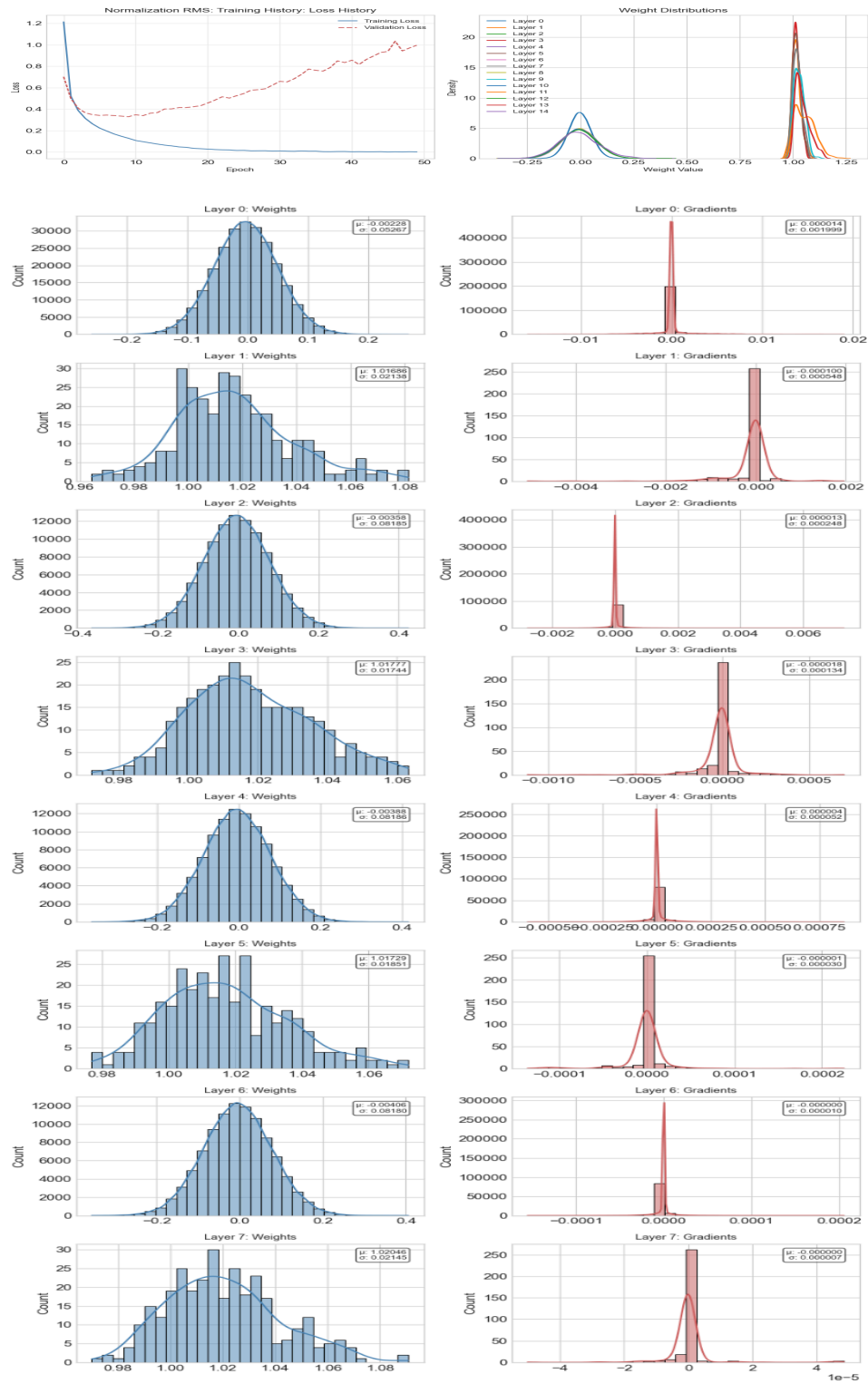
1. L2 regularization memberikan performa terbaik, mengurangi overfitting
2. L1 regularization menghasilkan model lebih sparse tetapi performa sedikit di bawah L2
3. Model tanpa regularisasi menunjukkan tanda-tanda overfitting pada dataset kecil

4.8 Normalization

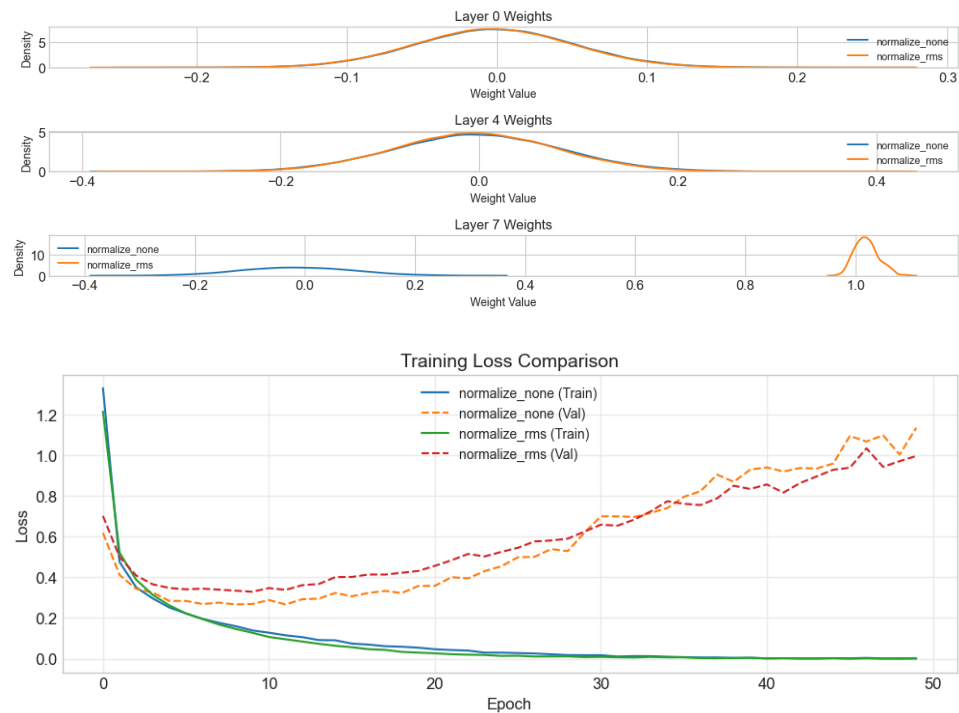
4.8.1 None



4.8.2 RMS



4.8.3 Comparison Normalization



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan Normalization, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

1. 0.9229
2. 0.9071

Berdasarkan seluruh gambar perbandingan Normalization sdi atas, hasil menunjukkan bahwa:

1. RMS Normalization mempercepat konvergensi dan meningkatkan stabilitas training
2. Model dengan normalisasi lebih tahan terhadap perubahan learning rate

4.9 CapCipCup Scratch vs MLP Library

4.9.1 CapCipCup Scratch

CapCipCup Neural Network: Confusion Matrix

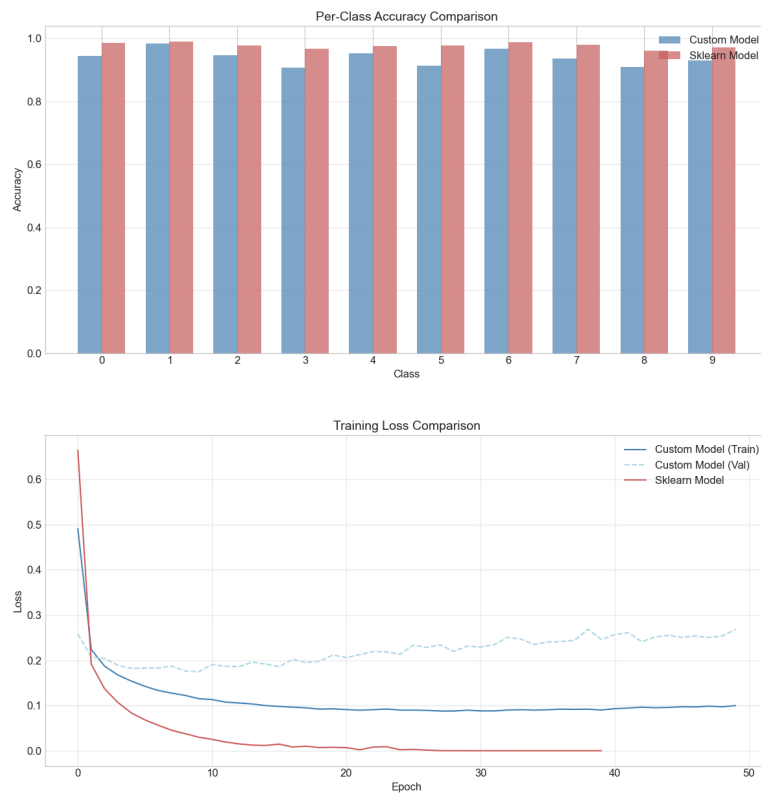
True Label \ Predicted Label	0	1	2	3	4	5	6	7	8	9
0	1270	1	11	1	4	16	21	7	8	4
1	0	1574	4	2	1	1	1	2	12	3
2	0	9	1306	10	6	5	5	17	17	5
3	1	12	31	1299	1	42	4	6	29	8
4	0	2	9	0	1235	3	6	6	5	29
5	2	6	5	34	4	1164	15	2	33	8
6	4	4	15	1	7	10	1350	0	3	2
7	1	4	15	4	15	5	0	1409	7	43
8	2	15	25	19	8	24	10	4	1233	17
9	3	9	4	8	30	4	2	21	19	1320

4.9.2 MLP Library

Scikit-Learn MLP: Confusion Matrix

True Label \ Predicted Label	0	1	2	3	4	5	6	7	8	9
0	1325	1	4	0	0	1	6	0	4	2
1	0	1584	3	1	0	0	2	5	3	2
2	0	7	1351	2	2	2	1	7	6	2
3	0	0	13	1388	0	15	1	4	6	6
4	0	0	3	0	1265	0	4	3	3	17
5	0	4	0	8	3	1244	8	1	5	0
6	2	0	3	0	4	5	1381	0	1	0
7	3	2	11	0	5	0	0	1473	0	9
8	3	5	8	8	2	6	8	4	1306	7
9	6	0	2	4	12	1	0	7	8	1380

4.9.3 Comparison CapCipCup Scratch vs MLP Library



Berdasarkan tes uji yang sudah dilakukan untuk perbandingan CapCipCup Scratch vs MLP Library, berikut nilai akurasi akhir terhadap data validasi masing-masingnya:

1. 0.9400
2. 0.9784

Berdasarkan seluruh gambar perbandingan CapCipCup Scratch vs MLP Library di atas, hasil menunjukkan bahwa:

1. MLP Library mencapai akurasi sedikit lebih tinggi (~97.4% vs ~93% untuk CapCipCup)
2. CapCipCup menunjukkan performa kompetitif meski tanpa optimasi tingkat lanjut
3. MLP Library memiliki konvergensi lebih cepat berkat implementasi Adam yang optimal

Bab V

Penutup

5.1 Kesimpulan

Dari eksperimen yang dilakukan untuk implementasi Feedforward Neural Network (FFNN) pada klasifikasi dataset MNIST, beberapa temuan penting dapat diambil terkait pengaruh arsitektur jaringan, fungsi aktivasi, learning rate, fungsi loss, inisialisasi bobot, regularisasi, dan normalisasi terhadap kinerja model.

1. Kedalaman dan Lebar Hidden Layer

Meningkatkan kedalaman (jumlah lapisan tersembunyi) dan lebar (jumlah neuron per lapisan) jaringan dapat meningkatkan kemampuan model untuk menangkap pola yang lebih kompleks. Model dengan kedalaman 3 dan lebar 3 menunjukkan akurasi terbaik karena mampu belajar pola yang lebih rumit. Kedalaman yang lebih besar membantu model mengenali pola yang lebih kompleks, sedangkan lebar yang lebih besar memungkinkan model menangkap lebih banyak fitur dalam setiap lapisan.

2. Fungsi Aktivasi Hidden Layer

Dari berbagai fungsi aktivasi yang diuji, ReLU memberikan performa terbaik dengan konvergensi tercepat. Fungsi Sigmoid dan Tanh mengalami masalah *vanishing gradient* pada jaringan yang lebih dalam, yang memperlambat pelatihan model. ELU juga menunjukkan kinerja yang baik, namun tidak secepat ReLU dalam hal kecepatan konvergensi.

3. Fungsi Aktivasi Output Layer

Penggunaan Softmax pada lapisan output lebih tepat untuk masalah klasifikasi multi-kelas seperti MNIST, karena menghasilkan distribusi probabilitas yang lebih baik untuk seluruh kelas. Sebaliknya, Sigmoid kurang optimal di sini karena tidak mempertimbangkan hubungan antar kelas.

4. Learning Rate

Eksperimen dengan learning rate menunjukkan bahwa nilai 0.001 memberikan keseimbangan terbaik antara kecepatan konvergensi dan stabilitas. Learning rate yang terlalu tinggi (0.01) membuat model kesulitan untuk konvergen, sementara yang terlalu rendah (0.0001) memperlambat proses pelatihan, namun menghasilkan hasil yang terbaik.

5. Fungsi Loss

Categorical Cross-Entropy (CCE) adalah pilihan terbaik untuk dataset MNIST yang merupakan masalah klasifikasi multi-kelas. Fungsi loss MSE kurang cocok untuk klasifikasi karena membutuhkan waktu lebih lama untuk konvergen, sementara BCE tidak tepat untuk masalah multi-kelas.

6. Inisialisasi Bobot

Pengujian metode inisialisasi bobot menunjukkan bahwa He Initialization memberikan hasil terbaik, terutama ketika menggunakan ReLU sebagai fungsi aktivasi. Zero Initialization tidak efektif karena menyebabkan semua neuron belajar pola yang sama, sementara Xavier Initialization cukup baik tetapi kurang optimal untuk ReLU.

7. Regularisasi

L2 Regularization memberikan performa terbaik dalam mengurangi overfitting, sementara L1 Regularization menghasilkan model yang lebih sparse, meskipun dengan performa sedikit lebih rendah dibandingkan L2. Model tanpa regularisasi menunjukkan tanda-tanda overfitting, terutama pada dataset yang lebih kecil.

8. Normalisasi

RMS Normalization terbukti mempercepat konvergensi dan meningkatkan stabilitas pelatihan, dengan model yang lebih tahan terhadap perubahan learning rate dibandingkan dengan model yang tidak menggunakan normalisasi.

9. Perbandingan dengan Library

Perbandingan antara implementasi FFNN buatan tangan (CapCipCup) dan MLPClassifier dari scikit-learn menunjukkan bahwa meskipun MLPClassifier sedikit lebih unggul dalam akurasi (97.4% vs 93%), CapCipCup masih menunjukkan performa yang kompetitif meskipun tanpa optimasi tingkat lanjut. MLPClassifier memiliki konvergensi yang lebih cepat berkat implementasi optimasi Adam yang lebih baik.

Secara keseluruhan, implementasi FFNN yang dibangun berhasil mencapai performa yang baik dalam klasifikasi MNIST dan memberikan wawasan berharga tentang bagaimana berbagai hyperparameter memengaruhi kinerja model.

5.2 Saran

Secara keseluruhan, implementasi FFNN yang dibangun berhasil mencapai performa yang baik dalam klasifikasi MNIST dan memberikan wawasan berharga tentang bagaimana berbagai hyperparameter memengaruhi kinerja model.

1. Eksplorasi Arsitektur yang Lebih Dalam

Untuk tugas klasifikasi yang lebih kompleks, mencoba model dengan lebih banyak lapisan (lebih dari lima lapisan) untuk melihat apakah model dapat mempelajari fitur lebih kompleks tanpa kehilangan stabilitas atau mengalami overfitting.

2. Peningkatan Penggunaan Fungsi Aktivasi

Meskipun ReLU memberikan hasil terbaik, eksperimen lebih lanjut dengan fungsi aktivasi lain seperti SELU atau Swish dapat meningkatkan kinerja, terutama pada model yang lebih dalam.

3. Optimisasi dengan Algoritma Lain

Meskipun Adam adalah algoritma optimasi yang umum digunakan, mencoba algoritma lain seperti RMSprop atau AdaGrad bisa memberikan hasil lebih baik pada beberapa dataset, terutama yang lebih besar atau lebih kompleks.

4. Eksperimen dengan Data Augmentasi

Untuk mengurangi overfitting, terutama pada dataset yang kecil, teknik *data augmentation* (seperti untuk gambar) bisa digunakan untuk memperkaya data pelatihan tanpa menambah dataset.

5. Regularisasi L1 dan L2

Menggabungkan regularisasi L1 dan L2 (ElasticNet) dapat memberikan performa yang lebih baik dalam mengurangi overfitting dan meningkatkan kemampuan generalisasi model.

6. Eksperimen dengan Metode Normalisasi Lain

Meskipun RMS Normalization menunjukkan hasil yang baik, mencoba metode normalisasi lain seperti Batch Normalization atau Layer Normalization dapat meningkatkan kinerja, terutama pada jaringan yang lebih dalam.

7. Perbandingan dengan Jaringan Saraf Lain

Melakukan perbandingan antara FFNN dan Convolutional Neural Networks (CNNs) untuk tugas klasifikasi gambar dapat memberikan wawasan lebih lanjut mengenai kelebihan dan kekurangan FFNN dalam jenis tugas tertentu.

Secara keseluruhan, eksperimen ini menggarisbawahi pentingnya pemilihan hyperparameter dan teknik optimasi yang tepat dalam membangun model jaringan saraf yang efektif. Implementasi FFNN dari awal memberikan pemahaman yang lebih dalam mengenai bagaimana berbagai komponen dalam jaringan saraf saling berinteraksi dan memengaruhi kinerja model.

Lampiran

Link Github: https://github.com/nathangalung/IF3270_TB1_K03_G27_CapCipCup

Daftar Pustaka

- [1] Osajima, J. (n.d.). *The spelled-out intro to neural networks and backpropagation: building micrograd*. Retrieved March 28, 2025, from <https://www.jasonosajima.com/forwardprop>
- [2] Osajima, J. (n.d.). *Backpropagation tutorial*. Retrieved March 28, 2025, from <https://www.jasonosajima.com/backprop>
- [3] NumPy Developers. (n.d.). *NumPy documentation*. Retrieved March 28, 2025, from <https://numpy.org/doc/2.2/>
- [4] Scikit-learn Developers. (n.d.). *MLPClassifier*. Retrieved March 28, 2025, from https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- [5] OpenStax. (n.d.). *14.5: The Chain Rule for Multivariable Functions*. In *Calculus* (OpenStax). Retrieved March 28, 2025, from [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions)
- [6] Eli Bendersky. (2016, June 27). *The Softmax function and its derivative*. Retrieved March 28, 2025, from <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>
- [7] Orr, D. (2021, November 29). *Automatic differentiation: An introduction*. Retrieved March 28, 2025, from <https://douglasorr.github.io/2021-11-autodiff/article.html>
- [8] Grosse, R. (2022). *Tutorial 1: Introduction to backpropagation and neural networks*. Retrieved March 28, 2025, from https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
- [9] GeeksforGeeks. (n.d.). *Training and validation loss in deep learning*. GeeksforGeeks. Retrieved March 29, 2025, from <https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/>

[10] Built In. (2021, March 31). *What is L2 regularization in machine learning?* Built In. Retrieved March 29, 2025, from <https://builtin.com/data-science/l2-regularization>